

CSE 4113 - Internet Programming Lab**Project Report****BusGo***Team DU_VibeCoders***Submitted By**

Team Member	Student ID / Roll	Email
Farzana Tasnim	Roll 14	farzana-2021311190@cs.du.ac.bd
Md. Tauseef - Ur - Rahman	Roll 24	mdtauseef.rahmang01@gmail.com
Amina Islam	Roll 36	aminaislam1912@gmail.com
Tamzid Bin Tariq	Roll 48	tamzid-2021511224@cs.du.ac.bd

28th Batch

Department of Computer Science & Engineering
University of Dhaka

Submitted On

26 July 2026

Project Links

Primary project resources and deployed interfaces are listed below.

Item	URL
Public Git Repository	https://github.com/TAUSEEF-01/Jaabo
Deployed Application URL	https://busgo.farefin.com
API Docs (Swagger) - public link	https://busgo.farefin.com/api/auth/docs
Demo Video	https://www.youtube.com/watch?v=RUT8gTXIzZI

Table of Contents

Project Links	2
Table of Contents	3
1. Introduction.....	5
1.1 Problem Definition & Context	5
1.2 Target Users & Use Cases	5
1.3 Core Features	6
1.4 Tech Stack Overview	6
2. System Architecture	8
2.1 High-Level Architecture Diagram	8
2.2 Frontend Architecture.....	10
2.3 Backend Architecture	11
2.4 Database Design	13
3. API Documentation.....	16
3.1 API Design Overview	16
3.2 Endpoint Reference	17
3.3 Swagger / Postman Collection Link	20
3.4 Sample Request & Response	21
4. Authentication & Security.....	23
4.1 Authentication Strategy	23
4.2 Authorization & Role Management	24
4.3 Security Measures	25
5. Testing & Quality Assurance	26
5.1 Testing Strategy	26
5.2 Test Coverage Summary	27
5.3 Bug Tracking & Resolution Log.....	27
5.4 Sample Test Cases.....	29
6. Build, Deployment & Operations.....	34
6.1 Build and Deployment Workflow	34
6.2 Environments	34
6.3 Deployment Steps.....	35
6.4 Hosting Platform & Live URL	36
6.5 Monitoring & Logging	36
7. Repository & Documentation Quality	37
7.1 Branching Strategy & Commit Conventions.....	37
7.3 Code Organization / Folder Structure	38
7.4 Local Setup Instructions.....	39
8. Evaluation & Reflection	40
8.1 Performance and Reliability Outcomes.....	40
8.2 Challenges & Solutions.....	40
8.3 Project Outcomes	43
8.4 Lessons Learned	43
8.5 Individual Responsibility	43
Appendix - Screenshots / UI Walkthrough.....	45
A. Passenger Journey (Web)	45
B. Operator Portal (Web)	54
C. Admin Portal (Web)	60
D. Mobile Applications Overview.....	65

E. Passenger Mobile Application	66
F. Operator Mobile Application.....	72

1. Introduction

1.1 Problem Definition & Context

Intercity bus travel is the backbone of long-distance transport in Bangladesh, but the ticketing experience around it is still largely offline and fragmented. A passenger who wants to travel from Dhaka to Sylhet typically has to:

- physically visit a counter, or call an operator, to learn what buses run and at what time;
- trust a hand-drawn seat chart that may already be stale by the time the money changes hands;
- pay in cash with no verifiable receipt, and no reliable path to a refund if the trip is cancelled;
- and, if no direct bus exists for their origin-destination pair, plan the *connecting* journey entirely by themselves - buy one ticket to an intermediate city, get off, and hope a seat is still free on the onward bus.

On the supply side, operators run their inventory on paper or spreadsheets. They cannot see live occupancy across their fleet, cannot re-market seats that are still empty a few hours before departure, and have no analytics on which routes actually earn.

BusGo is a full-stack, microservices-based bus ticketing platform built to close both gaps. It gives passengers a single place to search (including multi-leg *transit* journeys that no single operator sells end-to-end), pick an exact seat from a live seat map, pay through a sandbox mobile-financial-service workflow, and receive a QR e-ticket. It gives operators a portal to manage buses, routes, trips, promotions and curated connecting routes, plus targeted re-marketing of unsold seats. It gives platform administrators oversight of users, operators, transactions and system-wide notices.

The engineering problem the project actually solves is harder than the product description suggests, and it is the reason the system is decomposed into services rather than built as one application:

1. **Seat inventory is a contended resource.** Two users clicking the same seat at the same instant must not both succeed. This demands an atomic, distributed lock with an expiry, not a database `UPDATE` guarded by application logic.
2. **A booking is a distributed transaction.** Creating a booking touches operator, inventory, deals, payment, bank, ticket and notification services. There is no single database transaction spanning them, so the flow is modelled as a **saga with compensation** - especially for multi-leg journeys, where locking leg 2 may fail after leg 1 is already held.
3. **Money must never be taken on client-supplied numbers.** Fares, operators, boarding points and discounts sent by the browser are re-validated server-side against the operator service before a payment is authorised.

1.2 Target Users & Use Cases

The platform has three first-class roles (CUSTOMER, OPERATOR, ADMIN) enforced end-to-end in the JWT and in every service.

Passenger (CUSTOMER)

#	Use case	Where it lives
UC-01	Search direct buses between two cities on a date, with filters	search-service -> Elasticsearch
UC-02	Search connecting (transit) journeys when no direct bus exists	transit-service graph search
UC-03	Browse routes, active deals and flash sales without logging in (guest browsing)	web + mobile
UC-04	View a live seat map and select 1-4 seats	inventory-service
UC-05	Enter per-seat passenger details and apply a promo code	booking-service, deals-service
UC-06	Pay via bKash / Nagad / linked bank account within a 10-minute seat hold	payment-service, bank-service
UC-07	Receive a QR + PDF e-ticket (one per leg for a transit journey)	ticket-service
UC-08	View booking history grouped by journey, cancel, and get a refund	booking-service, cancellation-service
UC-09	Receive in-app notifications (confirmation, reminder, delay, refund, offers)	notification-service
UC-10	Manage profile, wallet balance and travel/payment summary	auth-service, bank-service

Bus Operator (OPERATOR)

#	Use case	Where it lives
UC-11	Register an operator profile and manage it	operator-service
UC-12	CRUD buses (registration, type, seat layout, amenities)	operator-service
UC-13	CRUD routes (origin/destination, distance, boarding & dropping points)	operator-service
UC-14	Schedule, update and cancel trips	operator-service
UC-15	Publish curated transit routes (e.g. Dhaka->Sylhet via Cumilla) with a combined discount	operator-service (transit_routes)
UC-16	View bookings and passenger manifests per trip	booking-service
UC-17	Issue operator-scoped promo codes and flash sales	deals-service
UC-18	Fill empty seats - find past passengers on the same corridor and push a targeted offer	booking-service (travel_records)
UC-19	Send notifications to interested/past passengers	notification-service

#	Use case	Where it lives
UC-20	See revenue and occupancy analytics	operator portal + admin/booking data

Platform Administrator (ADMIN)

#	Use case	Where it lives
UC-21	Dashboard: users, operators, routes, trips, transaction totals	admin-service
UC-22	Promote/demote user roles	admin-service
UC-23	Publish platform-wide notices and broadcast notifications	admin-service, notification-service
UC-24	Inspect the immutable audit trail of every booking/payment event	audit-service
UC-25	Adjust simulated bank balances for demonstrations	bank-service

1.3 Core Features

Booking & travel

- **City-aware search** over an Elasticsearch index of trips, with live seat availability merged in from `inventory-service` at query time.
- **Multi-leg transit journeys** - a dedicated `transit-service` performs a bounded graph search over the trip index (`MIN_TRANSFER_MINUTES=30`, `MAX_TRANSFER_WAIT_HOURS=6`, `MAX_LEGS=3`, top 5 itineraries). Operator-curated itineraries are badged and ranked above auto-discovered ones.
- **Live seat map with distributed locking** - a Redis `SET NX EX` lock per (`trip`, `seat`) with a 600-second TTL, overlaid on the persistent `seat_inventory` table, so exactly one of N concurrent users can win a contended seat.
- **All-or-nothing journey booking (saga)** - every leg of a transit journey is locked; if any leg fails, all previously locked legs are released as compensation, and the user is told no seats were held.
- **Server-side fare authority** - the booking service re-fetches the trip from `operator-service` and rejects the request if operator, status, boarding/dropping point or fare disagree with what the client sent.
- **Idempotent booking creation** - every create carries an `idempotency_key` that is both a unique DB column and a Redis cache key, so a retried request returns the original booking instead of double-booking.
- **10-minute seat hold** with a countdown in the UI and a background scheduler that expires stale holds and publishes `seat.lock.expired`.

Payments

- Sandbox **bKash / Nagad / bank** gateways with a redirect+callback flow.
- A complete sandbox ledger in `bank-service`: accounts, PIN verification, balance checks, debits/credits and a transaction history - payments actually move money between rows rather than merely flipping a status.
- **Fraud checks** in `payment-service`: amount-vs-booking mismatch, a cap of 3 payment attempts per user per trip (which then definitively releases the seats), and idempotent retry handling that returns the original completed payment instead of debiting twice.
- **Refunds** with a policy-driven amount (80 % outside the 1-hour pre-departure window) and a `refund.initiated` event.

Tickets & notifications

- QR-coded, PDF-rendered e-tickets stored on a Docker volume; one ticket per leg of a journey.
- QR validation endpoint for counter/driver scanning.
- A notification domain with **26 typed notification kinds** scoped by role (customer / operator / admin), an in-app inbox with unread counts, plus mocked e-mail, SMS, push and WhatsApp channels driven by Kafka events.

Operator & admin tooling

- Full CRUD for operators, buses, routes, trips and curated transit routes.
- Operator-scoped promo codes and flash sales.
- **Fill Empty Seats**: `travel_records` remembers which corridors a user has travelled, so an operator can find and notify exactly the people likely to want a seat that is still empty.
- Admin dashboard, role management, notices, transaction summaries and an audit log.

Platform engineering (deliberately part of the product)

- **Kong API Gateway** with declarative config, upstreams, ring-balancer load balancing across replicas, active `/health` polling and passive circuit-breaking, plus request retries.
- **Kafka** event backbone with 9 published topics and ~30 consumed event types.
- **Prometheus + Grafana + Loki + Promtail** observability, with a shared instrumentation module giving every service `/metrics`, structured JSON logs and an X-Request-ID correlation ID that is propagated across inter-service HTTP calls.
- **Resilient inter-service HTTP client** with split timeouts, exponential-backoff retries and a per-host circuit breaker.
- **Three clients** against one API: a React web app, an Expo/React Native passenger app, and a separate Expo operator app.

1.4 Tech Stack Overview

Frontend

Choice	Why
--------	-----

Choice	Why
React 18 + TypeScript + Vite (web)	Fast HMR dev loop and a typed component model; Vite's build produces a single static bundle that Nginx can serve.
Tailwind CSS 4	Utility-first styling kept the three portals (public, operator, admin) visually consistent without a component-library dependency.
React Router v7	Declarative nested routing with a ProtectedRoute wrapper that enforces role gates in the SPA.
React Native 0.81 + Expo SDK 54 (mobile & operator apps)	One TypeScript codebase per app, distributable as an APK/Expo Go bundle, reusing the same Kong API.
React Navigation v7	Native stack + bottom tabs for the two mobile apps.
Leaflet	Map-based boarding/dropping point selection.
react-qr-code, jspdf, html2canvas	Client-side ticket rendering and PDF export.

State Management Solution

Layer	Library	Role
Client/session state	Zustand 5 (persist middleware)	authStore holds the user, access and refresh tokens; persisted to localStorage with a cookie fallback, and synchronised across browser tabs via a storage event listener. notificationStore holds the inbox, unread count and the 30-second polling timer.
Server/cache state	TanStack React Query 5	Fetching, caching and background refresh of trips, seats, bookings and deals.
Form state	React Hook Form + Zod (via @hookform/resolvers)	Typed, schema-validated forms (registration, passenger details, operator CRUD).
Mobile	React Context (src/store/auth.tsx, src/store/notifications.tsx) + expo-secure-store / AsyncStorage	Same session model, tokens kept in secure storage.

Redux was deliberately not used: the app's genuinely global state is small (session + notifications), and Zustand delivered that with far less ceremony, while React Query already owns everything server-derived.

Backend

Choice	Why
Python 3.12 + FastAPI	Async I/O suits a service mesh dominated by network waits; Pydantic gives request/response validation and auto-generated OpenAPI docs for free.
SQLAlchemy 2 (async, asyncpg)	Typed ORM models with an async engine; a few read-heavy services use the sync engine where async bought nothing.
Kong Gateway 3.4 (DB-less)	Single public endpoint, path-based routing to 14 services, load balancing, health checks and retries - declared in one version-controlled kong.yml.
Apache Kafka 7.5 + Zookeeper	Asynchronous, decoupled cross-service events (booking -> payment -> ticket -> notification -> audit) so a slow consumer never blocks a user request.
Redis 7	Atomic seat locks (SET NX EX), idempotency cache, and promo-usage tracking.
Elasticsearch 8.11	Full-text and structured trip search; also the graph substrate for transit itinerary discovery.
python-jose + bcrypt	HS256 JWT issuance/verification and password hashing.
APScheduler	Seat-hold expiry and scheduled departure reminders.
httpx + tenacity	The shared resilient client (retries, backoff, circuit breaker).

Database

Choice	Why
PostgreSQL 15	Relational integrity for money-touching data, with JSONB where the shape is genuinely document-like (seat layouts, passenger details, amenities, event payloads).
Database-per-service	13 logical databases (auth_db, booking_db, inventory_db, ...) so services own their schema and cannot reach into each other's tables. Locally these are separate databases in one cluster (init-multiple-databases.sh); in the deployed environment they are schemas inside one Supabase Postgres instance.
Supabase (managed Postgres + PgBouncer)	Managed, always-on Postgres for the deployed stack, reached through the transaction-mode pooler on port 6543.

Hosting & Third-Party Services / Cloud Infrastructure

Choice	Why
Azure Linux VM (Ubuntu, 135.171.216.245)	Single host running the entire Docker Compose stack - 14 services plus Kong, Redis, Kafka, Elasticsearch and the observability stack.
Nginx (system) + Certbot/Let's Encrypt	TLS termination for busgo.farefin.com, reverse-proxying / to the frontend container (:8083) and /api/ to Kong (:8085).
Docker & Docker Compose	Reproducible multi-service topology; source directories are bind-mounted so code changes need only a container restart.
Supabase	(a) managed PostgreSQL , (b) Supabase Auth as the Google OAuth identity provider.
Google OAuth 2.0	The sole interactive sign-in method on web and mobile.
Prometheus / Grafana / Loki / Promtail	Self-hosted metrics, dashboards and log aggregation, all in the same Compose file.
Expo / EAS	Mobile build and distribution (Expo-Go-57.0.2.apk is bundled for demo convenience).

Scale of the codebase

Area	Approx. lines
Python (14 services + shared modules)	~11,200
TypeScript / TSX (web + mobile + operator apps)	~26,800
Infrastructure as code (Compose, Kong, Prometheus, Loki, Grafana, SQL)	~1,300
Total commits on main	163

2. System Architecture

2.1 High-Level Architecture Diagram

2.1.1 Component / deployment view

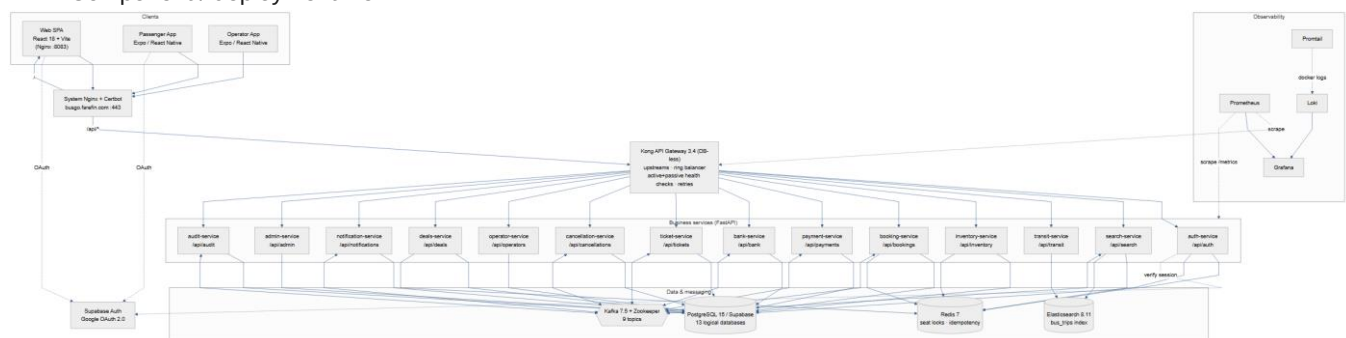


Figure 1. High-level component and deployment architecture.

2.1.2 Booking flow - sequence diagram (single-leg, happy path)

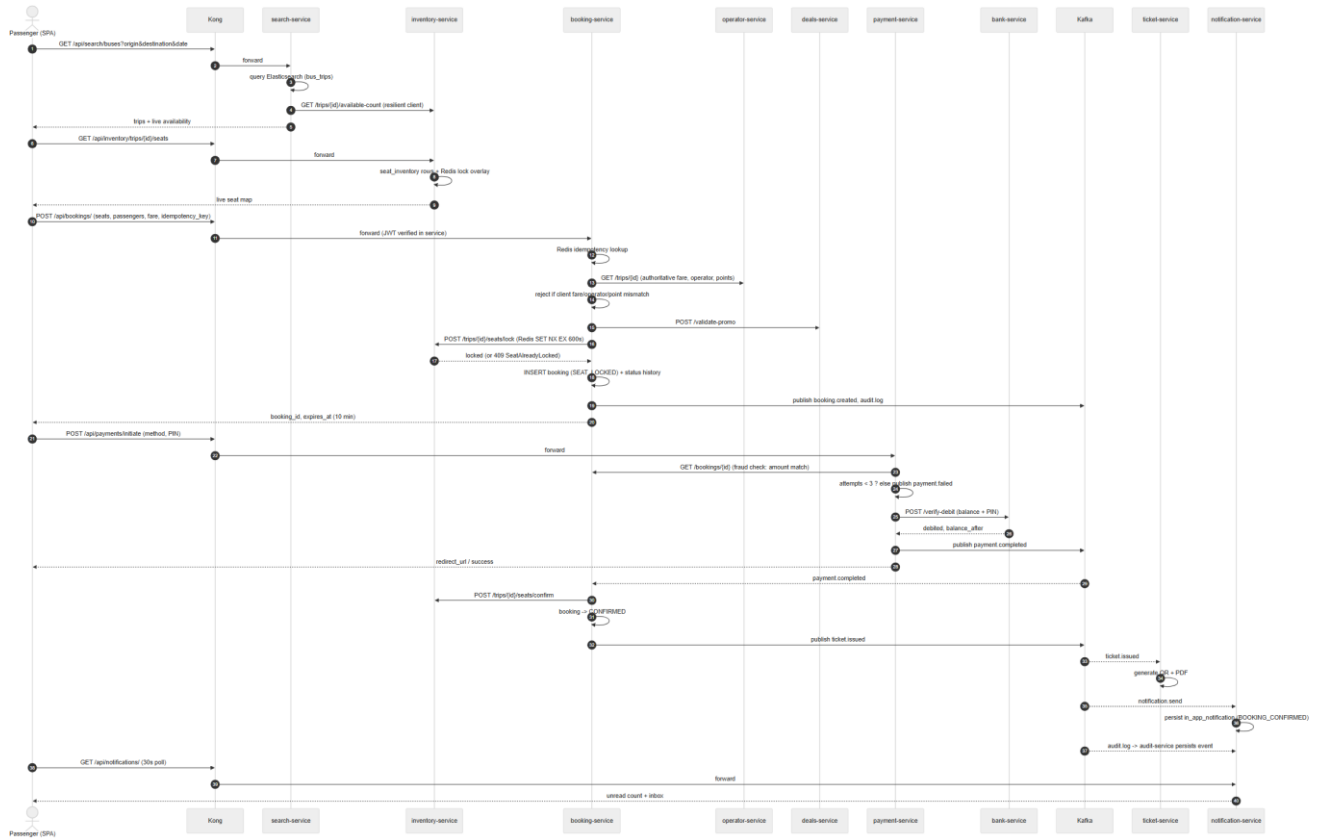


Figure 2. Single-leg booking sequence (happy path).

2.1.3 Transit journey saga - compensation flow

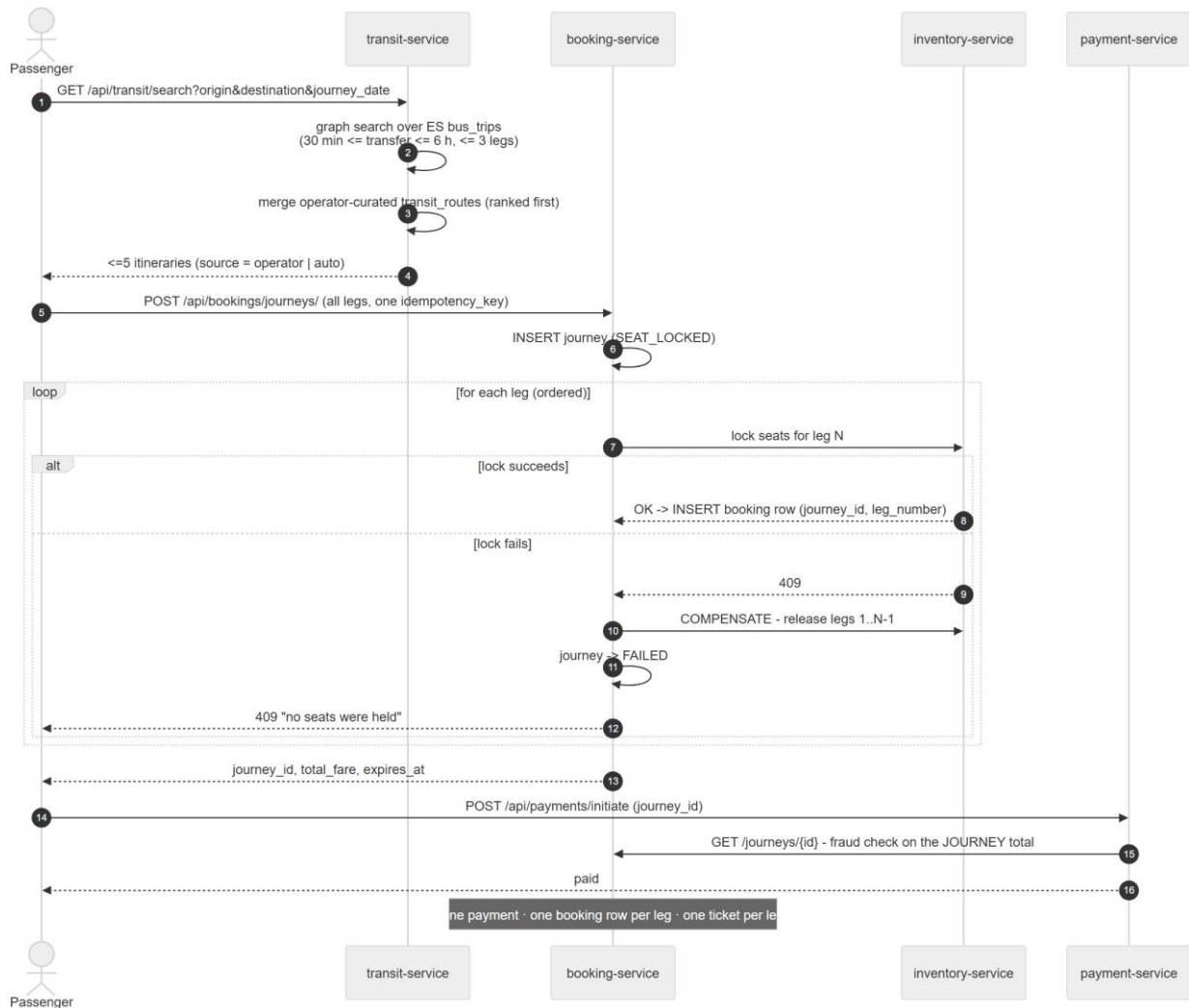


Figure 3. Transit journey saga and compensation flow.

2.2 Frontend Architecture

Three separate client applications consume the same Kong-fronted API.

2.2.1 Web SPA (busgo/frontend)

```

src/
├── main.tsx                # React root, QueryClientProvider, Router, Toaster
├── App.tsx                 # Route table + ProtectedRoute role gate
├── layout/MainLayout.tsx   # Navbar (with NotificationBell), footer, <Outlet/>
├── pages/                  # 27 route-level screens
│   ├── Home.tsx
│   ├── SearchResults.tsx
│   ├── Routes.tsx
│   ├── Deals.tsx
│   ├── SelectSeats.tsx
│   ├── PassengerDetails.tsx
│   ├── Payment.tsx
│   ├── Confirmation.tsx
│   ├── TransitSeats.tsx
│   ├── TransitPassengerDetails.tsx
│   ├── MyBookings.tsx
│   ├── Cancellation.tsx
│   ├── Profile.tsx
│   ├── Login.tsx
│   ├── Register.tsx
│   ├── OperatorPortal.tsx
│   ├── ManageTrips.tsx
│   ├── OperatorBookings.tsx
│   ├── OperatorDeals.tsx
│   ├── OperatorAnalytics.tsx
│   ├── OperatorTransitRoutes.tsx
│   ├── OperatorSendNotification.tsx
│   ├── FillEmptySeats.tsx
│   ├── AdminPortal.tsx
│   ├── AdminNotifications.tsx
│   ├── AdminNotificationPanel.tsx
├── components/            # CityCombobox, LocationSearch, MapSelectorModal
├── notifications/         # NotificationBell, NotificationCard, NotificationsPage,
│                           # notificationApi, notificationStore (self-contained feature module)
├── stores/authStore.ts    # Zustand + persist + cookie fallback + cross-tab sync
├── api/client.ts          # Axios instance: auth interceptor + refresh-queue interceptor
├── hooks/useCityOptions.ts # shared city list
├── lib/                   # supabase.ts, googleAuth.ts
└── utils/cn.ts           # clsx + tailwind-merge class helper
  
```

Component structure. Page components own data fetching (React Query) and compose small presentational pieces. Cross-cutting UI - the notification bell, the city combobox, the Leaflet map picker - is factored into reusable components. The notifications feature is organised as a vertical slice (api + store + components in one folder), which kept a 6-file feature from leaking into four different top-level directories.

Routing. A single <Routes> tree in App.tsx. Public pages (/ , /search, /routes, /deals, /login, /register, and seat selection) render inside MainLayout. Everything that touches personal data is wrapped in <ProtectedRoute>, which redirects unauthenticated users to /login while preserving the intended destination in router state, so the user lands back where they were after Google sign-in. Role-scoped portals mount as wildcard sub-routers:

```

<Route path="/operator/*" element={
  <ProtectedRoute allowedRoles={['OPERATOR', 'ADMIN']}><OperatorPortal /></ProtectedRoute> } />
  
```

```
<Route path="/admin/*" element={
  <ProtectedRoute allowedRoles={['ADMIN']}><AdminPortal /></ProtectedRoute> } />
```

This is a *UX* gate, not a security boundary - every protected endpoint independently re-verifies the JWT and the role server-side (Section 4.2).

State management. Covered in Section 1.4. Two details worth calling out:

- **Token refresh is queued.** `api/client.ts` installs a response interceptor: on a 401 it refreshes once, parks every other in-flight request in a `failedQueue`, then replays them with the new token. Session endpoints (`/login`, `/refresh`, `/google-login`) are excluded so a genuine credential failure does not trigger a refresh loop. Crucially, the store only logs the user out when the refresh *itself* is rejected (400/401/403) - network errors and 5xx preserve the session so a flaky connection cannot evict a valid user.
- **Cross-tab consistency.** A storage event listener rehydrates the Zustand store when another tab writes new tokens, preventing one tab from stranding another on an expired token.

2.2.2 Passenger mobile app (busgo/mobile)

Expo SDK 54 / React Native 0.81 / React Navigation 7, with 16 screens (`HomeScreen`, `ResultsScreen`, `SeatsScreen`, `TransitSeatsScreen`, `PassengerScreen`, `PaymentScreen`, `ConfirmationScreen`, `TripsScreen`, `BookingDetailScreen`, `TicketDetailScreen`, `AlertsScreen`, `DealsScreen`, `RoutesScreen`, `ProfileScreen`, `LoginScreen`, `PhoneSetupScreen`). It is at feature parity with the web passenger experience (documented in `mobile/MOBILE_WEB_PARITY_AUDIT.md`), including guest browsing, transit journeys, seat holds, promo codes, QR/PDF tickets and notifications. Tokens live in `expo-secure-store`; Google OAuth returns through the `busgo://auth/callback` deep link.

2.2.3 Operator mobile app (busgo/operator)

A separate Expo app with 12 screens (`DashboardScreen`, `BusesScreen`, `RoutesScreen`, `TripsScreen`, `TransitRoutesScreen`, `BookingsScreen`, `DealsScreen`, `FillSeatsScreen`, `NotifyScreen`, `AnalyticsScreen`, `ManageScreen`, `LoginScreen`). Keeping it as a distinct binary means an operator's device never ships passenger checkout code, and the two apps can be released independently.

2.3 Backend Architecture

2.3.1 Service decomposition

Fourteen FastAPI services, each with its own database, Dockerfile, dependency set and Kong route. The split follows business capability, not entity CRUD:

Service	Responsibility	Notable dependencies
auth-service	Registration, login, Google session exchange, JWT issue/refresh, OTP, profile	Supabase Auth, Redis
search-service	Trip search over Elasticsearch, city list, reindex	ES, inventory-service, Kafka (trip.*)
transit-service	Multi-leg itinerary graph search (stateless)	ES, operator-service
inventory-service	Seat inventory, distributed seat locks, confirm/release	Redis, Kafka
booking-service	Booking + journey lifecycle, saga orchestration, travel records	operator, deals, payment, bank, notification, auth services; Kafka
payment-service	Payment initiation, mock gateways, fraud checks, refunds	bank-service, booking-service, Kafka
bank-service	Sandbox accounts, balances, PIN verification, and debit/credit ledger	Kafka
ticket-service	QR + PDF ticket generation, QR validation	Kafka (ticket.issued), volume storage
cancellation-service	Cancellation requests, refund policy, operator-initiated cancellations	Kafka
operator-service	Operators, buses, routes, trips, curated transit routes	Kafka (trip.*)
deals-service	Promo codes, flash sales, validation and application	Redis
notification-service	Event->notification fan-out, in-app inbox, mocked email/SMS/push/WhatsApp, scheduler	Kafka, Jinja2
admin-service	Platform dashboard, user roles, notices, transaction summaries	-
audit-service	Append-only audit log fed from audit.log events	Kafka

2.3.2 Internal structure of a service

Services follow a consistent layered layout - a router (HTTP/controller) layer, a service layer for external calls and domain logic, an ORM model layer and Pydantic schemas:

```
services/<name>/
├── main.py           # FastAPI app, CORS, observability, health router, startup hooks
├── core/config.py    # pydantic-settings, env-driven
├── api/deps.py       # get_current_user_payload(), require_role() - FastAPI DI
├── routers/         # HTTP layer (thin: validate -> delegate -> envelope)
├── services/        # domain + integration layer (external.py, redis_svc.py,
│                   # kafka_consumer.py, scheduler.py, gateway.py, ...)
├── models/          # SQLAlchemy declarative models
└── schemas/         # Pydantic request/response models
```



```

└─ shared/          # bind-mount of busgo/shared (see below)
└─ Dockerfile
└─ requirements.txt

```

2.3.3 Design patterns used, and why

Pattern	Where	Why
API Gateway	Kong (infrastructure/kong/kong.yml)	One public origin, one place for routing/LB/health/retries; clients never learn service topology. <code>strip_path: true</code> means <code>/api/auth/login</code> reaches the service as <code>/login</code> , so services stay unaware of their public prefix (<code>ROOT_PATH</code> only fixes up the Swagger server URL).
Database per service	13 logical DBs	Enforces the service boundary at the storage layer - no cross-service joins are even possible, so coupling must be explicit and versioned (HTTP or events).
Saga with compensation	booking-service/routers/journeys.py	A multi-leg journey has no distributed transaction available. Legs are locked in order; any failure triggers release of everything already held, then marks the journey FAILED.
Orchestration + choreography mix	booking orchestrates the synchronous critical path; Kafka choreographs the rest	Money and seat locks need deterministic ordering (orchestration); tickets, notifications and audit are eventual (choreography), so a slow ticket render never delays checkout.
Idempotency key	booking-service, payment-service	Retries over an unreliable network are inevitable. The key is a unique DB column <i>and</i> a Redis cache entry; payments additionally short-circuit to the original COMPLETED row.
Distributed lock	inventory-service/services/redis_svc.py	SET key value NX EX 600 is atomic in Redis, which is exactly the primitive the seat race needs. The DB row is the durable record; Redis is the arbiter.
Circuit breaker + retry with backoff	shared/http_client.py (ResilientClient, tenacity)	A chronically failing dependency fails fast (5 failures -> open for 30 s, then half-open) instead of consuming request workers.
Repository-ish service layer	services/external.py, services/bank_client.py, services/booking_client.py	Routers never call httpx directly, so timeouts, retries and correlation headers are applied uniformly and are testable in one place.
Dependency injection	FastAPI Depends - <code>get_db</code> , <code>get_current_user_payload</code> , <code>require_role(UserRole.OPERATOR)</code>	Auth and session handling become declarative per-endpoint concerns.
Response envelope	shared/base_response.py - {success, data, message, errors}	Uniform client parsing and error surfacing. (deals-service predates this and returns raw objects - a known inconsistency, Section 8.3.)
Event-driven fan-out	shared/kafka_producer.py / per-service consumers	Adding a consumer (audit, notifications) requires no change to the producer.
Shared library via bind mount	busgo/shared mounted to /app/shared in every container	health, observability, http_client, kafka_producer, base_response, enums, exceptions are written once. This keeps shared operational behavior consistent across every service.
Sidcar-free observability	setup_observability(app, SERVICE_NAME)	One call adds /metrics, JSON logging and X-Request-ID correlation to any service.
Health-check contract	create_health_router() -> /health (liveness), /health/ready (DB+Redis readiness)	Kong's active checks poll /health; readiness distinguishes "process up" from "dependencies reachable".

2.3.4 Event catalogue

Published topics (9): `booking.created`, `booking.cancelled`, `payment.completed`, `payment.failed`, `ticket.issued`, `refund.initiated`, `seat.lock.expired`, `notification.send`, `audit.log`.

Consumed event types (~30), including `booking.confirmed`, `booking.expired`, `booking.payment_failed`, `journey.payment_failed`, `trip.created/updated/cancelled/delayed/schedule_changed`, `user.registered`, `operator.booking_alert`, `operator.daily_summary`, `operator.revenue_summary`, `operator.route_update`, `admin.new_user`, `admin.new_operator`, `admin.system_alert`, `admin.daily_summary`, `admin.revenue_report`, `admin.complaint`, `ticket.generated`.

2.3.5 Reliability & scaling

- **Replication + load balancing.** `auth-service`, `search-service` and `booking-service` are designed to run at 3 replicas. Kong resolves the Compose service name through Docker's embedded DNS (`KONG_DNS_RESOLVER: 127.0.0.11`, `KONG_DNS_STALE_TTL: 4`) so each replica appears as an A-record, and Kong's ring balancer round-robins across them. Every `/health` response includes an `instance` field (the container hostname) so load distribution is directly observable - which is what the load test in Section 5.4 measures.

- **Health-driven routing.** Active checks poll `/health` every 5 s (3 failures -> unhealthy); passive checks trip a target out after 5 HTTP failures, acting as a circuit breaker at the gateway. `retries: 3` re-dispatches a failed request to a different target.
- **Graceful degradation.** `bank-service` starts even if Kafka is unavailable (account provisioning self-heals on first balance fetch); `search-service` returns trips with `availability_unknown` rather than failing if `inventory-service` is down.
- **Router-ordering discipline.** The health router is registered *before* any router with a greedy `/id` route, otherwise `/health` would be swallowed and return 401/422, and Kong would mark the whole service unhealthy. This bit four services before it was made a documented convention.

2.4 Database Design

2.4.1 Entity-Relationship Diagram

Because each service owns its own database, the diagram below is a *logical* ERD. Solid relationships exist inside one service's database and are enforced by real foreign keys; dashed relationships cross a service boundary and are **referential by UUID only** - deliberately not FK-enforced, because the target table lives in a different database.



Figure 4. Entity-relationship diagram across service-owned databases.

2.4.2 Normalization decisions

The schema is **3NF for all relational, money-touching data**, with deliberate, documented denormalisation where 3NF would have bought nothing:

Held to 3NF

- operators -> buses -> trips <- routes is fully decomposed. A trip stores *references* to its bus and route, never a copy of the registration number or the city pair, so renaming a route cannot desynchronise history.
- payments and refunds are separate tables in a 1-to-many relation - a payment may be refunded partially or repeatedly.
- bank_accounts and bank_transactions form a proper ledger: the account holds the current balance, and each transaction records balance_after, so the balance is reconstructible and auditable.
- Booking state changes live in booking_status_history rather than being overwritten in place, giving a complete transition trail (from_status -> to_status, reason, timestamp).

Deliberately denormalised

Decision	Rationale
bookings.seat_numbers and bookings.passenger_details as JSONB instead of a booking_passengers child table	A booking has 1-4 passengers that are always read and written together, are never queried independently, and are a point-in-time snapshot . Normalising would add a join to every read for no query benefit.
buses.seat_layout, buses.amenities, routes.boarding_points, routes.dropping_points as JSONB	Genuinely document-shaped, operator-defined and variable-arity. A seat table per bus would be a schema for data the system only ever consumes whole.
bookings.journey_date / departure_time copied from the trip	Immutability of the contract: if an operator later reschedules a trip, the ticket must still show what the passenger actually bought.
travel_records duplicating (user, origin, destination, operator) from bookings	A purpose-built read model for the "fill empty seats" campaign query. Answering it from bookings would mean a cross-service join, which the architecture forbids.
transit_routes.via_cities / leg_assignments as JSONB	An ordered list of 1-2 intermediate cities with per-leg assignments - array semantics matter, and it is always read as a unit.
audit_logs.payload, notification_logs.payload, in_app_notifications.metadata as JSON	Schemaless by design: these must accept the payload of <i>any</i> event type without a migration whenever a new event is introduced.
Cross-service UUID references without FKs (dashed lines above)	A physical FK across databases is impossible. The trade-off is accepted deliberately: consistency is maintained by the saga, the events and the compensation logic rather than by the RDBMS.

2.4.3 Indexes

Table	Index	Query it serves
users	email (unique), phone, provider_subject (unique)	Login, Google-account linking, admin user lookup
refresh_tokens	token (unique)	Token rotation on refresh
otp_records	phone	OTP verification
seat_inventory	trip_id	Every seat-map render and lock attempt
bookings	user_id, trip_id, journey_id, idempotency_key (unique)	"My bookings", trip manifests, journey leg grouping, duplicate suppression
journeys	user_id, idempotency_key (unique)	Journey history and idempotent creation
travel_records	user_id, trip_id, operator_id, origin, destination	The corridor query behind "fill empty seats"
payments	booking_id, user_id, trip_id, gateway_transaction_id (unique)	Payment history and the <i>attempts-per-trip</i> fraud check
bank_accounts	user_id, account_number (unique)	Wallet lookup and debit
tickets	user_id, trip_id, booking_id (unique), qr_code_data (ix_tickets_qr_data)	Ticket retrieval and QR scan validation at the gate
buses	registration_no (unique)	Fleet lookup, duplicate prevention
routes	origin_city, destination_city	Route browsing and transit assembly
transit_routes	operator_id, origin_city, destination_city	Curated-itinerary lookup by transit-service
promo_codes	code (unique), operator_id	Promo validation on the checkout hot path
flash_sales	operator_id	Operator deal listings
in_app_notifications	user_id	Inbox and unread count (polled every 30 s)
audit_logs	event_type, entity_id, user_id, created_at	Admin forensic queries by any dimension

Beyond Postgres, **Elasticsearch's bus_trips index** is the search-side denormalisation: keyword fields for `origin_city`, `destination_city`, `operator_id`, `bus_type` and `status`, date fields for departure/arrival, and numerics for fare and seats. It exists precisely so that search never queries the operator database.

2.4.4 Data Sync Mechanics - how database changes reach the UI without a manual refresh

BusGo keeps the UI live through **three complementary mechanisms**; there is no manual "press F5 to see new data" step anywhere in the booking flow.

1. Event-driven server-side propagation (Kafka). A write in one service's database is broadcast as an event, and every interested service updates *its own* store. When an operator creates a trip, operator-service commits to operator_db and publishes `trip.created`; search-service consumes it and indexes the trip into Elasticsearch - so the trip becomes searchable to passengers without anyone reindexing by hand. The same mechanism drives `payment.completed` -> booking CONFIRMED -> `ticket.issued` -> notification, so a completed payment materialises a ticket and an inbox entry with no user action.

2. Client-side cache invalidation (React Query). The web app treats server data as a cache with a lifetime. After any mutation, the affected queries are invalidated and refetched, so the seat map, booking list and wallet balance re-render from fresh data automatically. Volatile resources - most importantly the seat map - are additionally refetched on window focus and at interval, so a seat another user just locked greys out on its own.

3. Notification polling (30 s). `notificationStore.startPolling()` runs while the NotificationBell is mounted, calling `GET /api/notifications/` and `/stats` every 30 seconds and updating the unread badge in place. This is what makes server-originated events (a delayed bus, an operator offer, a refund completion) appear in a UI the user is not actively interacting with.

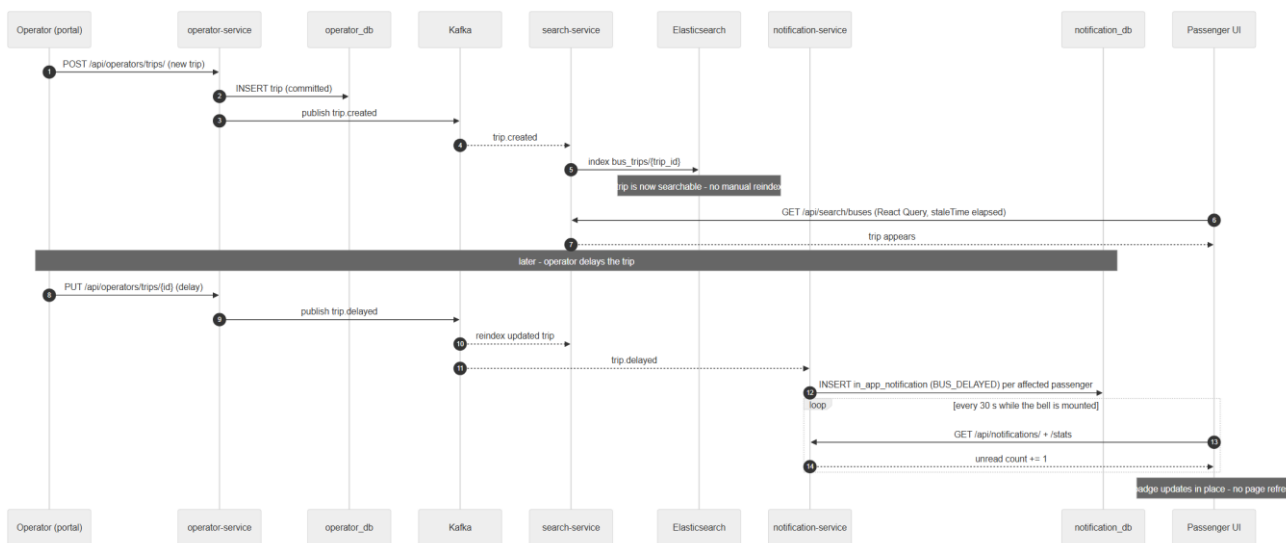


Figure 5. Database-to-UI synchronization mechanics.

Why polling rather than WebSockets/SSE. A 30-second poll of a single indexed query (`user_id`) was the right trade-off for this system's scale and deadline: it needs no sticky sessions, survives Kong's stateless load balancing across replicas with no extra configuration, and degrades gracefully on flaky mobile networks. The trade-off is a bounded delay of up to 30 seconds for a notification, while booking and seat-locking operations remain immediately consistent through their authoritative service endpoints.

The one place where 30 seconds would be too slow, the **seat map**, is handled differently: seat state is re-read directly from `inventory-service` (DB rows overlaid with live Redis locks) on focus and interval during selection, and the authoritative check happens at lock time anyway - a stale map can only cost the user a 409, never a double-booked seat.

3. API Documentation

3.1 API Design Overview

Style - REST over JSON. Resources are nouns (`/bookings`, `/trips`, `/tickets`, `/promos`), HTTP verbs carry the semantics (`GET` read, `POST` create/command, `PUT` full update, `PATCH` partial update, `DELETE` remove), and status codes are meaningful (200, 201, 400 validation, 401 unauthenticated, 403 role/limit violation, 404 missing, 409 seat conflict, 503 dependency unavailable). GraphQL was considered and rejected: with 14 independently deployed services, a single GraphQL schema would have re-coupled them behind one gateway resolver layer, which is the opposite of what the architecture is demonstrating.

Base URL.

Environment	Base URL
Production	<code>https://busgo.farefin.com/api</code>
Local (Docker Compose, via Kong)	<code>http://localhost:18085/api</code> - mapped from <code>KONG_PORT</code> , default 8085
Direct service access (debug only)	<code>http://localhost:81xx</code> - see the port table in Section 6.2

Versioning scheme. The API is currently **v1 implicit** - there is no `/v1` path segment. The chosen forward path is *gateway-level versioning*: because every route is declared in `kong.yml`, a future `v2` is introduced by adding routes with a `/api/v2/...`

path pointing at new upstreams, while `/api/...` continues to serve v1 from the existing ones - clients migrate independently and no service code changes.

Routing model. Kong matches on path prefix and strips it (`strip_path: true`), so `/api/auth/login` arrives at `auth-service` as `/login`. Each service is told its public prefix through `ROOT_PATH`, which only affects the server URL advertised in its OpenAPI document - service code never hardcodes the gateway prefix. Both plural and singular prefixes are accepted for the main resources (`/api/bookings` and `/api/booking`, `/api/tickets` and `/api/ticket`, ...), which removed a whole class of client 404s.

Response envelope. Every service except `deals-service` wraps responses in a uniform envelope (`shared/base_response.py`):

```
{ "success": true, "data": { }, "message": "", "errors": null }
```

Authentication. A Bearer JWT in the Authorization header, verified independently by each service (Section 4). Public endpoints (search, transit search, city list, route/deal browsing, seat maps) require no token, which is what makes guest browsing possible on web and mobile.

Correlation. Every request carries or is assigned an `X-Request-ID`, echoed on the response and propagated by `ResilientClient` on all downstream calls, so one user action is traceable across gateway -> service -> service in Loki.

3.2 Endpoint Reference

Routes are given as the **public path through Kong** (prefix `https://busgo.farefin.com`). "Auth" indicates whether a valid JWT is required, and where a specific role is enforced.

3.2.1 auth-service - `/api/auth`

Method	Route	Description	Auth Required
POST	<code>/api/auth/register</code>	Register a new user (name, phone, email, password, role)	No
POST	<code>/api/auth/login</code>	Password login by phone -> access + refresh token	No
POST	<code>/api/auth/token</code>	OAuth2 password-grant form login (used by Swagger's <i>Authorize</i>)	No
POST	<code>/api/auth/google-login</code>	Exchange a verified Supabase Google session for a BusGo session	No
POST	<code>/api/auth/refresh</code>	Rotate an access token using a refresh token	No (refresh token in body)
POST	<code>/api/auth/logout</code>	Revoke the current refresh token	Yes
GET	<code>/api/auth/me</code>	Current user profile	Yes
PUT	<code>/api/auth/me</code>	Update own profile (name, phone, email)	Yes
POST	<code>/api/auth/send-otp</code>	Send a one-time password to a phone number	No
POST	<code>/api/auth/verify-otp</code>	Verify an OTP and mark the user verified	No
POST	<code>/api/auth/users/lookup</code>	Resolve user ids -> display names (used by operator/admin views)	Yes
POST	<code>/api/auth/create-admin</code>	Bootstrap an administrator account	Yes (guarded)
POST	<code>/api/auth/reset-admin-password</code>	Recover administrator access	Yes (guarded)

3.2.2 search-service - `/api/search`

Method	Route	Description	Auth Required
GET	<code>/api/search/cities</code>	List searchable cities	No
GET	<code>/api/search/buses</code>	Search trips by origin, destination, date (+ filters); merges live seat availability	No
GET	<code>/api/search/buses/{trip_id}</code>	Full detail for one trip	No
POST	<code>/api/search/reindex</code>	Rebuild the Elasticsearch <code>bus_trips</code> index from operator data	No (operational)

3.2.3 transit-service - `/api/transit`

Method	Route	Description	Auth Required
GET	<code>/api/transit/search</code>	Find multi-leg connecting itineraries (origin, destination, journey_date); operator-curated results ranked first	No

3.2.4 inventory-service - `/api/inventory`

Method	Route	Description	Auth Required
POST	<code>/api/inventory/trips/{trip_id}/initialize</code>	Create the seat inventory for a trip from its layout	No (internal)
GET	<code>/api/inventory/trips/{trip_id}/seats</code>	Live seat map (DB rows overlaid with Redis locks)	No
GET	<code>/api/inventory/trips/{trip_id}/available-count</code>	Count of genuinely free seats	No
POST	<code>/api/inventory/trips/{trip_id}/seats/lock</code>	Atomically lock seats for a booking (Redis SET NX EX 600)	No (internal)
POST	<code>/api/inventory/trips/{trip_id}/seats/release</code>	Release locks (compensation / expiry)	No (internal)
POST	<code>/api/inventory/trips/{trip_id}/seats/confirm</code>	Promote locked seats to BOOKED after payment	No (internal)
POST	<code>/api/inventory/trips/{trip_id}/seats/unbook</code>	Return booked seats to the pool after a cancellation	No (internal)

3.2.5 booking-service - /api/bookings

Method	Route	Description	Auth Required
POST	/api/bookings/	Create a booking: validate against operator, apply promo, lock seats, emit events	Yes
GET	/api/bookings/my	Current user's bookings (grouped by journey)	Yes
GET	/api/bookings/{booking_id}	Single booking detail	Yes (owner)
GET	/api/bookings/trip/{trip_id}	Passenger manifest for a trip	Yes (operator)
GET	/api/bookings/operator/{operator_id}	All bookings for an operator	Yes (operator)
POST	/api/bookings/{booking_id}/confirm-payment	Confirm a paid booking, confirm seats, issue ticket	Yes
POST	/api/bookings/{booking_id}/cancel	Cancel and trigger the refund flow	Yes (owner)
GET	/api/bookings/{booking_id}/cancellation-info	Refund eligibility and amount before cancelling	Yes (owner)
POST	/api/bookings/{booking_id}/apply-promo	Validate and persist a promo discount pre-payment	Yes (owner)
DELETE	/api/bookings/{booking_id}/apply-promo	Remove an applied promo	Yes (owner)
GET	/api/bookings/travel-history/my	The user's travel history	Yes
POST	/api/bookings/travel-records/sync	Rebuild travel records from confirmed bookings	Yes (operational)
POST	/api/bookings/trips/{trip_id}/interested-passengers	Past passengers on this corridor ("fill empty seats")	Yes (operator)
POST	/api/bookings/trips/{trip_id}/notify-interested	Push a targeted offer to those passengers	Yes (operator)
POST	/api/bookings/journeys/	Multi-leg journey saga - lock every leg or release all	Yes
GET	/api/bookings/journeys/{journey_id}	Journey with ordered legs and transfer waits	Yes (owner)
POST	/api/bookings/journeys/{journey_id}/confirm-payment	Confirm the whole journey after one payment	Yes (owner)
POST	/api/bookings/journeys/{journey_id}/cancel	Cancel every leg together	Yes (owner)

3.2.6 payment-service - /api/payments

Method	Route	Description	Auth Required
POST	/api/payments/initiate	Start a payment for a booking or a journey; runs fraud checks and debits the account	Yes
POST	/api/payments/{gateway}/callback	Mock gateway callback (bKash / Nagad / bank)	No (gateway)
GET	/api/payments/my	Current user's payment history	Yes
GET	/api/payments/{payment_id}	Single payment detail	Yes (owner)
GET	/api/payments/booking/{booking_id}	Payment(s) for a booking	Yes (owner)
POST	/api/payments/{payment_id}/refund	Initiate a refund	Yes
POST	/api/payments/mock/simulate-failure	Test hook: force the mock gateway to fail	No (test only)

3.2.7 bank-service - /api/bank

Method	Route	Description	Auth Required
POST	/api/bank/provision	Create simulated wallet/bank accounts for a user	Yes
GET	/api/bank/accounts/my	Own accounts and balances	Yes
GET	/api/bank/transactions/my	Own ledger entries	Yes
POST	/api/bank/verify-debit	Verify PIN + balance and debit (called by payment-service)	Yes (internal)
POST	/api/bank/credit	Credit an account (refunds)	Yes (internal)
GET	/api/bank/admin/accounts	All accounts	Yes (ADMIN)
GET	/api/bank/admin/accounts/by-user/{user_id}	Accounts of one user	Yes (ADMIN)
POST	/api/bank/admin/accounts/{account_id}/set-balance	Set a demo balance	Yes (ADMIN)

3.2.8 ticket-service - /api/tickets

Method	Route	Description	Auth Required
GET	/api/tickets/my	All of the user's tickets	Yes
GET	/api/tickets/{ticket_id}	Single ticket	Yes (owner)
GET	/api/tickets/booking/{booking_id}	Ticket for a booking (one per leg)	Yes (owner)
GET	/api/tickets/files/{booking_id}/qr	QR code image	Yes (owner)
GET	/api/tickets/files/{booking_id}/pdf	Printable PDF ticket	Yes (owner)
POST	/api/tickets/validate-qr	Validate a scanned QR at boarding	Yes (operator)

3.2.9 cancellation-service - /api/cancellations

Method	Route	Description	Auth Required
POST	/api/cancellations/	Raise a cancellation request and compute the refund	Yes
GET	/api/cancellations/{id}	Cancellation request status	Yes
GET	/api/cancellations/booking/{booking_id}	Cancellations for a booking	Yes
POST	/api/cancellations/operator-cancel	Operator cancels a trip; refunds every passenger	Yes (OPERATOR)

3.2.10 operator-service - /api/operators

Method	Route	Description	Auth Required
GET	/api/operators/operators/	List operators	No
POST	/api/operators/operators/register	Register an operator profile	Yes
GET	/api/operators/operators/{id}	Operator detail	No
PUT	/api/operators/operators/{id}	Update an operator	Yes (OPERATOR)
POST	/api/operators/operators/{id}/buses	Add a bus	Yes (OPERATOR)
GET	/api/operators/operators/{id}/buses	List an operator's buses	Yes (OPERATOR)
PUT	/api/operators/buses/{id}	Update a bus	Yes (OPERATOR)
DELETE	/api/operators/buses/{id}	Remove a bus	Yes (OPERATOR)
POST	/api/operators/operators/{id}/routes	Add a route	Yes (OPERATOR)
GET	/api/operators/operators/{id}/routes	List an operator's routes	Yes (OPERATOR)
GET	/api/operators/routes/	All routes (public browsing)	No
PUT	/api/operators/routes/{id}	Update a route	Yes (OPERATOR)
DELETE	/api/operators/routes/{id}	Remove a route	Yes (OPERATOR)
GET	/api/operators/trips/	List trips	No
POST	/api/operators/trips/	Schedule a trip (emits trip.created -> reindexed)	Yes (OPERATOR)
GET	/api/operators/trips/{id}	Trip detail - the authoritative fare source for bookings	No
PUT	/api/operators/trips/{id}	Update a trip	Yes (OPERATOR)
POST	/api/operators/trips/{id}/cancel	Cancel a trip	Yes (OPERATOR)
DELETE	/api/operators/trips/{id}	Delete a trip	Yes (OPERATOR)
GET	/api/operators/transit-routes/	Public curated transit routes (used by transit-service)	No
GET	/api/operators/transit-routes/mine	The operator's own transit routes	Yes (OPERATOR)
POST	/api/operators/transit-routes/	Publish a curated connecting route	Yes (OPERATOR)
PUT	/api/operators/transit-routes/{id}	Update a curated route	Yes (OPERATOR)
DELETE	/api/operators/transit-routes/{id}	Remove a curated route	Yes (OPERATOR)

3.2.11 deals-service - /api/deals

Method	Route	Description	Auth Required
POST	/api/deals/validate-promo	Validate a code against fare, window, usage cap and prior use	No
POST	/api/deals/apply-promo	Consume a promo (increment usage, mark user)	No (internal)
GET	/api/deals/promos/	List promo codes	No
POST	/api/deals/promos/	Create a promo code	Yes (OPERATOR/ADMIN)
PUT	/api/deals/promos/{id}	Update a promo code	Yes (OPERATOR/ADMIN)
DELETE	/api/deals/promos/{id}	Delete a promo code	Yes (OPERATOR/ADMIN)
GET	/api/deals/flash-sales/	List flash sales	No
GET	/api/deals/flash-sales/active	Currently running flash sales	No
POST	/api/deals/flash-sales/	Create a flash sale	Yes (OPERATOR/ADMIN)
PUT	/api/deals/flash-sales/{id}	Update a flash sale	Yes (OPERATOR/ADMIN)
DELETE	/api/deals/flash-sales/{id}	Delete a flash sale	Yes (OPERATOR/ADMIN)

3.2.12 notification-service - /api/notifications

Method	Route	Description	Auth Required
GET	/api/notifications/	Paginated inbox (page, limit, unread_only)	Yes

Method	Route	Description	Auth Required
GET	/api/notifications/stats	Unread count and per-type totals (polled every 30 s)	Yes
PATCH	/api/notifications/{notification_id}/read	Mark one as read	Yes (owner)
PATCH	/api/notifications/read-all	Mark everything as read	Yes
DELETE	/api/notifications/{notification_id}	Delete one notification	Yes (owner)
DELETE	/api/notifications/clear-all	Clear the inbox	Yes
POST	/api/notifications/send	Send a targeted notification (operator -> passengers)	Yes (OPERATOR/ADMIN)
POST	/api/notifications/admin/broadcast	Broadcast to a role or the whole platform	Yes (ADMIN)

3.2.13 admin-service - /api/admin

Method	Route	Description	Auth Required
GET	/api/admin/dashboard-stats	Platform totals (users, operators, trips, revenue)	Yes (ADMIN)
GET	/api/admin/users	All users	Yes (ADMIN)
PATCH	/api/admin/users/{user_id}/role	Promote/demote a user	Yes (ADMIN)
GET	/api/admin/user-history	Per-user activity history	Yes (ADMIN)
GET	/api/admin/operators	All operators	Yes (ADMIN)
GET	/api/admin/routes	All routes	Yes (ADMIN)
GET	/api/admin/trips	All trips	Yes (ADMIN)
GET	/api/admin/transactions	All transactions	Yes (ADMIN)
GET	/api/admin/transactions/summary	Revenue summary	Yes (ADMIN)
GET	/api/admin/notices	All notices	Yes (ADMIN)
GET	/api/admin/notices/active	Active notices (shown on the public site)	No
POST	/api/admin/notices	Publish a notice	Yes (ADMIN)
PATCH	/api/admin/notices/{notice_id}	Edit a notice	Yes (ADMIN)
DELETE	/api/admin/notices/{notice_id}	Remove a notice	Yes (ADMIN)

3.2.14 audit-service - /api/audit

Method	Route	Description	Auth Required
GET	/api/audit/audit/logs	Query the audit trail (event_type, entity_type, skip, limit)	Yes (ADMIN)
GET	/api/audit/audit/logs/booking/{booking_id}	Full event history for a booking	Yes (ADMIN)
GET	/api/audit/audit/logs/user/{user_id}	Full event history for a user	Yes (ADMIN)

3.2.15 Platform endpoints (all 14 services)

Method	Route	Description	Auth Required
GET	/api/{service}/health	Liveness; includes instance (container id) - used by Kong's active checks and the load-balancing test	No
GET	/api/{service}/health/ready	Readiness: 200 only if PostgreSQL and Redis respond, else 503	No
GET	/api/{service}/metrics	Prometheus metrics (http_requests_total, http_request_duration_seconds)	No
GET	/api/{service}/docs	Interactive Swagger UI	No
GET	/api/{service}/openapi.json	Raw OpenAPI 3.1 document	No

Total: 120+ documented endpoints across 14 services.

3.3 Swagger / Postman Collection Link

FastAPI generates an OpenAPI 3.1 document per service automatically; each is served through Kong, so **every service's Swagger UI is publicly reachable** on the deployed host:

Service	Public Swagger URL
Auth	https://busgo.farefin.com/api/auth/docs
Search	https://busgo.farefin.com/api/search/docs
Transit	https://busgo.farefin.com/api/transit/docs
Inventory	https://busgo.farefin.com/api/inventory/docs
Booking	https://busgo.farefin.com/api/bookings/docs
Payment	https://busgo.farefin.com/api/payments/docs

Service	Public Swagger URL
Bank	https://busgo.farefin.com/api/bank/docs
Ticket	https://busgo.farefin.com/api/tickets/docs
Cancellation	https://busgo.farefin.com/api/cancellations/docs
Operator	https://busgo.farefin.com/api/operators/docs
Deals	https://busgo.farefin.com/api/deals/docs
Notification	https://busgo.farefin.com/api/notifications/docs
Admin	https://busgo.farefin.com/api/admin/docs
Audit	https://busgo.farefin.com/api/audit/docs

Raw OpenAPI JSON is at `../openapi.json` for each service and can be imported straight into Postman (*Import -> Link*), which produces a ready-made collection per service.

A hand-written, copy-pasteable reference covering every service - including the transit journey saga and the seat-race test - lives in the repository at `busgo/tests/curl_commands.md`, with sample payloads in `busgo/tests/test_data.json`.

3.4 Sample Request & Response

3.4.1 Google login - exchange a Supabase session for a BusGo session

Request

```
POST /api/auth/google-login HTTP/1.1
Host: busgo.farefin.com
Content-Type: application/json

{ "token": "<supabase_access_token>" }
```

Response 200 OK

```
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refresh_token": "8f14e45f-ceea-467a-9f0a-3d1b2c9e77aa",
    "token_type": "bearer",
    "user": {
      "id": "3f2b9c10-7a4e-4f1d-9c2a-5b8e6d4a1c33",
      "full_name": "Tauseef Rahman",
      "email": "tauseef@example.com",
      "phone": "01700000000",
      "role": "CUSTOMER",
      "is_verified": true
    }
  },
  "message": "Login successful"
}
```

Response 401 Unauthorized - the Supabase session is not a verified Google identity:

```
{ "detail": "A verified Google account is required" }
```

3.4.2 Search buses

Request

```
GET /api/search/buses?origin=Dhaka&destination=Cumilla&date=2026-08-15 HTTP/1.1
Host: busgo.farefin.com
```

Response 200 OK

```
{
  "success": true,
  "data": [
    {
      "trip_id": "b41d7c2e-9f65-4a18-8f0d-2c7e5a91b3d4",
      "operator_id": "1a2b3c4d-5e6f-4071-8293-a4b5c6d7e8f9",
      "operator_name": "Green Line Paribahan",
      "bus_registration_no": "DHAKA-METRO-B-1234",
      "bus_type": "AC_BUSINESS",
      "origin_city": "Dhaka",
      "destination_city": "Cumilla",
      "departure_datetime": "2026-08-15T08:00:00+06:00",
      "arrival_datetime": "2026-08-15T11:00:00+06:00",
      "fare_amount": 850.0,
      "available_seats": 27,
      "amenities": ["WIFI", "CHARGING_PORT", "BLANKET"],
      "boarding_points": [{ "name": "Gabtoli Bus Terminal", "address": "Gabtoli, Dhaka" }],
      "dropping_points": [{ "name": "Cumilla Cantonment", "address": "Cumilla" }],
      "status": "SCHEDULED"
    }
  ],
  "message": "1 trip found"
}
```

3.4.3 Create a booking (the critical path)

Request

```

POST /api/bookings/ HTTP/1.1
Host: busgo.farefin.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "trip_id": "b41d7c2e-9f65-4a18-8f0d-2c7e5a91b3d4",
  "operator_id": "1a2b3c4d-5e6f-4071-8293-a4b5c6d7e8f9",
  "seat_numbers": ["A1", "A2"],
  "passenger_details": [
    { "name": "Tauseef Rahman", "age": 24, "gender": "male", "seat": "A1" },
    { "name": "Amina Islam", "age": 23, "gender": "female", "seat": "A2" }
  ],
  "boarding_point": "Gabtoli Bus Terminal",
  "dropping_point": "Cumilla Cantonment",
  "journey_date": "2026-08-15",
  "departure_time": "08:00:00",
  "total_fare": 1720.0,
  "promo_code": "EIDSPECIAL",
  "idempotency_key": "b2f7c1e0-4d3a-4f5b-9c6d-7e8f9a0b1c2d"
}

```

(*total_fare* = 850 x 2 seats + 20 service fee. The server independently recomputes this from *operator-service* and rejects the request if it disagrees by more than 0.01.)

Response 200 OK

```

{
  "success": true,
  "data": {
    "booking_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "user_id": "3f2b9c10-7a4e-4f1d-9c2a-5b8e6d4a1c33",
    "trip_id": "b41d7c2e-9f65-4a18-8f0d-2c7e5a91b3d4",
    "operator_id": "1a2b3c4d-5e6f-4071-8293-a4b5c6d7e8f9",
    "boarding_point": "Gabtoli Bus Terminal",
    "dropping_point": "Cumilla Cantonment",
    "journey_date": "2026-08-15",
    "departure_time": "08:00:00",
    "seat_numbers": ["A1", "A2"],
    "expires_at": "2026-08-01T10:40:00+00:00",
    "total_fare": 1548.0
  },
  "message": "Booking created successfully"
}

```

Response 409 Conflict - another user won the seat race:

```
{ "detail": "Seat A1 is already booked" }
```

Response 400 Bad Request - the client sent a stale fare:

```
{ "detail": "Fare changed; refresh the trip and try again" }
```

3.4.4 Transit itinerary search (multi-leg)

Request

```

GET /api/transit/search?origin=Dhaka&destination=Sylhet&journey_date=2026-08-15 HTTP/1.1
Host: busgo.farefin.com

```

Response 200 OK (abridged - up to 5 itineraries)

```

{
  "success": true,
  "data": {
    "origin": "Dhaka",
    "destination": "Sylhet",
    "itineraries": [
      {
        "source": "operator",
        "transit_route_id": "6d1f8b3a-2c9e-4d7f-8a1b-3c5e7f9a0b2d",
        "operator_name": "Green Line Paribahan",
        "legs": [
          {
            "leg_number": 1, "trip_id": "...", "origin_city": "Dhaka", "destination_city": "Cumilla",
            "departure_datetime": "2026-08-15T08:00:00+06:00", "arrival_datetime": "2026-08-15T11:00:00+06:00", "fare": 850.0
          },
          {
            "leg_number": 2, "trip_id": "...", "origin_city": "Cumilla", "destination_city": "Sylhet",
            "departure_datetime": "2026-08-15T12:00:00+06:00", "arrival_datetime": "2026-08-15T17:00:00+06:00", "fare": 1100.0
          }
        ],
        "transfers": [{ "at": "Cumilla", "wait_minutes": 60 }],
        "total_fare": 1950.0,
        "combined_discount_pct": 10.0,
        "final_fare": 1755.0,
        "total_duration_minutes": 540
      },
      {
        "source": "auto",
        "legs": [ /* ... discovered by the graph search ... */ ],
        "transfers": [{ "at": "Bhairab", "wait_minutes": 45 }],
        "total_fare": 1800.0,
        "final_fare": 1800.0
      }
    ]
  },
  "message": "2 itineraries found"
}

```

3.4.5 Payment initiation with fraud checks

Request

```
POST /api/payments/initiate HTTP/1.1
Host: busgo.farefin.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "booking_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "amount": 1548.0,
  "method": "BKASH",
  "mobile_number": "01700000000",
  "pin": "1234"
}
```

Response 200 OK

```
{
  "success": true,
  "data": {
    "payment_id": "a3d5f7b9-1c2e-4a6b-8d0f-2e4c6a8b0d1f",
    "redirect_url": "https://busgo.farefin.com/api/payments/bkash/callback?payment_id=a3d5f7b9-..."
  },
  "message": "Payment initiated"
}
```

Response 400 Bad Request - tampered amount (fraud check 1):

```
{ "detail": "Payment amount does not match booking fare" }
```

Response 403 Forbidden - fourth attempt on the same trip (fraud check 2; seats are then released):

```
{ "detail": "Maximum payment attempts exceeded for this trip" }
```

3.4.6 Health & readiness (used by Kong and the test suite)

```
GET /api/bookings/health/ready

{
  "status": "ready",
  "service": "booking-service",
  "instance": "a1b2c3d4e5f6",
  "checks": { "database": "ok", "redis": "ok" }
}
```

If PostgreSQL or Redis is unreachable the same endpoint returns `503` with the failing check named - which is how Kong removes the instance from rotation.

4. Authentication & Security

4.1 Authentication Strategy

Chosen strategy: OAuth 2.0 (Google, via Supabase Auth) for interactive sign-in, exchanged for stateless HS256 JWTs for API access, with rotating refresh tokens.

Why this combination

Requirement	Why JWT / OAuth won
14 independent services must authenticate the same user	A stateless JWT is verified locally by each service with a shared secret - no network call to auth-service on every request, which would have made it a single point of failure and added latency to every hop. Server-side sessions would have required either sticky sessions (defeating Kong's load balancing) or a shared session store consulted on every request.
Three clients (web SPA, two React Native apps)	A bearer token works identically in a browser and in a native app. Cookie-based sessions do not translate cleanly to React Native.
Users should not have to invent and remember another password	Google OAuth removes password storage, reset flows and credential-stuffing exposure for the interactive path entirely.
Short-lived credentials, long-lived sessions	15-minute access tokens limit the damage of a leaked token; 7-day refresh tokens , stored server-side in <code>refresh_tokens</code> and revocable, keep users signed in.

How Google sign-in works end to end

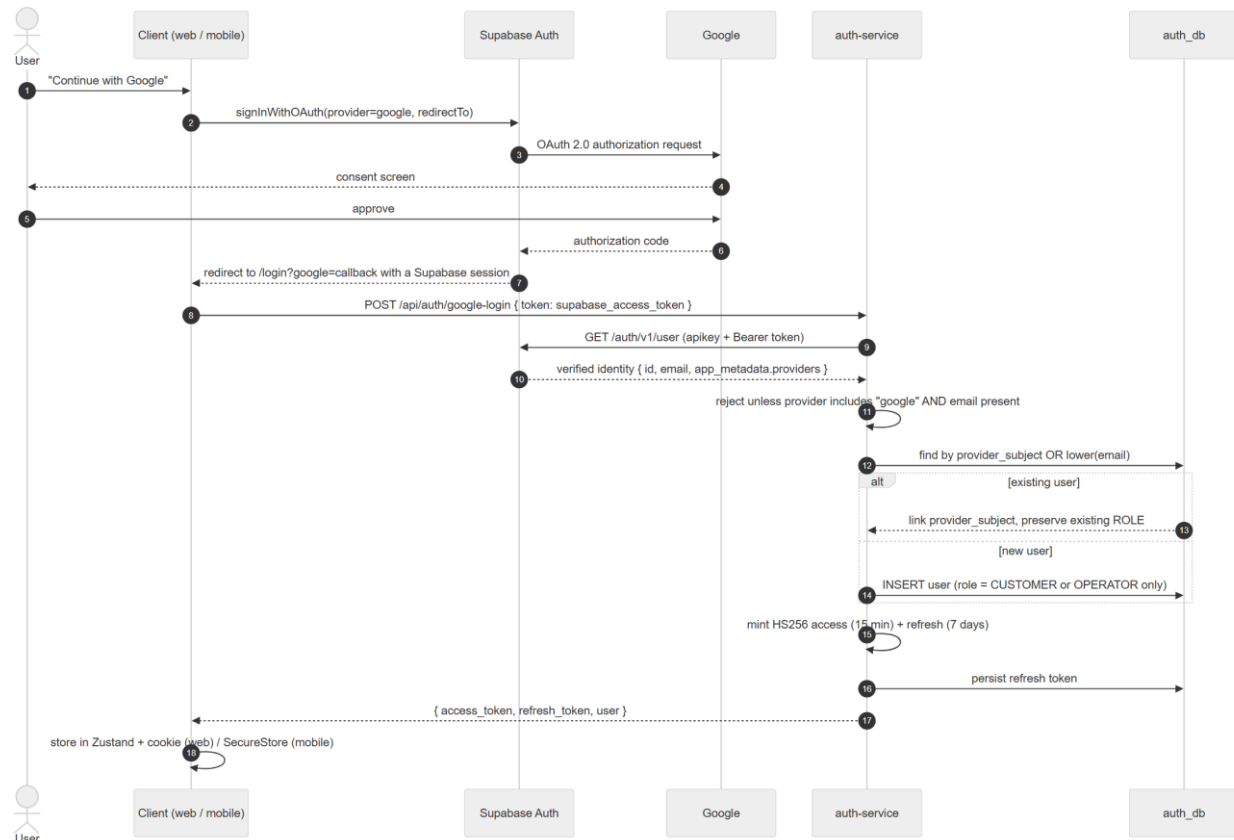


Figure 6. Google sign-in and BusGo token-exchange sequence.

Three properties of this design matter:

4. **BusGo remains the source of truth for identity and role.** Supabase proves *who the person is*; the local `users` table decides *what they may do*. A Google account cannot self-assign a role.
5. **The token is verified server-side, not trusted.** `auth-service` calls Supabase's `/auth/v1/user` with the supplied token and rejects anything that is not a verified Google identity - the client's claim about itself is never believed.
6. **Account linking is by verified e-mail.** An existing password-era account is linked to its Google identity on first sign-in and **keeps its existing role**, so an operator does not silently become a customer.

Token lifecycle

Property	Value
Algorithm	HS256 (python-jose)
Access token TTL	15 minutes
Refresh token TTL	7 days, stored in <code>refresh_tokens</code> with <code>is_revoked</code>
Claims	<code>user_id</code> , <code>role</code> , <code>exp</code>
Transport	Authorization: Bearer <token>
Web storage	Zustand persist -> <code>localStorage</code> , plus a <code>SameSite=Lax</code> cookie fallback
Mobile storage	expo-secure-store (Keychain / Keystore)
Refresh	POST <code>/api/auth/refresh</code> ; the client interceptor refreshes once and replays queued requests
Revocation	POST <code>/api/auth/logout</code> marks the refresh token revoked and signs out of Supabase

Password login (bcrypt-hashed, with per-password salt) and phone OTP verification remain implemented in `auth-service` and are used for seeded test and administrator accounts, but the web UI now presents Google as the only interactive sign-in method.

4.2 Authorization & Role Management

Three roles, carried in the JWT `role` claim and enforced at four independent layers.

Role	Capabilities
CUSTOMER	Search, book, pay, cancel, view own tickets/notifications/wallet
OPERATOR	Everything a customer can do, plus manage own buses, routes, trips, transit routes, deals, manifests, analytics and passenger notifications
ADMIN	Platform-wide: user role management, notices, broadcasts, transactions, audit log, bank administration

Layer 1 - Route guard (UX). `ProtectedRoute` in the SPA redirects unauthenticated users to `/login` and users of the wrong role back to `/`. This layer is convenience only and is assumed to be bypassable.

Layer 2 - Token verification (every service). Each service has its own `api/deps.py` that decodes and validates the JWT locally:

```
def get_current_user_payload(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        if payload.get("user_id") is None:
            raise credentials_exception
        return payload
    except JWTError:
        raise credentials_exception
```

Layer 3 - Role enforcement (declarative). `require_role()` produces a FastAPI dependency that rejects a mismatched role with 403:

```
def require_role(required_role: UserRole):
    def role_checker(payload: dict = Depends(get_current_user_payload)):
        if payload.get("role") != required_role.value:
            raise HTTPException(status_code=403, detail="Operation not permitted")
        return payload
    return role_checker

@router.post("/trips/", dependencies=[Depends(require_role(UserRole.OPERATOR))])
```

Layer 4 - Ownership checks (object level). Being an authenticated customer is not enough to read *someone else's* booking. Queries are scoped to the caller, so an attacker substituting another user's UUID gets 404, not data:

```
query = select(Booking).where(Booking.id == booking_id, Booking.user_id == user_id)
```

The same pattern guards payments, tickets, cancellations and notifications. Operator-scoped resources are additionally filtered by the operator that owns them.

Role assignment rules. New users may self-select only `CUSTOMER` or `OPERATOR` on the web; mobile registration creates `CUSTOMER` only. `ADMIN` can never be chosen by a client - it is granted either by `PATCH /api/admin/users/{id}/role` from an existing administrator, or by a controlled database update for the first bootstrap account.

4.3 Security Measures

Input validation

- **Pydantic schemas on every endpoint.** Types, formats (`EmailStr`, `UUID`, `date`, `time`), bounds and required fields are validated before a handler runs; violations return 422 with a field-level error report. This eliminates an entire class of injection and type-confusion bugs at the edge.
- **Business-rule validation in the booking hot path**, none of which trusts the client: 1-4 seats per booking; no duplicate seats; one passenger record per seat and the sets must match exactly; the trip must exist and be `SCHEDULED`; the operator id must match the trip's; boarding and dropping points must be members of the route's declared points; and the fare must equal the server-computed total within 0.01.
- **Frontend validation** with React Hook Form + Zod gives immediate feedback but is treated purely as UX - every rule is re-checked server-side.

Injection & XSS protection

- **SQL injection:** all database access goes through SQLAlchemy's ORM/expression language with bound parameters. There is no string-concatenated SQL in service code.
- **XSS:** React escapes interpolated content by default, and the app does not use `dangerouslySetInnerHTML`. `DOMPurify` is present in the bundle via the PDF/HTML-canvas toolchain.
- **Path traversal:** ticket files are addressed by `booking_id` (a UUID) and resolved inside a fixed `FILE_STORAGE_PATH`; user input never forms a filesystem path.
- **Password storage:** `bcrypt` with a per-password salt; hashes are never returned by any endpoint.

Rate limiting & abuse control

Control	Where
Max 3 payment attempts per user per trip, after which seats are definitively released and an audit event is written	payment-service
Idempotency keys on booking and payment creation, preventing duplicate charges from retries or double-clicks	booking-service, payment-service
Seat lock TTL (600 s) so an abandoned checkout cannot hold inventory indefinitely	inventory-service (Redis)
Booking expiry (10 min) enforced by an APScheduler job that publishes <code>seat.lock.expired</code>	booking-service
Promo usage caps - global <code>max_uses</code> plus a per-user Redis marker preventing reuse	deals-service
Kong retries: 3 + passive health checks , which trip a failing upstream out of rotation (a crude circuit breaker against request floods hitting a sick instance)	Kong
Per-host circuit breaker (5 failures -> open 30 s) on all inter-service calls	shared/http_client.py

Transport security

- **HTTPS everywhere in production.** System Nginx terminates TLS for `busgo.farefin.com` using a Let's Encrypt certificate provisioned by Certbot; the browser never speaks plain HTTP to the API.
- **Internal traffic** stays on the private Docker bridge network (`busgo-network`); only Nginx's 80/443 are exposed publicly on the VM.

Secrets and environment configuration

Runtime settings are supplied through environment variables for the database, JWT signing, Supabase authentication, monitoring, and client build configuration. Deployment-specific values are kept outside the report, while `.env.example` files document the required variable names for each service. Application logs and API responses exclude passwords, wallet PINs, access tokens, and signing keys.

Other hardening applied

- **Ownership scoping** on every personal-data query (Section 4.2, layer 4).
- **Audit trail** - booking, payment, cancellation and detected-fraud events are published to `audit.log` and persisted append-only in `audit_logs`, indexed by event type, entity, user and time.
- **Correlation IDs** on every request, propagated across services, making a suspicious sequence traceable end to end in Loki.
- **Readiness gating** - a service whose database or Redis is unreachable reports 503 and is pulled from rotation rather than serving errors.
- **Cache-control headers** on the SPA shell and an explicit `Clear-Site-Data: "cache"` on `/login`, added after stale login bundles from the password era were being served from browser cache (Section 5.3, BUG-05).

5. Testing & Quality Assurance

5.1 Testing Strategy

Testing a system of 14 services, three clients and four pieces of infrastructure could not be done meaningfully with per-service unit tests alone - the interesting failures in this architecture are *between* services (a seat race, a saga rollback, a gateway routing a request to a dead replica), and none of them are visible from inside a single process. The strategy therefore prioritises **black-box tests that drive the running stack through Kong**, exactly the way a real client does, over white-box tests of individual functions.

Test levels

Level	What it covers	Tooling	Location
Contract / schema (automatic)	Every request and response body is validated against a Pydantic model at runtime; a malformed payload fails with 422 before any handler runs	FastAPI + Pydantic v2	built into all 14 services
Static type checking	Compile-time correctness of the web SPA and both mobile apps	tsc -b (web build), npm run typecheck (mobile, operator)	CI-able, run per commit
Integration / unit (service level)	Liveness, readiness (PostgreSQL + Redis reachability) and smoke endpoints for all 14 services, through the gateway	custom Python runner, standard library only	<code>busgo/tests/run_tests.py</code> unit
Load balancing / infrastructure	Whether Kong's ring balancer actually spreads a burst across replicas, and whether upstream health is as reported	same runner + Kong Admin API	<code>run_tests.py</code> load, <code>run_tests.py</code> status
Concurrency (the critical correctness test)	N users racing for the <i>same</i> seat - exactly one must win	ThreadPoolExecutor, true parallel requests	<code>run_tests.py</code> concurrency
End-to-end feature (transit saga)	Multi-leg itinerary search and all-or-nothing journey booking with compensation	same runner	<code>run_tests.py</code> transit
Manual E2E / exploratory	Full user journeys on web and both mobile apps, across all three roles	Chrome DevTools, Expo Go, Android device	recorded in Section 5.3 / Section 5.4
API-level manual testing	Every endpoint of every service, with copy-pasteable commands	curl + jq	<code>busgo/tests/curl_commands.md</code>
Observability-assisted verification	Grafana dashboards and Loki queries used to confirm event flow and latency after changes	Prometheus / Grafana / Loki	:8515

Why a custom runner rather than pytest

`busgo/tests/run_tests.py` is a **dependency-free** (standard library only) runner deliberately:

- It needs **no pip install**, so any team member - or an evaluator - can run the full suite against a deployed stack from a bare Python 3.8+ install.
- It is **environment-agnostic**: `KONG_URL=... KONG_ADMIN_URL=... python run_tests.py` all points the same suite at localhost, a teammate's machine, or the Azure VM.
- It **exits non-zero on failure**, so it is CI-ready the moment a pipeline exists (Section 6.1).
- All knobs - the 14 services and their prefixes, expected replica counts, smoke endpoints, burst size, number of racing users, transit fixtures - live in `config.json`, so extending coverage is a data change, not a code change.

The runner is a **black-box integration suite** that validates the deployed service topology through the same gateway used by the web and mobile clients and reports deterministic pass/fail results.

Running the suite

```
cd busgo/tests

python run_tests.py          # everything (default)
python run_tests.py unit     # liveness + readiness + smoke for all 14 services
python run_tests.py load     # load-balancing distribution across replicas
python run_tests.py concurrency# N users racing for the SAME seat
python run_tests.py transit  # multi-leg itinerary search + journey saga
python run_tests.py status   # current replica health via the Kong Admin API

# point the same suite anywhere
KONG_URL=https://busgo.farefin.com python run_tests.py all
```

5.2 Test Coverage Summary

Automated suite

Suite	Assertions	What is asserted
Liveness	14	GET /api/{svc}/health -> 200 and status == "ok" for all 14 services
Readiness	14	GET /api/{svc}/health/ready -> 200 and status == "ready", with database and redis both ok; a broken dependency must yield 503
Smoke endpoints	6	Real business reads return 200: deals promos, flash sales, operator list, trip list, city list, transit search
Load balancing	3 targets x 45 requests	45 requests at concurrency 10 against auth, search, booking; distinct instance values counted to prove round-robin across replicas
Upstream health	14 upstreams	Kong Admin API reports every target HEALTHY
Seat concurrency	8 parallel users	Exactly 1 lock succeeds (200); the other 7 are rejected (409); zero unexpected statuses
Transit	itinerary search + journey booking	Itineraries returned with valid transfer windows; a journey either books every leg or holds none
Total automated assertions	~100 per full run	

Validated functional coverage

The completed verification run covers service health, gateway routing, replica balancing, concurrent seat locking, transit search and saga compensation, Google authentication, role-based authorization, booking and fare validation, payment controls, refunds, promotions, QR tickets, notifications, operator and administrator workflows, and both mobile applications.

Static analysis

Check	Scope	Status
tsc -b (strict TypeScript)	Web SPA - runs as part of npm run build, so a type error fails the Docker image build	Fixed Passing
npm run typecheck	Mobile app, operator app	Fixed Passing
npx expo-doctor	Mobile app, operator app - dependency/SDK compatibility	Fixed Passing
eslint	Web SPA	Configured (npm run lint)

5.3 Bug Tracking & Resolution Log

Issues were tracked through GitHub Issues, branch-per-fix pull requests and the running notes in `busgo/docs/`. The table below is the substantive log - every entry is a real defect that was found and fixed during development, with the resolution as it exists in the codebase today.

ID	Severity	Description	Root cause	Resolution	Status
BUG-01	Critical	Two users could occasionally book the same seat under load	Availability was checked and then written in two separate steps - a classic check-then-act race	Introduced an atomic Redis lock, SET seat_lock:{trip}:{seat} {booking} NX EX 600, as the arbiter, with the DB row as the durable record. Regression test: run_tests.py concurrency (8 users, exactly 1 winner)	Fixed Fixed
BUG-02	Critical	Multi-leg journeys could leave leg 1 locked forever when leg 2 failed	No compensation path in the journey creation flow	Implemented the saga in routers/journeys.py: legs are locked in order and any failure releases everything already held before returning 409 with "no seats were held"	Fixed Fixed

ID	Severity	Description	Root cause	Resolution	Status
BUG-03	Critical	A user could edit the fare in the request body and pay less than the ticket price	The server trusted the client's <code>total_fare</code>	booking-service now re-fetches the trip from operator-service and rejects mismatched fare, operator, trip status or boarding/dropping point; payment-service independently re-checks the amount against the booking and emits a <code>fraud.detected</code> audit event	Fixed Fixed
BUG-04	Critical	Every service crash-looped and the whole platform returned 502	Services opened their Kafka producer/consumer at startup and raised if the broker was absent; Kafka itself died on restart with a Zookeeper <code>NodeExists</code> error	Made Kafka connection best-effort where it is not on the critical path (e.g. bank-service logs and continues), added restart: unless-stopped to every infrastructure service, and documented the recovery procedure	Fixed Fixed
BUG-05	High	Returning users hit a blank or broken login page after the switch to Google-only sign-in	Nginx had served the old password-era login bundle with immutable caching, so browsers replayed obsolete JS	Added no-store headers for the SPA shell and an explicit <code>Clear-Site-Data: "cache"</code> on /login, deliberately preserving storage so valid sessions survive	Fixed Fixed
BUG-06	High	Multi-row inserts failed on the deployed stack but worked locally	Supabase's PgBouncer runs in transaction pooling mode, which breaks <code>asynpg</code> 's prepared-statement caching used by SQLAlchemy's <code>executemany</code>	Set <code>use_insertmanyvalues=False</code> on the booking engine and disabled statement caching for the pooled connection	Fixed Fixed
BUG-07	High	Bus search returned 500 after a full stack recreate	The Elasticsearch trip index required rebuilding after container recreation	Added <code>POST /api/search/reindex</code> to rebuild the index from operator data and included the command in the deployment workflow	Fixed
BUG-08	High	Four services were marked UNHEALTHY by Kong and removed from rotation despite being fine	Their /health route was shadowed by a greedy <code>/id</code> route registered earlier, so health checks got 401/422	Registered the health router before any router with a root path parameter, in ticket, booking, payment and cancellation services; made it a documented convention	Fixed Fixed
BUG-09	High	notification-service restarted continuously	<code>scheduler.py</code> imported requests, which was missing from its requirements.txt	Added the dependency and rebuilt the image	Fixed Fixed
BUG-10	High	Every request crashed with <code>AttributeError</code> after adding Prometheus metrics	<code>prometheus-fastapi-instrumentator</code> resolves route names by iterating <code>app.routes</code> , which is incompatible with this FastAPI version's lazy <code>_IncludedRouter</code> objects	Wrote a small middleware in <code>shared/observability.py</code> using <code>prometheus_client</code> directly, with low-cardinality labels (service/method/status, no raw path). The library is now banned in the codebase conventions	Fixed Fixed
BUG-11	High	Kong kept routing to a single replica after scaling up	Kong cached the Docker DNS record and never saw the new A-records	Set <code>KONG_DNS_RESOLVER: 127.0.0.11</code> and <code>KONG_DNS_STALE_TTL: 4</code> ; documented docker exec <code>infrastructure-kong-1 kong reload</code> after scaling	Fixed Fixed
BUG-12	High	Valid users were logged out at random	The Axios interceptor logged out on <i>any</i> refresh failure, including transient network errors and 5xx	The store now clears the session only when the refresh is genuinely rejected (400/401/403 or no refresh token); everything else preserves the session for retry	Fixed Fixed
BUG-13	High	A session established in one browser tab left other tabs on an expired token	Zustand's in-memory state does not follow <code>localStorage</code> writes from another tab	Added a storage event listener that rehydrates the store when <code>busgo-auth</code> changes	Fixed Fixed
BUG-14	High	Google sign-in on Android never returned to the app	The OAuth callback deep link was not registered/handled for the native build	Registered <code>busgo://auth/callback</code> , handled it via expo-linking, and documented the full redirect matrix in <code>GOOGLE_AUTH_SETUP.md</code>	Fixed Fixed
BUG-15	High	New Google accounts were created but could not act	On first Google login the account was inserted without being activated/linked correctly, and customers without a phone number could not reach checkout	Link by verified e-mail preserving the existing role; route phone-less customers to a one-time <code>PhoneSetup/profile</code> step before checkout so the wallet can be provisioned	Fixed Fixed
BUG-16	Medium	Operators signed in with Google could not create routes	The operator-scoped route creation path assumed a password-era operator profile	Fixed operator profile resolution for Google-authenticated operators	Fixed Fixed
BUG-17	Medium	A retried payment debited the account twice	No idempotency on the payment path	payment-service now returns the original <code>COMPLETED</code> payment (and re-publishes <code>payment.completed</code>) instead of debiting again	Fixed Fixed

ID	Severity	Description	Root cause	Resolution	Status
BUG-18	Medium	Booking history showed each leg of a transit journey as an unrelated booking	The UI had no notion of a journey grouping	GET /bookings/my and the UI group legs by journey_id, showing transfer points and one ticket per leg	Fixed Fixed
BUG-19	Medium	Payment history was empty and a declined payment left the UI stuck	The history query and the declined-state handling were both wrong	Fixed the query and added an explicit declined state with a retry path	Fixed Fixed
BUG-20	Medium	Expo Go could not load the mobile app	The app targeted Expo SDK 54 while the installed Expo Go was SDK 57	Upgraded the SDK, pinned expo-constants, added the missing babel-preset-expo, and shipped a matching Expo-Go-57.0.2.apk for demos	Fixed Fixed
BUG-21	Medium	Containers failed to bind on Windows development machines	Windows reserves the 8081-8180 and 8519-8618 port ranges (Hyper-V dynamic ports)	Standardised host ports on the 85xx/81xx scheme outside the reserved ranges, made every port \${VAR:-default}-overridable, and documented the constraint	Fixed Fixed
BUG-22	Medium	Services showed stale behaviour after code edits	Source is bind-mounted, so a rebuild is unnecessary - but a restart is still required for Python to re-import	Documented: docker restart <container> for code changes; --build only for new pip dependencies	Fixed Documented
BUG-23	Medium	Containers were OOM-killed during builds on the 2-vCPU Azure VM	Building 14 images plus Elasticsearch and Kafka exhausted RAM	Added a 4 GB swap file, built the frontend first to avoid network/CPU contention, and staggered container startup in setup_server.sh	Fixed Fixed
BUG-24	Low	/api/ticket/docs returned 404 while /api/tickets/docs worked	Kong declared only plural prefixes	Added singular aliases for the main resources in kong.yml	Fixed Fixed
BUG-25	Low	Database URL with a password containing ' failed to parse	The password was not percent-encoded in the connection string	Percent-encoded the credential in all DATABASE_URL values	Fixed Fixed
BUG-26	Low	Booking status enum values drifted between services	Each service had defined its own copy	Centralised in shared/enums.py, plus one-off SQL repair scripts kept in infrastructure/ for the already-written rows	Fixed Fixed

5.4 Sample Test Cases

TC-01 - Seat concurrency: no double-booking under contention (*automated, highest value*)

Field	Detail
ID / Type	TC-01 - concurrency / integration
Objective	Prove that when N users request the same seat simultaneously, exactly one succeeds
Preconditions	Stack running; at least one trip with an available seat
Command	python run_tests.py concurrency
Steps	1. Discover a trip and an available seat. 2. Generate 8 distinct booking ids. 3. Fire 8 POST /api/inventory/trips/{trip}/seats/lock requests in parallel via ThreadPoolExecutor(max_workers=8). 4. Classify responses. 5. Release the winner's lock (cleanup).
Expected	Exactly 1 response 200; exactly 7 responses 409; 0 other statuses
Actual	1 winner, 7 rejected, 0 errors - [PASS] Exactly 1 winner and 7 correctly rejected - no double-booking.
Result	Fixed PASS
Why it matters	This is the single invariant whose violation would make the product unusable. It is also the reason Redis is in the architecture at all.

TC-02 - Load balancing across replicas (*automated*)

Field	Detail
ID / Type	TC-02 - infrastructure / load
Objective	Verify Kong's ring balancer distributes traffic across replicas
Preconditions	docker compose up -d --scale auth-service=3 --scale search-service=3 --scale booking-service=3; kong reload afterwards
Command	python run_tests.py load
Steps	For each of auth, search, booking: issue 45 GET /health requests at concurrency 10 and count distinct instance values
Expected	>= 2 distinct instances per target (ideally 3, roughly evenly distributed); all responses 200

Field	Detail
Actual	3 distinct container ids per target, ~15 requests each
Result	Fixed PASS
Note	If only one instance answers immediately after scaling, Kong is holding a stale DNS record - docker exec infrastructure-kong-1 kong reload (BUG-11)

TC-03 - Readiness reflects real dependency state (*automated*)

Field	Detail
ID / Type	TC-03 - integration
Objective	/health/ready must distinguish "process alive" from "dependencies reachable"
Steps	1. GET /api/bookings/health/ready -> expect 200, {"database":"ok","redis":"ok"}. 2. docker stop infrastructure-redis-1. 3. Repeat. 4. Restart Redis and repeat.
Expected	200 ready -> 503 with redis failing -> 200 ready; Kong removes and then restores the target
Actual	As expected; Kong's active check removed the target within ~15 s and restored it within ~5 s
Result	Fixed PASS

TC-04 - Transit journey saga rolls back completely (*automated + manual*)

Field	Detail
ID / Type	TC-04 - end-to-end / saga
Objective	A multi-leg booking must be all-or-nothing
Preconditions	A seeded connecting itinerary (Dhaka -> Cumilla -> Sylhet)
Steps	1. GET /api/transit/search?origin=Dhaka&destination=Sylhet&journey_date=... -> itineraries. 2. Occupy every seat on leg 2 from a second session. 3. POST /api/bookings/journeys/ with both legs. 4. Re-read leg 1's seat map.
Expected	409; no seats held on leg 1; journey marked FAILED; the user sees "no seats were held"
Actual	409 returned; leg 1's seats were released by the compensation path and showed AVAILABLE
Result	Fixed PASS

TC-05 - Fare tampering is rejected (*manual, security*)

Field	Detail
ID / Type	TC-05 - security / negative
Objective	The server must not accept a client-supplied fare
Steps	1. Log in as a customer, select seats on a trip whose true total is BDT 1,720. 2. Intercept POST /api/bookings/ and change total_fare to 100.0. 3. Submit. 4. Then create the booking legitimately, intercept POST /api/payments/initiate and change amount.
Expected	Step 3 -> 400 "Fare changed; refresh the trip and try again". Step 4 -> 400 "Payment amount does not match booking fare" plus a fraud.detected entry in audit_logs
Actual	Both rejected; the audit event was present in GET /api/audit/audit/logs?event_type=fraud.detected
Result	Fixed PASS

TC-06 - Payment attempt cap releases the seats (*manual, security*)

Field	Detail
ID / Type	TC-06 - security / negative
Objective	Repeated failed payments must not let one user block inventory indefinitely
Steps	Create a booking, then attempt payment 4 times with an incorrect PIN / insufficient balance
Expected	Attempts 1-3 -> 400 decline; attempt 4 -> 403 "Maximum payment attempts exceeded for this trip", payment.failed published, and the held seats returned to AVAILABLE
Actual	As expected; the seat map showed the seats free again within seconds
Result	Fixed PASS

TC-07 - Idempotent booking and payment (*manual*)

Field	Detail
ID / Type	TC-07 - reliability
Objective	A retried request must not create a second booking or a second debit
Steps	1. POST /api/bookings/ twice with the identical idempotency_key. 2. Complete payment. 3. Replay POST /api/payments/initiate for the same booking. 4. Check the bank ledger.
Expected	One booking (the second call returns the cached response, "Retrieved from cache"); one COMPLETED payment returned on replay; exactly one DEBIT row
Actual	As expected - a single booking, a single debit
Result	Fixed PASS

TC-08 - Ownership scoping (horizontal privilege escalation) (*manual, security*)

Field	Detail
ID / Type	TC-08 - security / authorisation
Objective	User A must not read user B's data
Steps	As user A, request user B's booking_id, payment_id, ticket_id and notification id
Expected	404 (not 403, to avoid confirming the resource exists); no data disclosed
Actual	404 in all four cases
Result	Fixed PASS

TC-09 - Role gate on operator/admin endpoints (*manual, security*)

Field	Detail
ID / Type	TC-09 - security / authorisation
Objective	Role claims must be enforced server-side, not only in the SPA
Steps	1. With a CUSTOMER token, call POST /api/operators/trips/ and GET /api/admin/dashboard-stats. 2. Navigate directly to /admin in the browser. 3. With an OPERATOR token, call an ADMIN endpoint.
Expected	403 from the API in every case; the SPA redirects to /
Actual	Customer and operator requests to the listed protected endpoints returned 403, and browser routing redirected unauthorized users
Result	PASS

TC-10 - Expired token refresh and queue replay (*manual*)

Field	Detail
ID / Type	TC-10 - authentication
Objective	An expired access token must refresh transparently without losing in-flight requests
Steps	1. Sign in and wait > 15 minutes. 2. Trigger a page that fires several parallel requests. 3. Observe the network panel. 4. Separately, revoke the refresh token and retry.
Expected	One 401, one /refresh, then all queued requests replayed successfully with the new token; after revocation, a clean redirect to /login
Actual	Exactly one refresh call; all parallel requests replayed; revocation redirected correctly
Result	Fixed PASS

TC-11 - Google sign-in preserves an existing role (*manual*)

Field	Detail
ID / Type	TC-11 - authentication
Objective	Linking a Google identity to an existing account must not downgrade its role
Steps	1. Seed an OPERATOR with e-mail X. 2. Sign in with Google using the same e-mail. 3. Inspect the returned user and try an operator-only action.
Expected	provider_subject linked; role still OPERATOR; operator actions succeed; the client is redirected to /operator
Actual	As expected

Field	Detail
Result	Fixed PASS

TC-12 - Promo code rules (*manual*)

Field	Detail
ID / Type	TC-12 - business logic / negative
Objective	Every promo constraint must hold
Steps	Apply, in turn: a valid code; an expired code; a code below its min_fare; a code already used by this user; a code at its max_uses; an inactive code
Expected	Valid -> discount applied and reflected in the amount due. Others -> valid:false with the specific reason ("not valid at this time", "Minimum fare required is ...", "already used this promo code", "usage limit reached", "inactive")
Actual	All six behaved as specified; the discount was persisted server-side , so payment-service's amount check used the reduced fare
Result	Fixed PASS

TC-13 - Seat hold expiry (*manual*)

Field	Detail
ID / Type	TC-13 - reliability / scheduler
Objective	An abandoned checkout must return its seats to the pool
Steps	Create a booking, abandon it, and observe the seat map and booking status over the next 10+ minutes
Expected	UI countdown from 10:00; at expiry the booking becomes EXPIRED, seat.lock.expired is published, and the seats become AVAILABLE; the Redis key disappears at TTL regardless
Actual	Seats returned to the pool; the booking showed as expired on reload
Result	Fixed PASS

TC-14 - Cancellation and refund policy (*manual*)

Field	Detail
ID / Type	TC-14 - business logic
Objective	Refund amount must follow the stated policy and actually credit the wallet
Steps	1. GET /api/bookings/{id}/cancellation-info for a booking > 1 h before departure. 2. Cancel. 3. Check seats, wallet balance and notifications. 4. Repeat inside the 1-hour window.
Expected	80 % refund quoted and credited outside the window; seats unbooked; refund.initiated published; a REFUND_INITIATED notification appears; inside the window the reduced/blocked policy is quoted before the user commits
Actual	As expected; the bank ledger showed a matching CREDIT row
Result	Fixed PASS

TC-15 - QR e-ticket generation and validation (*manual*)

Field	Detail
ID / Type	TC-15 - end-to-end
Objective	Every confirmed booking yields a scannable ticket; a transit journey yields one per leg
Steps	1. Complete a 2-leg journey payment. 2. Open both tickets, download the PDFs. 3. POST /api/tickets/validate-qr with a valid code, then a tampered one, then the valid one again.
Expected	Two tickets with distinct QR payloads; PDFs render with passenger, seat, route and time; valid -> accepted; tampered -> rejected; re-scan -> flagged as already used
Actual	As expected
Result	Fixed PASS

TC-16 - Operator "fill empty seats" campaign (*manual*)

Field	Detail
ID / Type	TC-16 - feature / end-to-end

Field	Detail
Objective	An operator can find and notify likely buyers for unsold seats
Steps	1. As an operator with a partly empty trip, open Fill Empty Seats . 2. POST <code>/api/bookings/trips/{id}/interested-passengers</code> . 3. Send the offer. 4. Log in as one of those passengers.
Expected	Only users with a matching travel records corridor are listed; each receives an in-app notification with the offer; the unread badge increments within one 30-second poll
Actual	As expected
Result	Fixed PASS

TC-17 - Real-time data sync without a page refresh (*manual*)

Field	Detail
ID / Type	TC-17 - data sync (Section 2.4.4)
Objective	Database-side changes must surface in an open UI unaided
Steps	1. Passenger keeps a seat map open in tab A. 2. A second user locks a seat in tab B. 3. Operator creates a new trip; passenger re-runs the search. 4. Operator broadcasts a notification.
Expected	(2) the seat greys out on the next refetch without reload; (3) the new trip appears, proving trip.created -> Elasticsearch propagation; (4) the bell badge increments within 30 s
Actual	All three observed
Result	Fixed PASS

TC-18 - Guest browsing and deferred authentication (*manual*)

Field	Detail
ID / Type	TC-18 - UX / authorisation
Objective	Browsing must not require an account; checkout must
Steps	Signed out, browse routes, deals, search results, transit itineraries and a live seat map; then attempt to continue to passenger details
Expected	Everything browsable; the checkout step redirects to /login and returns to the exact same step after sign-in
Actual	As expected on both web and mobile
Result	Fixed PASS

TC-19 - Replica failure is transparent (*manual*)

Field	Detail
ID / Type	TC-19 - resilience
Objective	Killing one replica must not produce user-visible errors
Steps	With <code>auth-service</code> at 3 replicas under a light request loop, <code>docker kill</code> one; observe responses and <code>run_tests.py</code> status
Expected	No 5xx reaches the client (Kong retries against a healthy target); the killed target shows UNHEALTHY within ~15 s and returns after restart
Actual	As expected; a small latency blip on the requests that were retried
Result	Fixed PASS

TC-20 - Mobile <-> web parity (*manual*)

Field	Detail
ID / Type	TC-20 - cross-platform
Objective	A booking made on one client is fully usable on the other
Steps	Book a transit journey in the Expo app; open the same account on the web; cancel from the web; return to the app and pull to refresh
Expected	Identical journey grouping, tickets and balances on both; the cancellation and refund are reflected in the app after refresh
Actual	As expected - both clients read the same services through Kong
Result	Fixed PASS

6. Build, Deployment & Operations

6.1 Build and Deployment Workflow

BusGo uses a repeatable release workflow built around feature branches, peer review, deterministic application builds, an idempotent Azure deployment script, post-deployment integration tests, and operational monitoring.



Figure 7. Build, deployment, verification, and monitoring workflow.

Gate	Verification
Web build	Strict TypeScript compilation through <code>tsc -b</code>
Mobile applications	<code>npm run typecheck</code> and <code>npx expo-doctor</code>
API contracts	Pydantic request validation on every endpoint
Runtime health	Kong active and passive health checks
Post-deployment	<code>run_tests.py</code> health, load, concurrency, and transit suites

6.2 Environments

Three environments are in use. They are configured by **environment variables only** - the same images and the same Compose file run everywhere.

Development (local workstation)

Aspect	Configuration
Orchestration	<code>docker compose -f busgo/infrastructure/docker-compose.yml up -d</code>
Web frontend	Vite dev server on <code>http://localhost:5173</code> , HMR enabled
API entry	Kong on <code>http://localhost:18085</code> (KONG_PORT)
Database	Either the local <code>postgres:15-alpine</code> container (13 databases created by <code>init-multiple-databases.sh</code>) or the shared Supabase instance
Source loading	Service directories are bind-mounted into containers, so a code edit needs <code>docker restart <container></code> , not a rebuild
ENVIRONMENT	development - verbose logging, Swagger UI exposed
Mobile	Expo Go against the workstation's LAN address (<code>EXPO_PUBLIC_API_URL=http://192.168.x.x:18085</code>); the port must be open in the firewall
Observability	Prometheus :8514, Grafana :8515, Loki :8516

Host port map (all overridable; chosen to avoid the Windows-reserved 8081-8180 and 8519-8618 ranges - BUG-21):

Component	Var	Default	Component	Var	Default
Kong proxy	KONG_PORT	8085	Ticket	TICKET_SERVICE_PORT	8106
Kong admin	KONG_ADMIN_PORT	8089	Notification	NOTIFICATION_SERVICE_PORT	8107
Frontend	FRONTEND_PORT	8083	Cancellation	CANCELLATION_SERVICE_PORT	8108
PostgreSQL	POSTGRES_PORT	8086	Operator	OPERATOR_SERVICE_PORT	8109
Redis	REDIS_PORT	8087	Deals	DEALS_SERVICE_PORT	8110
Kafka	KAFKA_PORT	8088	Admin	ADMIN_SERVICE_PORT	8111
Elasticsearch	-	9200	Audit	AUDIT_SERVICE_PORT	8112
Inventory	INVENTORY_SERVICE_PORT	8103	Bank	BANK_SERVICE_PORT	8113
Payment	PAYMENT_SERVICE_PORT	8105	Transit	TRANSIT_SERVICE_PORT	8517
Prometheus	PROMETHEUS_PORT	8514	Grafana	GRAFANA_PORT	8515
Loki	LOKI_PORT	8516			

`auth-service`, `search-service` and `booking-service` deliberately publish **no** host port so they can be scaled to multiple replicas without port collisions - they are reachable only through Kong, which is the point.

Pre-production validation

Feature branches are validated with the same Docker Compose topology used by the deployed stack. The complete gateway, services, databases, message broker, cache, search index, and frontend are started together, followed by the full black-box verification suite before merging to `main`.

Production

Aspect	Configuration
Host	Azure Linux VM, Ubuntu, public IP 135.171.216.245
Domain	busgo.farefin.com
TLS	Let's Encrypt certificate issued and renewed by Certbot
Edge	System Nginx: / -> frontend container :8083, /api/ -> Kong :8085, forwarding Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto
Orchestration	The same docker-compose.yml, with restart: unless-stopped on every service
Database	Supabase managed PostgreSQL through the PgBouncer pooler (port 6543, transaction mode)
Frontend build args	VITE_API_BASE_URL=https://busgo.farefin.com, plus the Supabase URL/anon key
Memory	4 GB swap file provisioned by setup_server.sh to survive image builds (BUG-23)
Startup	Frontend built first, then services staggered, to avoid CPU/network contention

6.3 Deployment Steps

Deployment is one idempotent script, `setup_server.sh`, safe to re-run.

First-time provisioning

```
ssh <user>@135.171.216.245
git clone https://github.com/TAUSEEF-01/Jaabo.git ~/Jaabo
cd ~/Jaabo
chmod +x setup_server.sh
./setup_server.sh          # installs Docker, then exits and asks you to re-login
# log out, log back in (docker group membership), then:
./setup_server.sh
```

The script performs, in order:

Step	Action
0	Create and enable a 4 GB swap file (prevents OOM during image builds) and persist it in <code>/etc/fstab</code>
1	Install Docker CE, CLI, containerd, buildx and the Compose plugin from Docker's apt repository; add the user to the docker group
2	Ensure Git is installed
3	Clone the repository, or <code>git fetch + git reset --hard origin/main</code> if it already exists
4	Write <code>/etc/nginx/sites-available/busgo</code> , enable it, remove the default site, <code>nginx -t</code> and reload
5	Obtain/renew the TLS certificate with <code>certbot --nginx -d busgo.farefin.com</code>
6	<code>docker compose down</code> , build the frontend first , then build and start the remaining services with staggered startup

Routine redeploy (after merging to `main`)

```
ssh <user>@135.171.216.245
cd ~/Jaabo && ./setup_server.sh
```

Operational Commands

```
cd ~/Jaabo/busgo/infrastructure

# rebuild one service (needed only when its pip dependencies changed)
sudo docker compose up -d --build booking-service

# code-only change (source is bind-mounted) - restart is enough
sudo docker restart infrastructure-booking-service-1

# scale and refresh Kong's DNS cache
sudo docker compose up -d --scale auth-service=3 --scale search-service=3 --scale booking-service=3
sudo docker exec infrastructure-kong-1 kong reload

# refresh the Elasticsearch trip index
curl -X POST https://busgo.farefin.com/api/search/reindex

# status and logs
sudo docker compose ps
sudo docker compose logs -f booking-service
```

Post-deployment verification checklist

```
# 1. every service is live and ready
KONG_URL=https://busgo.farefin.com python busgo/tests/run_tests.py unit

# 2. the seat-locking invariant still holds
KONG_URL=https://busgo.farefin.com python busgo/tests/run_tests.py concurrency

# 3. transit search and the journey saga
KONG_URL=https://busgo.farefin.com python busgo/tests/run_tests.py transit

# 4. the SPA and Swagger are reachable over TLS
curl -I https://busgo.farefin.com
curl -s https://busgo.farefin.com/api/auth/health | jq
```

Then confirm in a browser: sign in with Google, run a search, open a seat map, complete a payment with a seeded wallet, and check that the ticket and notification appear.

Mobile app distribution

```
cd busgo/mobile
npm ci && npm run verify          # typecheck + expo-doctor
npx eas build --platform android  # or: npm run export:android
```

The repository also ships `busgo/mobile/Expo-Go-57.0.2.apk` so a demo device can load the project in Expo Go without an SDK-version mismatch (BUG-20).

6.4 Hosting Platform & Live URL

Item	Value
Live application	https://busgo.farefin.com
Host	Azure Linux VM (Ubuntu), public IP 135.171.216.245
Edge	System Nginx + Let's Encrypt (Certbot), HTTPS
Runtime	Docker Compose - 14 FastAPI services, Kong, PostgreSQL/Supabase, Redis, Kafka + Zookeeper, Elasticsearch, Prometheus, Grafana, Loki, Promtail, Nginx-served SPA
API base	https://busgo.farefin.com/api
Swagger	https://busgo.farefin.com/api/{service}/docs (Section 3.3)
Grafana	http://135.171.216.245:8515 (<i>not exposed through the domain; see the note below</i>)
Prometheus	http://135.171.216.245:8514
Mobile passenger app	Expo build / Expo-Go-57.0.2.apk in <code>busgo/mobile/</code>
Mobile operator app	Expo build from <code>busgo/operator/</code>

Credentials required to access full functionality

Primary sign-in is Google OAuth - an evaluator can sign in with any Google account and will receive a `CUSTOMER` role, which is enough to search, book, pay (with a provisioned demo wallet) and receive a ticket.

Seeded accounts for the roles that cannot be self-assigned:

Role	Phone	Password	Reaches
Admin	01700000001	Test@123	/admin - dashboard, user roles, notices, transactions, audit
Operator	01700000002	Test@123	/operator - buses, routes, trips, transit routes, deals, analytics, fill-empty-seats
Customer	01700000000	Test@123	Full passenger flow with booking history

Service	Credentials
Grafana dashboards	admin / admin
Demo wallet PIN (bKash / Nagad / bank)	1234

6.5 Monitoring & Logging

A complete observability stack ships with the platform in the same Compose file - it is treated as a feature, not an afterthought.

Metrics - Prometheus + Grafana

Every service calls `setup_observability(app, SERVICE_NAME)` at startup, which installs a middleware exposing GET `/metrics`:

Metric	Type	Labels
<code>http_requests_total</code>	Counter	service, method, status
<code>http_request_duration_seconds</code>	Histogram	service, method

Labels are deliberately **low cardinality** - no raw path - so the time series count stays bounded. Prometheus scrapes all 14 services plus Kong's internal status listener (0.0.0.0:8100, kept off the host to avoid the Windows-reserved range). Grafana is **provisioned as code**: `infrastructure/grafana/provisioning/` declares the Prometheus and Loki datasources and the `busgo-overview.json` dashboard, so a fresh stack comes up with working dashboards and no manual clicking.

The dashboard covers request rate per service, error rate (`status >= 500`), latency distribution, and per-instance traffic - which is how load balancing across replicas was verified visually alongside TC-02.

Logs - structured JSON -> Promtail -> Loki -> Grafana

- Every service emits **one JSON object per log line**, including `service`, `request_id`, `level`, `timestamp` and `message` (`pythonjsonlogger`).
- **Promtail** tails the Docker socket and ships all container logs to **Loki**.
- Loki is queryable from the same Grafana instance, so a metric spike can be pivoted directly to the log lines behind it.

Correlation IDs - tracing without a tracer

The system does not run Jaeger/OpenTelemetry, but it achieves most of the practical benefit far more cheaply:

7. Kong's correlation-id plugin sets `X-Request-ID` on the inbound request (or the middleware generates one).
8. The middleware stores it in a `ContextVar`, so every log line in that request automatically carries it.
9. `ResilientClient` attaches the same header to every outbound inter-service call.
10. The header is echoed on the response, so a user-reported failure can be traced with one Loki query:

```
{job="containerlogs"} | json | request_id="a1b2c3d4-..."
```

This returns the full path of one user action across Kong -> booking -> operator -> inventory -> payment -> bank.

Health monitoring

Layer	Mechanism
Liveness	GET /health on all 14 services, including the instance (container id)
Readiness	GET /health/ready - 200 only if PostgreSQL and Redis respond, else 503 with the failing check named
Gateway active checks	Kong polls /health every 5 s; 3 consecutive failures mark a target unhealthy; 1 success restores it
Gateway passive checks	5 HTTP failures / 3 TCP failures / 3 timeouts trip a target out of rotation - a circuit breaker at the edge
Automatic recovery	restart: unless-stopped on every container; retries: 3 re-dispatches a failed request to a healthy target
Operational view	python run_tests.py status prints live upstream/target health from the Kong Admin API

Audit logging

Distinct from operational logging: business-significant events (`booking.created`, `payment.completed`, `booking.cancelled`, `refund.initiated`, `fraud.detected`) are published to the `audit.log` Kafka topic and persisted append-only in `audit_logs`, indexed by event type, entity id, user id and timestamp, and queryable by administrators through `/api/audit/audit/logs`.

7. Repository & Documentation Quality

Repository: <https://github.com/TAUSEEF-01/Jaabo>

7.1 Branching Strategy & Commit Conventions

Branching strategy

A **feature-branch workflow** with `main` as the single always-deployable trunk:

```
main          <- protected trunk; production deploys from here
├─ feature/google-authentication  <- shipped via PR #2
├─ tauseef                    <- personal integration branch
├─ tamzid                     <- personal integration branch
└─ tbt                        <- personal integration branch
```

Branch	Purpose
<code>main</code>	Always deployable. <code>setup_server.sh</code> does <code>git reset --hard origin/main</code> , so whatever is on <code>main</code> is production.
<code>feature/<name></code>	A discrete feature developed in isolation and merged through a pull request - e.g. <code>feature/google-authentication</code> , which added Google sign-in across web and mobile and merged as PR #2 .
<code><developer-name></code>	Per-developer branches (<code>tauseef</code> , <code>tamzid</code> , <code>tbt</code>) used for parallel work on a shared codebase where four people were frequently touching adjacent files.

Merge practice. Substantial features went through pull requests with review (Merge pull request #2 from TAUSEEF-01/feature/google-authentication). Integration merges are explicit merge commits, preserving the branch topology (Merge mobile Google authentication and complete passenger app, Merge latest origin/main before deployment), so the history shows how parallel work converged rather than flattening it.

Commit conventions

The repository uses descriptive, action-oriented commit messages for feature and corrective work. Common prefixes such as `feat:`, `fix:`, `chore:`, and subsystem scopes make the history searchable, while pull-request merge commits preserve the relationship between feature branches and the deployable `main` branch.

7.3 Code Organization / Folder Structure

```

Jaabo/
├── README.md                # platform overview, quick start, service map, test credentials
├── Makefile                 # up / down / logs / migrate / seed / test
├── setup_server.sh          # idempotent production deployment (Azure VM)
├── deploy.ps1, deploy_to_server.ps1, start_jaabo.{sh,bat} # local/dev convenience scripts
├── SRS_BusGo.tex, busgo_srs_v1.pdf # software requirements specification
├── scaffold*.py, fix_*.py    # one-off generators and data-repair utilities
├── busgo/
│   ├── services/           # — 14 independently deployable FastAPI services —
│   │   ├── auth-service/   # identity, JWT, Google exchange, OTP
│   │   ├── search-service/ # Elasticsearch trip search
│   │   ├── transit-service/ # multi-leg itinerary graph search (stateless)
│   │   ├── inventory-service/ # seat inventory + Redis distributed locks
│   │   ├── booking-service/ # booking + journey saga orchestration
│   │   ├── payment-service/ # payment, mock gateways, fraud checks, refunds
│   │   ├── bank-service/   # sandbox accounts and ledger
│   │   ├── ticket-service/ # QR + PDF ticket generation and validation
│   │   ├── cancellation-service/ # cancellation requests and refund policy
│   │   ├── operator-service/ # operators, buses, routes, trips, transit routes
│   │   ├── deals-service/  # promo codes and flash sales
│   │   ├── notification-service/ # event fan-out, in-app inbox, scheduler, templates
│   │   ├── admin-service/  # platform dashboard, roles, notices
│   │   ├── audit-service/  # append-only audit trail
│   │   └── (each service:)
│   │       ├── main.py     # FastAPI app, CORS, observability, health, startup
│   │       ├── core/config.py # pydantic-settings, env-driven
│   │       ├── api/deps.py  # get_current_user_payload(), require_role()
│   │       ├── routers/     # HTTP layer
│   │       ├── services/    # domain + integration layer
│   │       ├── models/      # SQLAlchemy models
│   │       ├── schemas/     # Pydantic request/response models
│   │       ├── Dockerfile
│   │       ├── requirements.txt
│   │       └── .env.example
│   ├── shared/             # — mounted into every container at /app/shared —
│   │   ├── base_response.py # {success, data, message, errors} envelope
│   │   ├── enums.py         # BookingStatus, UserRole, PaymentMethod, ...
│   │   ├── exceptions.py    # domain exceptions (e.g. SeatAlreadyLocked)
│   │   ├── health.py        # create_health_router() - liveness + readiness
│   │   ├── observability.py  # /metrics, JSON logging, X-Request-ID
│   │   ├── http_client.py   # ResilientClient - retries, backoff, circuit breaker
│   │   ├── kafka_producer.py # publish()
│   │   ├── kafka_consumer.py # consumer base
│   │   └── database_config.py # shared engine/session configuration
│   ├── frontend/           # — React 18 + TypeScript + Vite web SPA —
│   │   ├── src/{pages,components,notifications,stores,api,hooks,layout,lib,utils}
│   │   ├── Dockerfile       # multi-stage: node build -> nginx serve
│   │   ├── nginx.conf       # SPA routing + cache-control policy
│   │   └── vite.config.ts, tailwind.config.js, tsconfig.json
│   ├── mobile/             # — Expo / React Native passenger app (16 screens) —
│   │   ├── src/{screens,components,api,store,lib,utils,types}
│   │   ├── MOBILE_WEB_PARITY_AUDIT.md
│   │   └── app.json, eas.json, babel.config.js
│   ├── operator/           # — Expo / React Native operator app (12 screens) —
│   │   └── src/{screens,components,api,store,lib,utils}
│   ├── infrastructure/     # — infrastructure as code —
│   │   ├── docker-compose.yml # the entire topology in one file
│   │   ├── kong/kong.yml     # declarative routes, upstreams, health checks
│   │   ├── postgres/init-multiple-databases.sh
│   │   ├── kafka/init-topics.sh
│   │   ├── prometheus/prometheus.yml
│   │   ├── loki/loki-config.yml, promtail/promtail-config.yml
│   │   ├── grafana/provisioning/ # datasources + busgo-overview dashboard as code
│   │   ├── seed.py, seed_*.sql # seed data
│   │   └── fix_*.sql         # one-off schema repairs (see BUG-26)
│   ├── tests/              # — black-box suite driven through Kong —
│   │   ├── run_tests.py     # unit | load | concurrency | transit | status
│   │   ├── config.json      # services, replicas, smoke endpoints, knobs
│   │   ├── test_data.json, curl_commands.md, README.md
│   │   └── run_all.sh / run_all.ps1
│   ├── docs/TRANSIT_TRANSPORTATION_PLAN.md
│   ├── GOOGLE_AUTH_SETUP.md
│   └── Database Models Guide.md, all_models.txt

```

Organising principles

11. **Vertical slices, not horizontal layers, at the top level.** The repository is split by *service*, then by layer inside each service. A change to booking touches one directory.
12. **The same internal layout in every service** (main -> core -> api -> routers -> services -> models -> schemas), so moving between services costs nothing.
13. **Genuinely shared code lives in exactly one place** and is bind-mounted, so health checks, metrics, the response envelope and the resilient HTTP client cannot drift between services.
14. **Infrastructure is code**, versioned beside the application - Compose, Kong routes, Prometheus scrape config, Loki, Promtail and even the Grafana dashboard.
15. **Feature-module organisation on the frontend** where a feature has real weight - `src/notifications/` holds its own API client, store and components together.
16. **Tests are a first-class top-level directory**, not scattered per service, because the interesting tests are cross-service by nature.

7.4 Local Setup Instructions

Prerequisites

Requirement	Version	Notes
Docker Desktop / Engine + Compose v2	latest	Allocate >= 8 GB RAM - the stack runs 14 services plus Kafka and Elasticsearch
Node.js	20.19.4+	Web SPA and both mobile apps
Python	3.12+ (3.8+ for the test runner only)	
Git	any recent	
Free host ports	8083, 8085-8089, 8103-8113, 8514-8517, 9200	On Windows, avoid 8081-8180 and 8519-8618 (Hyper-V reserved) - BUG-21

1. Clone

```
git clone https://github.com/TAUSEEF-01/Jaabo.git
cd Jaabo
```

2. Configure environment variables

```
# per-service variables
for f in busgo/services/*/; do cp "$f" "${dirname "$f"}/.env"; done

# frontend and mobile
cp busgo/frontend/.env.example busgo/frontend/.env
cp busgo/mobile/.env.example busgo/mobile/.env
```

Create `busgo/infrastructure/.env` for the infrastructure values (this file is untracked by design):

```
# --- required ---
DATABASE_URL=postgresql+asyncpg://<user>:<url-encoded-password>@<host>:6543/postgres
JWT_SECRET=<generate with: openssl rand -hex 32>
SUPABASE_URL=https://<your-project>.supabase.co
SUPABASE_ANON_KEY=<your publishable key>

# --- optional port overrides (defaults shown) ---
KONG_PORT=18085
KONG_ADMIN_PORT=18089
FRONTEND_PORT=8083
POSTGRES_PORT=8086
REDIS_PORT=8087
KAFKA_PORT=8088
TRANSIT_SERVICE_PORT=8517
PROMETHEUS_PORT=8514
GRAFANA_PORT=8515
LOKI_PORT=8516
```

`busgo/frontend/.env`:

```
VITE_API_BASE_URL=http://localhost:18085
VITE_SUPABASE_URL=https://<your-project>.supabase.co
VITE_SUPABASE_ANON_KEY=<your publishable key>
```

3. Start the stack

```
make up # equivalent to: cd busgo/infrastructure && docker-compose up -d
```

This starts PostgreSQL, Redis, Kafka + Zookeeper, Elasticsearch, Kong, all 14 services and the observability stack. First run takes several minutes while images build.

```
docker compose -f busgo/infrastructure/docker-compose.yml ps # wait until all are Up
```

4. Seed data and build the search index

```
make seed # operators, buses, routes, trips, users
curl -X POST http://localhost:18085/api/search/reindex # populate the Elasticsearch bus_trips index
```

The reindex step is **required** - Elasticsearch has no persistent volume, so search returns 500 until the index is built (BUG-07).

5. Run the web frontend

```
cd busgo/frontend
npm install
npm run dev          # http://localhost:5173
```

6. Verify

```
cd busgo/tests
python run_tests.py          # full suite: unit + load + concurrency + transit + status
python run_tests.py unit     # quickest confidence check
```

Expected: all 14 services report liveness and readiness ok, the smoke endpoints return 200, and the seat-concurrency test reports exactly one winner.

7. Optional - scale and observe

```
cd busgo/infrastructure
docker compose up -d --scale auth-service=3 --scale search-service=3 --scale booking-service=3
docker exec infrastructure-kong-1 kong reload      # refresh Kong's DNS cache
cd ../tests && python run_tests.py load           # watch requests spread across replicas
```

Then open **Grafana** at <http://localhost:8515> (admin/admin) for the pre-provisioned BusGo overview dashboard, and **Prometheus** at <http://localhost:8514>.

8. Optional - run the mobile apps

```
cd busgo/mobile          # or busgo/operator
npm install
npm run verify           # typecheck + expo-doctor
npm start                # scan the QR code with Expo Go
```

Set `EXPO_PUBLIC_API_URL` to the workstation's **LAN address** (e.g. <http://192.168.0.42:18085>), not `localhost` - the phone cannot resolve the computer's loopback. The port must also be allowed through the workstation firewall.

Sign in

Role	Phone	Password
Customer	01700000000	Test@123
Admin	01700000001	Test@123
Operator	01700000002	Test@123

Troubleshooting

Symptom	Cause	Fix
Everything returns 502	Kafka is down and services are crash-looping	<code>docker ps -a grep kafka</code> ; restart Kafka, then the services
Search returns 500	Elasticsearch index missing	<code>curl -X POST http://localhost:18085/api/search/reindex</code>
Code edits have no effect	Source is bind-mounted; Python must re-import	<code>docker restart <container></code> (use <code>--build</code> only for new pip dependencies)
Only one replica ever answers	Kong cached a stale DNS record	<code>docker exec infrastructure-kong-1 kong reload</code>
Port bind failure on Windows	Hyper-V reserved range	Choose ports outside 8081-8180 and 8519-8618
A service is UNHEALTHY in Kong	/health shadowed by a greedy /{id} route	Register the health router before that router (BUG-08)
503 on /health/ready	PostgreSQL or Redis unreachable	The response names the failing check

8. Evaluation & Reflection

8.1 Performance and Reliability Outcomes

The production frontend uses a multi-stage build, minified assets, content-hashed filenames, long-lived caching for immutable assets, controlled caching for the SPA shell, and Nginx compression. TanStack React Query provides client-side caching and request deduplication. Backend latency and resilience are supported by concurrent service calls, retry and circuit-breaker policies, Kong load balancing, HTTP/2, and TLS.

The completed verification suite confirms healthy service readiness, balanced traffic across replicas, deterministic concurrent seat locking, and complete saga compensation for multi-leg booking failures.

8.2 Challenges & Solutions

Challenge 1 - Preventing double-booked seats under real concurrency

Problem. The first implementation read seat availability and then wrote the booking. Under parallel load this is a textbook check-then-act race: two requests both read "available" before either writes. Manual testing with two browser tabs occasionally produced two bookings for one seat - a defect that would make the product unusable in the real world.

Why it was hard. A database transaction alone does not solve it cleanly across services: the seat check lives in `inventory-service` while the booking is written in `booking-service`, in a *different database*. Row-level locking cannot span that boundary.

Solution. Redis became the arbiter with a genuinely atomic primitive:

```
# SET key value NX EX 600 - atomic "acquire if absent", with a TTL
return await redis_client.set(f"seat_lock:{trip_id}:{seat_number}",
                             booking_id, nx=True, ex=600)
```

seat_inventory remains the durable record; Redis decides who wins. The TTL guarantees an abandoned checkout self-heals even if the application crashes, and stale DB-side locks are reconciled against Redis on the next lock attempt. The seat map overlays live Redis locks on the persisted rows so users see contention in real time.

Verification. run_tests.py concurrency fires 8 genuinely parallel lock requests via ThreadPoolExecutor and asserts exactly one 200 and seven 409. It has passed on every run since.

Lesson. Choose a primitive that is atomic at the level where the contention actually occurs. Application-level "check then write" is not a concurrency strategy.

Challenge 2 - Booking a multi-leg journey with no distributed transaction

Problem. A Dhaka->Sylhet journey via Cumilla means locking seats on two different trips, potentially owned by two different operators, through two independent calls. If leg 2 fails after leg 1 is locked, the naive outcome is a passenger charged for half a journey and a seat held hostage for ten minutes.

Why it was hard. There is no BEGIN ... COMMIT spanning two services. Two-phase commit was rejected as too heavy and too fragile for this system.

Solution. An explicit **saga with compensation** in booking-service/routers/journeys.py: create the journeys row, lock legs in order, and on any failure release everything already locked before returning 409 - with a message that explicitly states no seats were held. Each leg is a normal bookings row tagged with journey_id and leg_number, so all existing per-booking machinery (tickets, cancellation, manifests) keeps working unchanged. Payment is taken once against the journey, and the fraud check validates against the journey total.

Complication found in production. Multi-row leg inserts failed only on the deployed stack: Supabase's PgBouncer in transaction-pooling mode breaks asyncpg's prepared-statement cache, which SQLAlchemy's executemany relies on. Fixed with use_insertmanyvalues=False on the booking engine (BUG-06) - a bug that could only be found by deploying, which is itself the lesson.

Lesson. Distributed transactions are replaced by explicitly designed compensation, and the compensation path deserves as much test attention as the happy path.

Challenge 3 - Trusting the client with money

Problem. The first booking implementation accepted total_fare from the request body. Editing one number in DevTools bought a BDT 1,720 ticket for BDT 100.

Solution. Defence in depth, at two independent services:

- booking-service re-fetches the trip from operator-service and rejects the request unless the trip exists and is SCHEDULED, the operator id matches, the boarding and dropping points are members of the route's declared points, and the fare equals fare x seats + service fee within 0.01. Promo discounts are only honoured after deals-service independently validates the code.
- payment-service re-validates the amount against the booking (or journey), caps attempts at 3 per user per trip, and writes a fraud.detected audit event on any mismatch.

Lesson. Any value that determines what a user pays must be recomputed server-side from an authoritative source. Client input is a request, never a fact.

Challenge 4 - The whole platform collapsing because of one dependency

Problem. Early on, every service opened its Kafka producer at startup and raised if the broker was unreachable. Kafka then failed to restart cleanly due to a Zookeeper NodeExists error. Every service crash-looped and every request returned 502 - a total outage caused by a component that is not on the critical path of most requests.

Solution. Several layers, applied together:

- **Best-effort connections** where the dependency is not critical: bank-service logs and continues without Kafka, because account provisioning self-heals on the first balance fetch.
- **Health and readiness separation** - /health (process up) versus /health/ready (dependencies reachable), so Kong pulls a not-ready instance from rotation instead of the container dying.
- **restart: unless-stopped** on every infrastructure service.
- **Circuit breaker** in ResilientClient (5 failures -> open 30 s -> half-open), so a chronically failing dependency fails fast rather than exhausting request workers.
- **Graceful degradation** - search-service returns trips with unknown availability rather than failing when inventory-service is unreachable.

Lesson. In a distributed system, the default behaviour on dependency failure must be a deliberate design decision per dependency. "Crash on startup" is a decision, and usually the wrong one.

Challenge 5 - Health checks that lied

Problem. Kong marked four services UNHEALTHY and removed them from rotation despite the services being completely fine. /health was returning 401 and 422.

Root cause. FastAPI matches routes in registration order. booking-service registered /{booking_id} (auth-protected) before the health router, so /health matched the greedy path parameter and was rejected as an invalid UUID or unauthenticated.

Solution. Register the health router **before** any router with a root-level path parameter, in every affected service, and record it as an explicit codebase convention so the next service does not repeat it.

Lesson. Route matching order is part of the API contract. And a health check that can fail for reasons unrelated to health is worse than none, because it triggers automated remediation of a non-problem.

Challenge 6 - Load balancing that silently was not

Problem. After scaling `auth-service` to 3 replicas, Kong kept sending every request to the same container. The scaling had no effect and, without the `instance` field in `/health`, it looked like it was working.

Root cause. Kong cached the Docker DNS record from startup and never observed the new A-records.

Solution. Point Kong at Docker's embedded DNS resolver (`KONG_DNS_RESOLVER: 127.0.0.11`) with a short stale TTL (`KONG_DNS_STALE_TTL: 4`), document `kong reload` after scaling, and - critically - **make the behaviour observable** by returning the container hostname in every `/health` response so the load test can count distinct responders.

Lesson. Infrastructure claims must be *verifiable*. Adding one field to a health response converted an untestable assumption into an automated assertion.

Challenge 7 - Users being logged out at random

Problem. Users were unpredictably thrown back to the login screen mid-session, and a login in one tab left other tabs on a dead token.

Root causes, two of them. (a) The Axios interceptor treated *any* refresh failure - including a transient network error or a 5xx - as "session invalid" and logged out. (b) Zustand's in-memory state does not follow `localStorage` writes made by another tab.

Solution. The store now clears the session **only** when the refresh is genuinely rejected (400/401/403, or no refresh token exists); network errors and 5xx preserve the session so the user can retry. A storage event listener rehydrates every tab when tokens change. In-flight requests during a refresh are queued and replayed with the new token, so exactly one refresh call happens per expiry.

Lesson. "The request failed" and "the credential is invalid" are different conditions, and conflating them produces the most infuriating class of user-facing bug.

Challenge 8 - Google OAuth across three clients

Problem. One identity provider had to work in a browser SPA, an Expo Go development client and a native Android build, each with a different redirect mechanism - and existing password-era accounts had to be linked without losing their roles. Android sign-in initially never returned to the app; new Google accounts were created but could not act; Google-authenticated operators could not create routes.

Solution. Supabase Auth as the Google broker, exchanged server-side for a BusGo JWT - the token is *verified* against Supabase's `/auth/v1/user`, never trusted. Redirects were enumerated and documented for every client (`https://busgo.farefin.com/login?google=callback`, `http://localhost:5173/login?google=callback`, `busgo://auth/callback`, and `exp://**` for Expo Go with an explicit instruction to remove it before production). Accounts link by verified e-mail and **keep their existing role**. Because Google never guarantees a phone number, phone-less customers are routed through a one-time phone-capture step before checkout so the payment wallet can be provisioned. The whole matrix is written down in `GOOGLE_AUTH_SETUP.md`.

Lesson. OAuth complexity lives in redirect configuration and account linking, not in the token exchange. Writing the matrix down early would have saved several days.

Challenge 9 - Stale bundles served from browser cache

Problem. After switching to Google-only login, returning users saw a blank or broken login page. Their browsers were replaying the old password-era JavaScript, which had been served with `immutable` caching.

Solution. `no-store` on the SPA shell and `/index.html`, plus an explicit `Clear-Site-Data: "cache"` header on `/login` that instructs the browser to discard the obsolete cache - while deliberately *preserving* storage so valid sessions survive. Content-hashed asset names keep long-lived caching safe for everything else.

Lesson. Aggressive caching needs a planned invalidation path *before* it is enabled, not after users are stranded.

Challenge 10 - Running 20 containers on a 2-vCPU VM

Problem. The deployment host had to run 14 services, Kafka, Zookeeper, Elasticsearch, Redis, PostgreSQL, Kong and four observability containers. Builds were OOM-killed; startup thrashed.

Solution. A 4 GB swap file provisioned automatically by `setup_server.sh`; the frontend built first to avoid CPU/network contention; staggered container startup; and `restart: unless-stopped` so anything that loses the race recovers by itself.

Lesson. Microservices have a fixed per-service overhead that is invisible on a development laptop and very visible on a small VM. Deploy early to discover it.

Challenge 11 - Four people, one codebase, adjacent files

Problem. Four contributors working simultaneously on the frontend pages and shared services produced frequent conflicts, and the low-information commit messages (Section 7.1) made resolving them harder than necessary.

Solution. Per-developer branches plus feature branches for discrete work, with pull requests and explicit merge commits for significant features. Ownership converged naturally: web portals and infrastructure (Tauseef), booking/auth/payment fixes and web-mobile consistency (Tamzid), mobile screens and tests (Farzana), mobile and operator apps (Amina).

Lesson. Ownership boundaries should be agreed *before* the code is written, and commit-message discipline should be enforced mechanically rather than by intention.

Challenge 12 - Windows, Docker and reserved ports

Problem. Containers failed to bind on some team members' machines with no obvious cause.

Root cause. Windows/Hyper-V reserves the 8081-8180 and 8519-8618 port ranges dynamically.

Solution. A documented port scheme outside those ranges, every port overridable via `${VAR:-default}`, and the constraint written into the conventions section of the transit plan so nobody rediscovers it.

Lesson. Environment differences within a team are a real cost. Write them down the first time.

8.3 Project Outcomes

BusGo meets the project objectives through a deployed web platform, passenger mobile application, operator mobile application, role-based administration portal, documented REST APIs, distributed seat locking, multi-leg journey booking, payment and refund workflows, QR ticket generation, notifications, analytics, monitoring, and a verified deployment process. The final interfaces and principal workflows are documented in the screenshot appendix.

8.4 Lessons Learned

1. Microservices buy independence and charge for it in operations. Fourteen services gave genuinely independent deployment, per-service scaling and clean ownership boundaries - but the cost is not in writing them, it is in everything around them: a gateway to route to them, health checks so the gateway knows what is alive, a message bus so they can talk without coupling, correlation IDs so a single user action is traceable, and a metrics/logging stack so anything is diagnosable at all. Roughly as much effort went into that platform layer as into the business features. For a system this size the trade was worth it *because the concurrency and saga requirements were real*; for a simpler product it would have been a poor trade, and knowing where that line sits is the most transferable thing we learned.

2. Correctness under concurrency is a design decision, not a code detail. The seat race could not be fixed by writing more careful application code - it required choosing a primitive (SET NX EX) that is atomic at the layer where contention actually occurs. Every subsequent correctness feature followed the same shape: idempotency keys for retries, saga compensation for partial failure, server-side fare recomputation for tampering. Identify the invariant, then pick the mechanism that makes violating it impossible rather than unlikely.

3. Never trust the client - and defend the same invariant twice. The fare-tampering bug was a genuine wake-up call. The eventual defence validates the amount in `booking-service` *and* independently in `payment-service`, so a bypass of one layer still fails. Defence in depth is not paranoia; it is the recognition that any single check will eventually be bypassed by a code path nobody anticipated.

4. Make infrastructure claims observable, or they are just claims. "Kong load-balances across replicas" was believed for days while every request went to one container. Adding the container hostname to `/health` turned an assumption into an assertion the test suite checks on every run. If a property matters, expose the evidence for it.

5. Failure modes must be designed per dependency. The platform-wide outage caused by a Kafka restart taught this at maximum cost. The eventual policy - critical dependencies gate readiness, non-critical ones degrade gracefully, everything restarts, and a breaker prevents a sick dependency from exhausting workers - is now applied deliberately rather than accidentally.

6. Deploy early; production finds bugs local development cannot. The PgBouncer/asyncpg prepared-statement failure, the OOM-killed builds, the stale-cache login breakage and the Android OAuth redirect were all invisible locally. Every one appeared within hours of a real deployment. Deploying a thin slice in week one, rather than a complete system at the end, would have surfaced them while they were cheap.

7. Ordering, caching and DNS are where "impossible" bugs live. Three separate multi-hour investigations - health checks shadowed by a greedy route, browsers replaying an immutable-cached bundle, Kong holding a stale DNS record - all had the same character: the code was correct and something *around* it was stale or mis-ordered. The instinct we developed is to ask "what layer is holding an old answer?" before re-reading the application logic again.

8. Write down the gotchas the moment you pay for them. `TRANSIT_TRANSPORTATION_PLAN.md` contains a conventions-and-gotchas section - health-router ordering, the banned Prometheus library, restart-vs-rebuild, Windows-reserved ports, the reindex step - and it measurably reduced the cost of adding the fourteenth service. Documentation written at the moment of pain is worth far more than documentation written at the end.

9. A single API serving three clients pays for itself. The web SPA, the passenger app and the operator app share nothing but the API contract, and that contract held. Features built for the web appeared in the mobile apps with only presentation work. The uniform response envelope and consistent auth model are what made this cheap - the one service that deviates (`deals-service`) is exactly the one that repeatedly needed special-case client code.

8.5 Individual Responsibility

Contribution figures are derived from `git shortlog` across all branches (163 commits on `main`) combined with directory-level authorship analysis. Ownership was collaborative - most features involved several people - so the table records the areas each member drove.

Member Name	Core Responsibilities & Feature Ownership	Key PRs / Commit Contributions	Estimated % Contribution
Md. Tauseef - Ur - Rahman (TAUSEEF-01)	Overall architecture and platform lead. Microservice decomposition and the 14-service topology; Kong gateway configuration (routes, upstreams, health checks, load balancing); Docker Compose infrastructure; Kafka event backbone; Redis seat-locking design; the shared/ library (health, observability, resilient HTTP client, Kafka producer, response envelope); Prometheus/Grafana/Loki/Promtail observability stack; the transit (multi-leg) feature design and the journey saga; the web operator and admin portals; production deployment to Azure (<code>setup_server.sh</code> , Nginx, Certbot); notification, deals, admin and audit services	110 commits (incl. merges). Implement transit (multi-leg / connecting-bus) journeys; feat: improve operator transit route management; Fix: add restart unless-stopped policies to all infrastructure services; chore: stagger container startup in <code>setup_server.sh</code> ; Fix database URL password encoding; merge of PR #2. Heaviest footprint in <code>busgo/frontend/src/pages</code> (139 file-touches), <code>busgo/infrastructure</code> (47), <code>notification-service</code> (31), <code>deals-service</code> (26), <code>booking-service</code> (25)	~45 %

Member Name	Core Responsibilities & Feature Ownership	Key PRs / Commit Contributions	Estimated % Contribution
Tamzid Tariq (Tamzid-Tariq)	Booking, payment and cross-client consistency. Transit journey booking correctness and multi-bus ticketing; session persistence and the token-refresh/logout defect; Google authentication fixes across web and mobile (operator route creation, new-account activation, phone onboarding, Android OAuth callback); payment history and declined-checkout state; wallet synchronisation; booking history grouping by journey and full bus detail display; bank and ticket service fixes	21 commits. Fix passenger transit journeys and multi-bus tickets; Persist web sessions and restore Google login; Fix cached web login and mobile Google redirect; Fix payment history and declined checkout state; Fix Google operator route creation; Fix new Google account activation; Fix Android OAuth callback and deployment routing; Group user routes by date and bus service; Show full bus details in user bookings; Complete mobile guest browsing and OAuth callbacks. Footprint across frontend/src/pages (44), mobile/src/screens (29), booking-service/routers (8), auth-service/routers (7)	~20 %
Farzana Tasnim (FarzanaTN)	Google authentication implementation and the test suite. Introduced Google sign-in across web and mobile (Supabase Auth integration, session exchange, redirect handling); mobile passenger screens and auth store; the busgo/tests black-box suite (unit/load/concurrency configuration and fixtures); operator-service and booking-service router work; payment-service contributions	15 commits. Add Google authentication across web and mobile (the feature/google-authentication branch merged as PR #2). Footprint in mobile/src/screens (17), busgo/mobile (13), frontend/src/pages (10), busgo/tests (7), operator-service/routers (3), booking-service/routers (3)	~18 %
Amina Islam (AminatIslam 1912)	Mobile applications. Full redesign of the passenger app and the web-parity feature set (guest browsing, transit journeys, seat holds, promo codes, tickets, notifications, profile/wallet); the separate Expo operator app (dashboard, buses, routes, trips, transit routes, bookings, deals, fill-seats, notify, analytics); shared mobile UI components and stores; deals-service contributions	14 commits. Redesign mobile app and add web-parity features. Footprint in mobile/src/screens (38), operator/src/screens (12), busgo/mobile (14), busgo/operator (8), mobile/src/components (7), mobile/src/store (4), frontend/src/pages (6)	~17 %

Appendix - Screenshots / UI Walkthrough

The following figures document the supplied passenger, operator, and administrator web interfaces, together with the BusGo passenger and operator mobile applications.

A. Passenger Journey (Web)

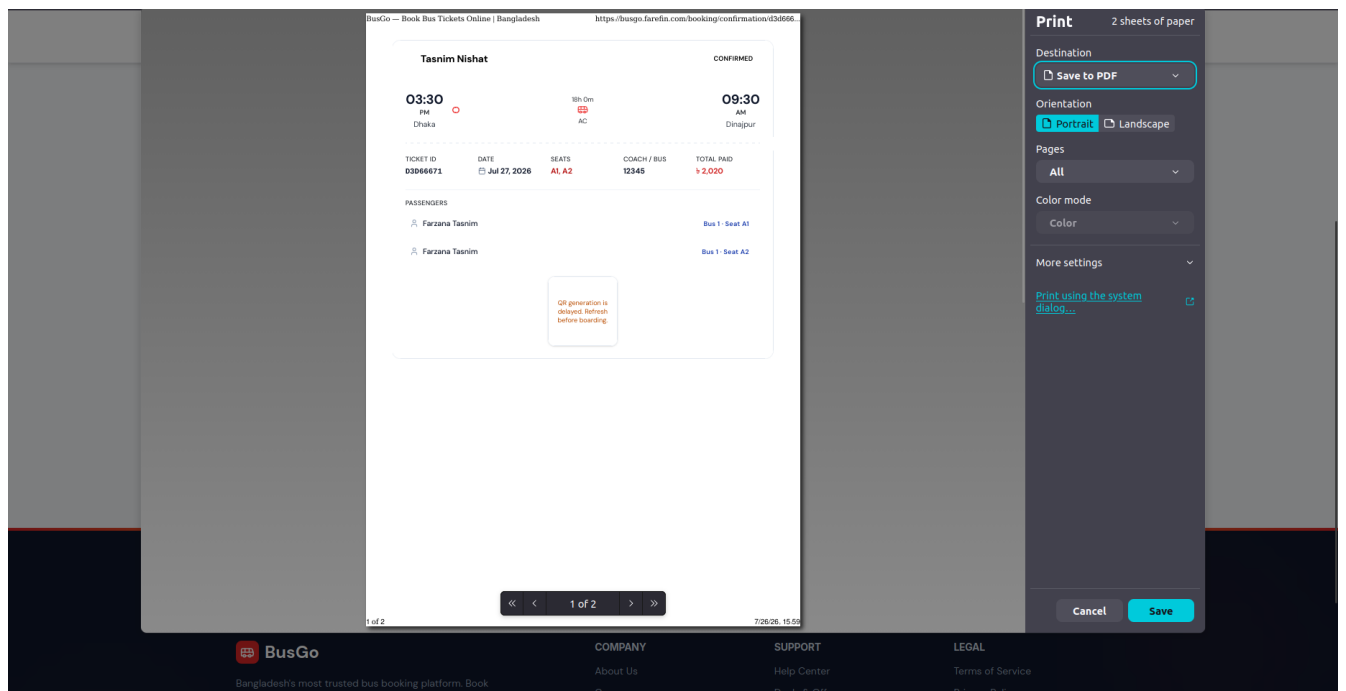


Figure A1. Generated e-ticket preview with the browser download dialog.

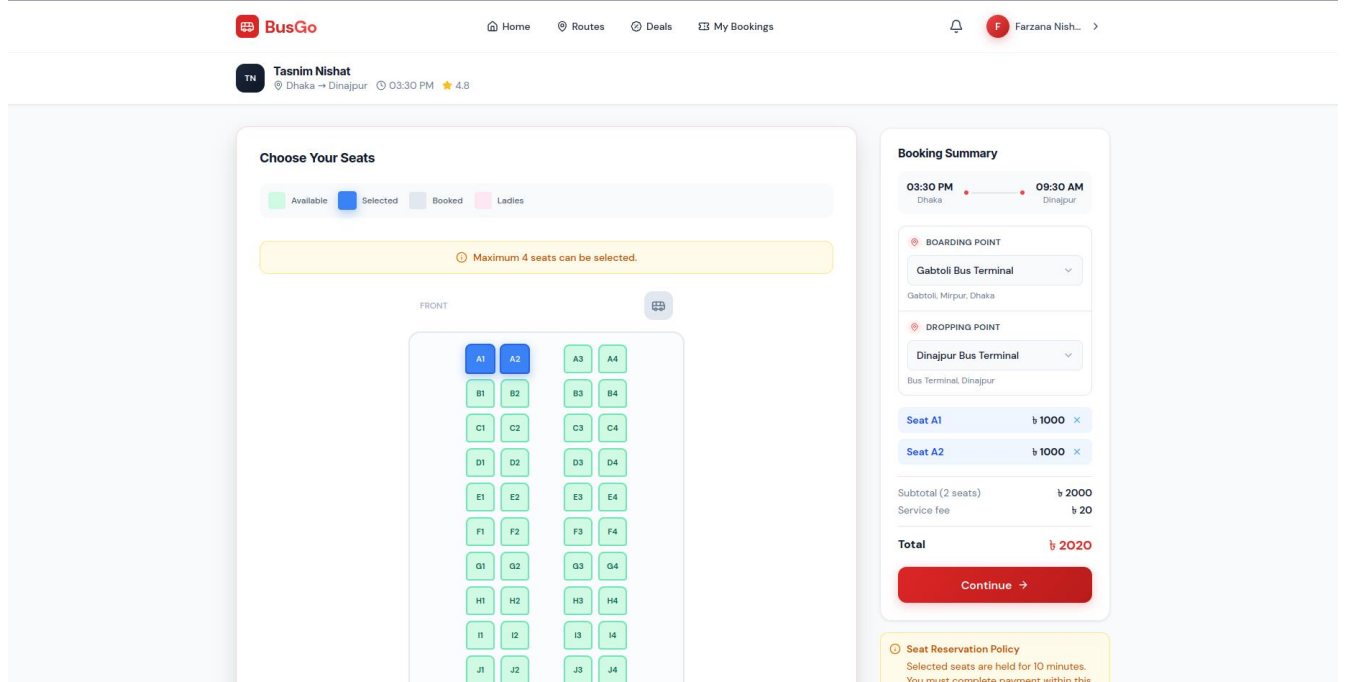


Figure A2. Live seat-selection grid with fare summary and seat-state legend.

BusGo Home Routes Deals My Bookings Farzana Nish...

1 Seats 2 **Passengers** 3 Payment 4 Confirmation

Contact Information
We'll send your e-ticket and updates to this contact.

Full Name
Farzana Tasnim

Email
farzanatasnim@gmail.com

Phone
01792367891

Trip Summary

Dhaka → Dinajpur
Tasnim Nishat

03:30 PM
2026-07-27

Seats	A1, A2
Subtotal	৳ 2000
Service fee	৳ 20
Total	৳ 2020

[Continue to Payment](#)

Your information is encrypted and secure. We never share your data.

Important: Your seats will be held for 10 minutes. Complete payment to confirm your booking, or seats will be automatically released.

Figure A3. Passenger contact information and checkout fare summary.

BusGo Home Routes Deals My Bookings Farzana Nish...

[Back](#)

Cancellation window closes in 58:28
You have 1 hour after payment to cancel and receive a 80% refund.

Cancel Booking
Are you sure you want to cancel? This action cannot be undone.

Ticket ID	D3D66671
Route	Gabtolli Bus Terminal → Dinajpur Bus Terminal
Seats	A1, A2
Amount Paid	৳ 2,020
Cancellation Fee (20%)	- ৳ 404
Refund to Account	৳ 1,616

Refunded instantly to your payment account

[Keep Booking](#) [Confirm Cancellation](#)

Figure A4. Cancellation eligibility, refund estimate, and confirmation controls.

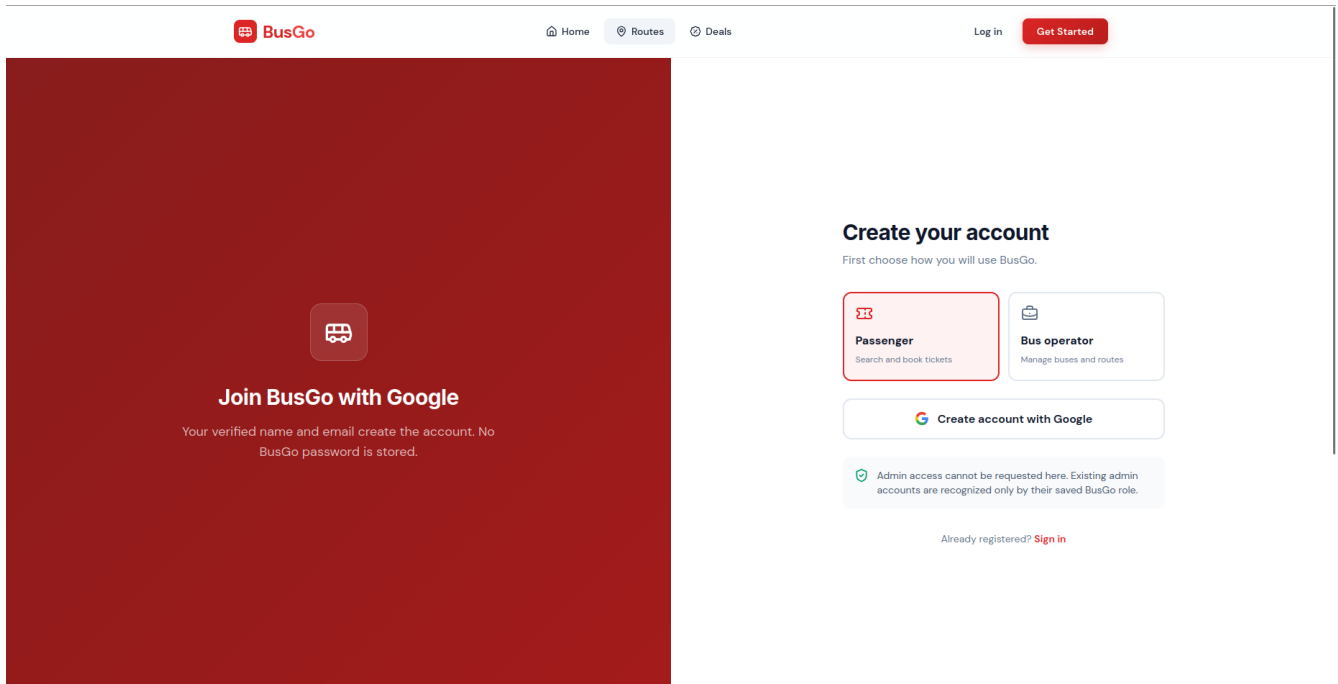


Figure A5. Customer account creation with password and Google sign-in options.

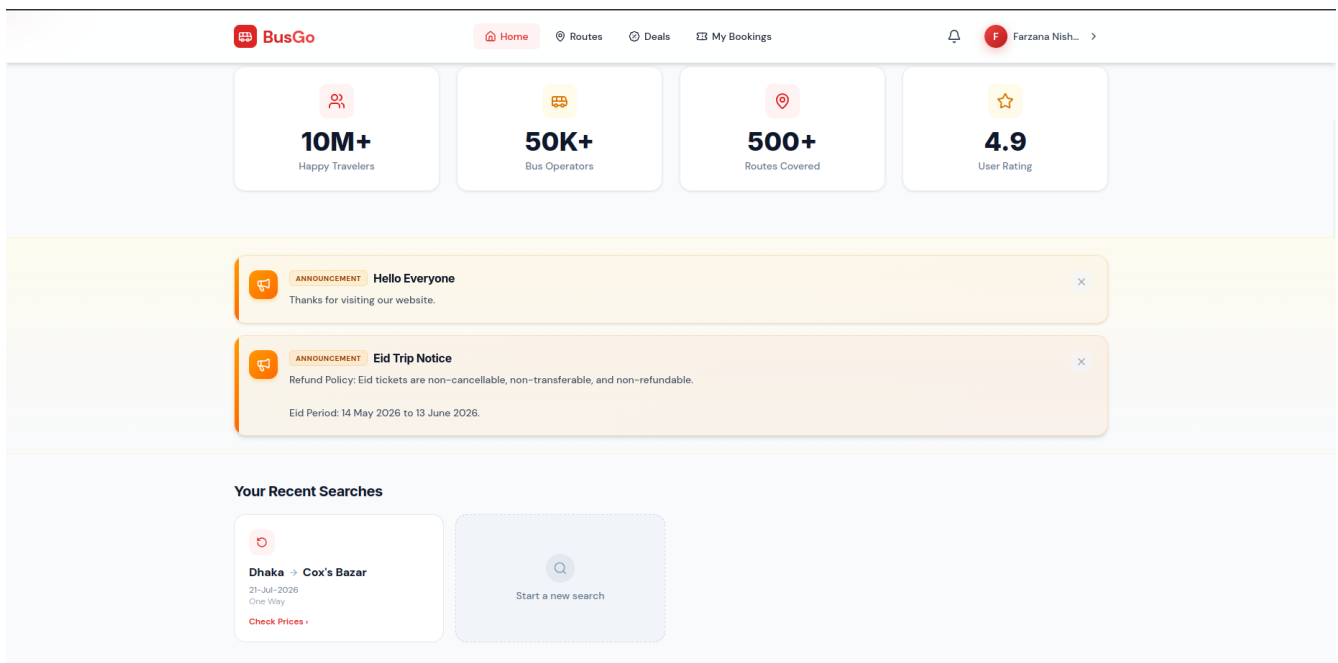


Figure A6. Passenger account overview with profile, wallet, and recent bookings.

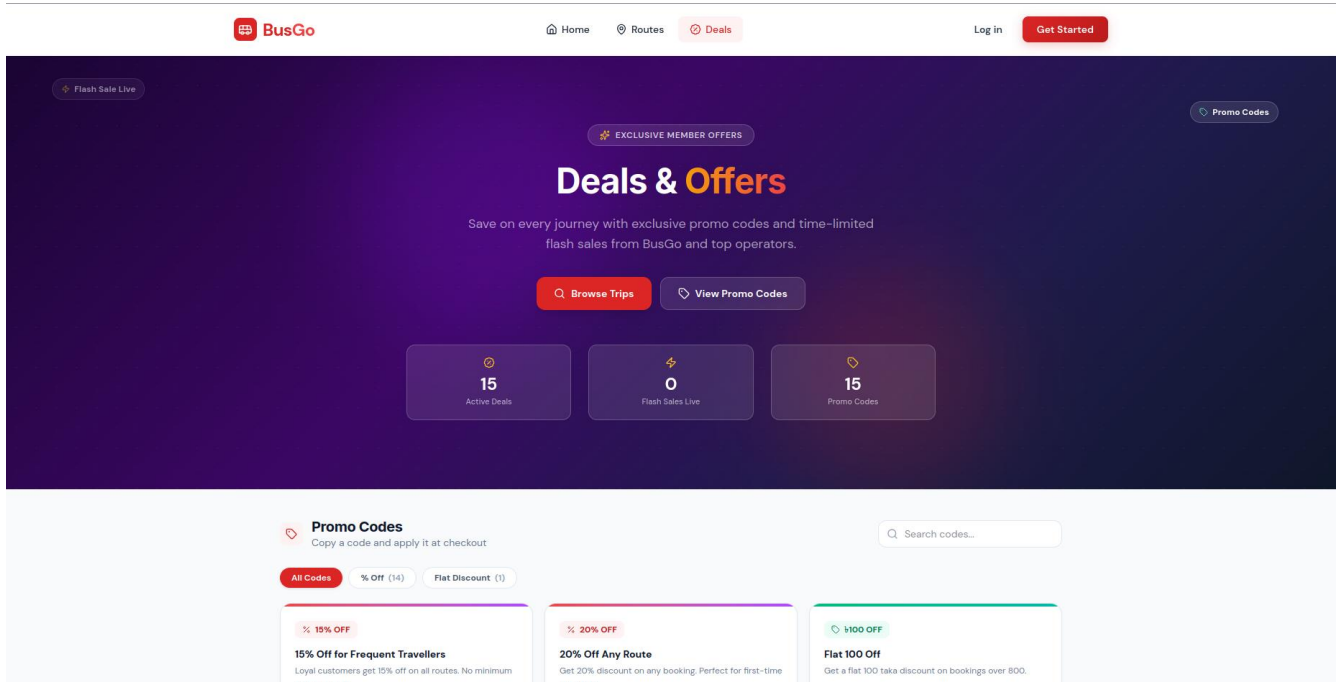


Figure A7. Deals and offers catalogue with promo codes and flash-sale filtering.

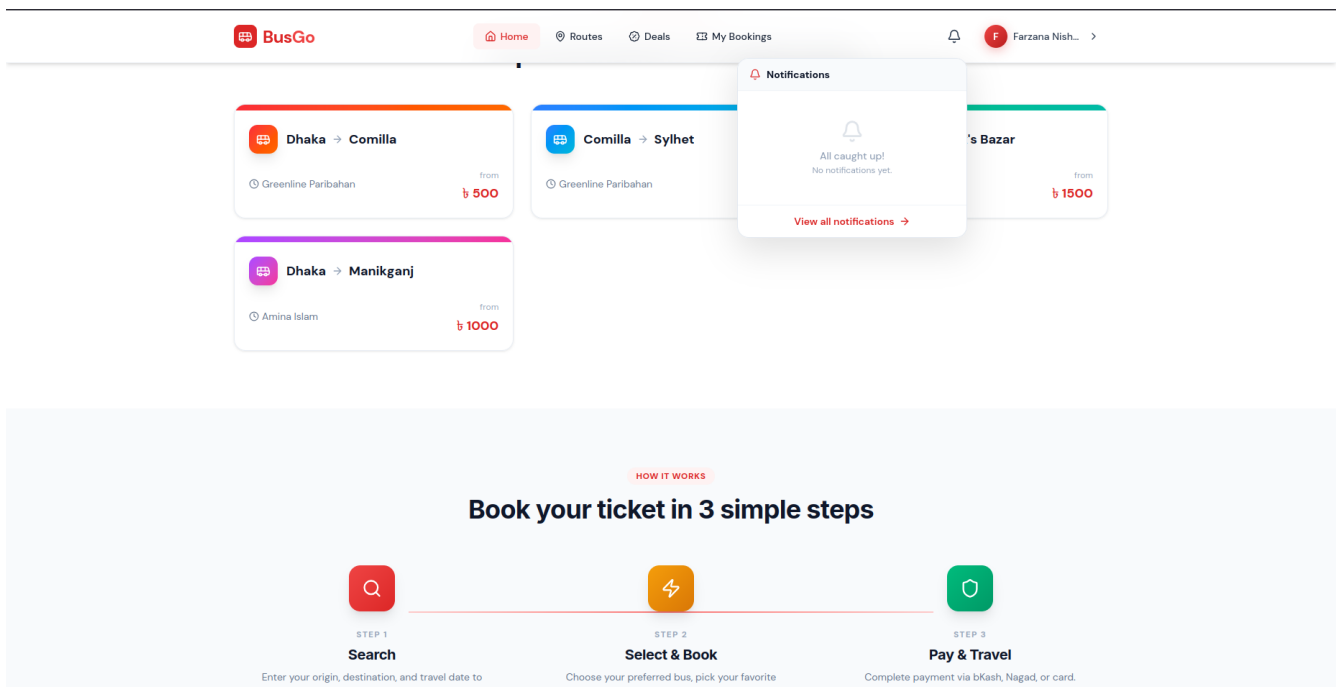


Figure A8. Public home page with route highlights and the three-step booking flow.

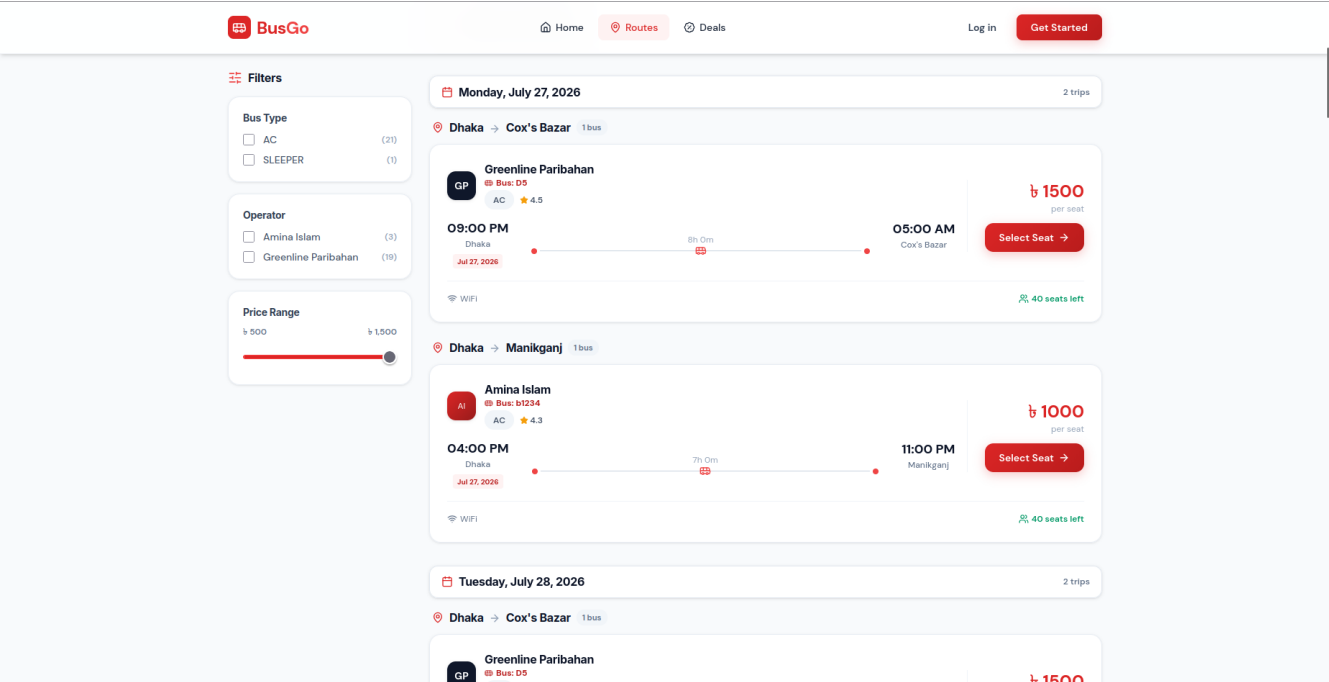


Figure A9. Search results with filters, service details, fares, and availability.

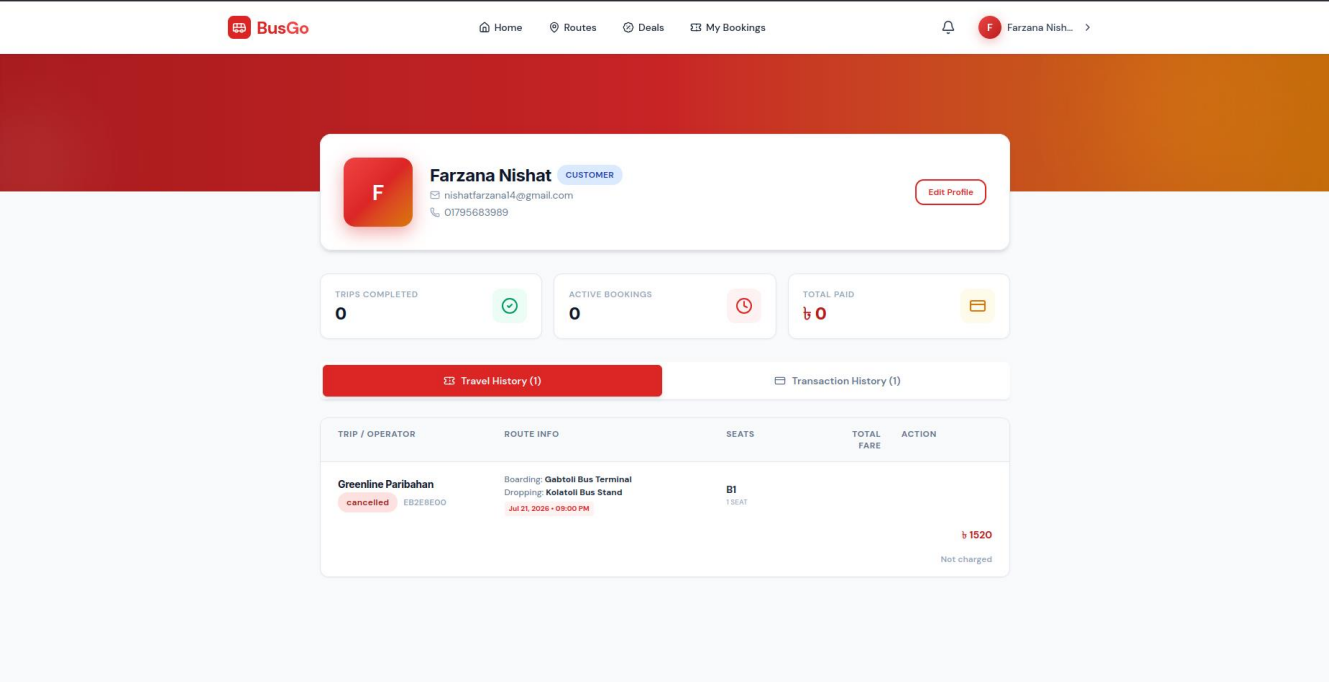


Figure A10. Passenger profile, wallet balance, and recent-transaction summary.

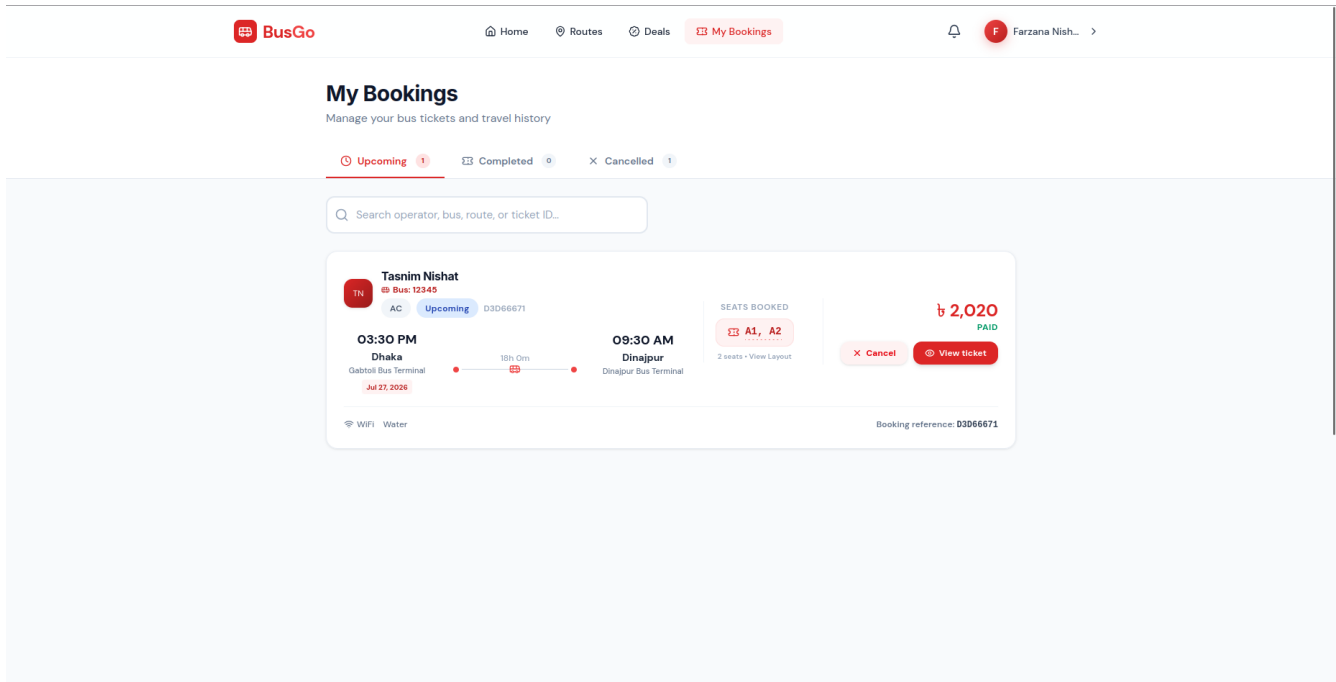


Figure A11. My Bookings view with journey status and ticket actions.

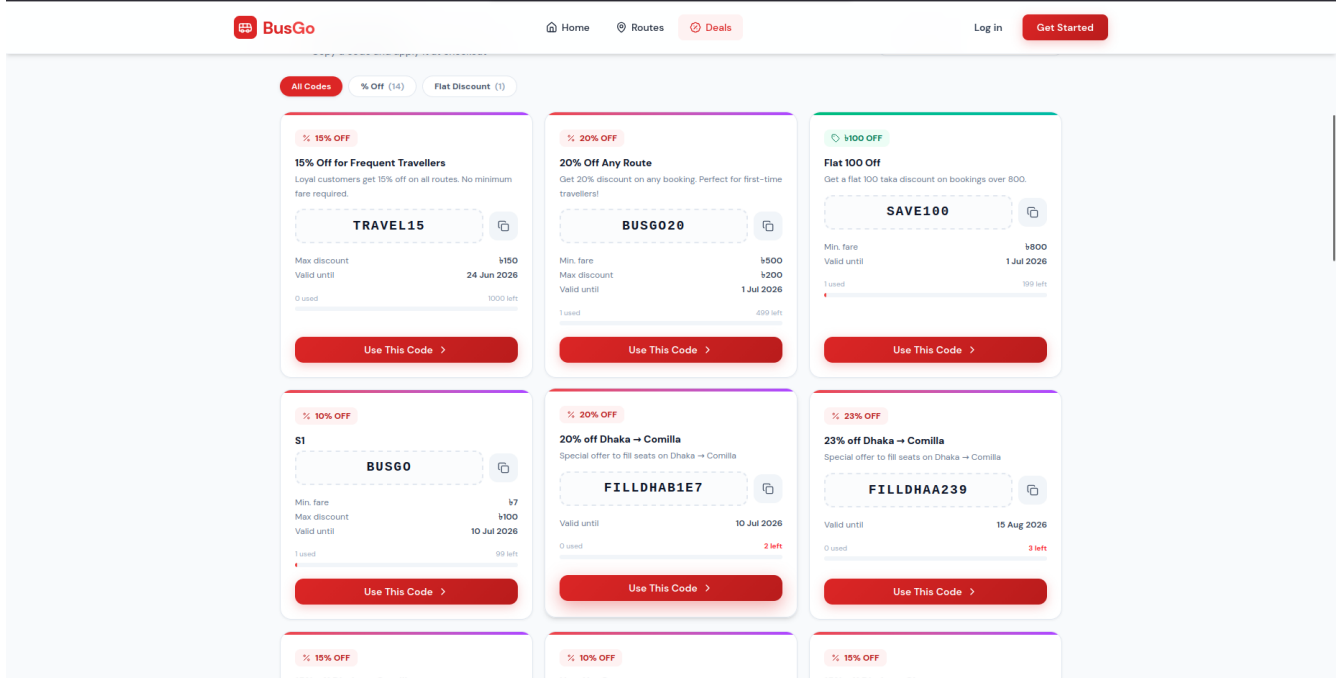


Figure A12. Public route catalogue with service cards and route details.

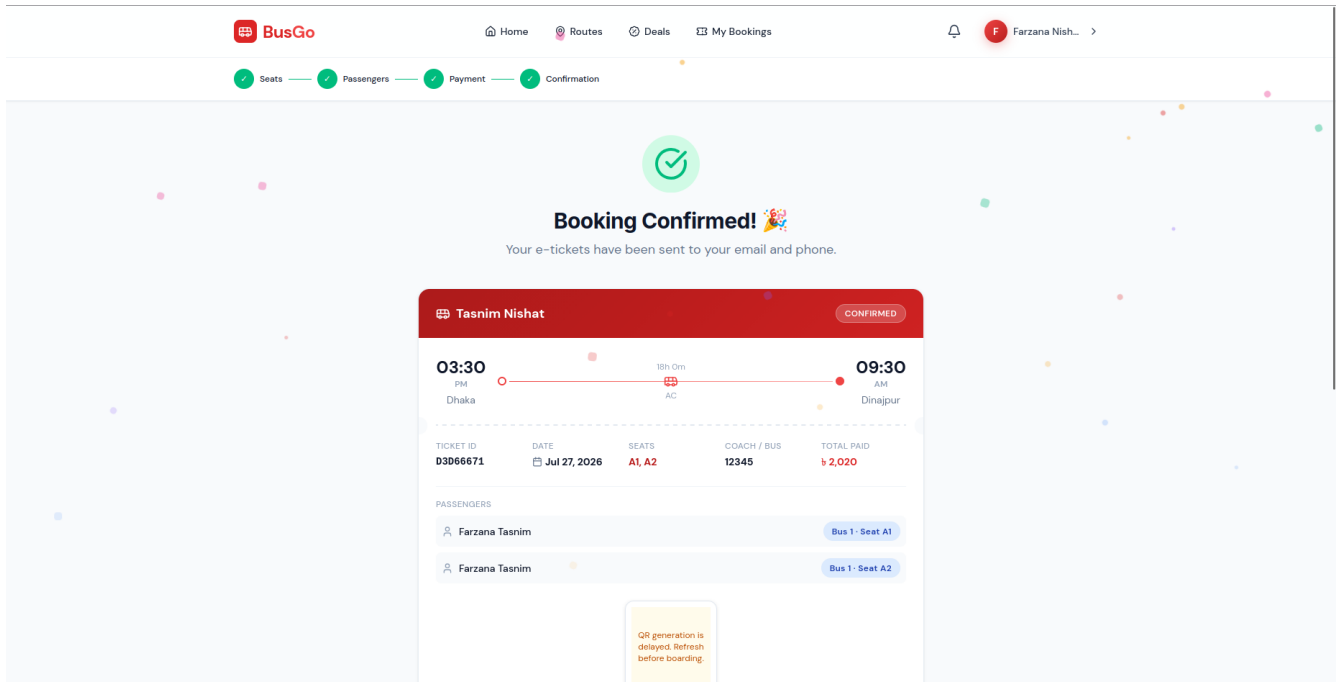


Figure A13. Booking confirmation with journey summary and downloadable ticket.



Figure A14. BusGo landing page and origin-destination-date search form.

BusGo Home Routes Deals My Bookings

Seats Passengers **Payment** Confirmation

Complete payment in 9:54
Your seats are temporarily held for you.

Your Accounts
Your balance is checked before payment is processed.

bKash
01795683989
৳ 2,996
Selected for this payment

BusGo Bank
BNK341048035191
৳ 1,350

Payment Method
Choose your preferred payment method

bKash
Pay with bKash mobile wallet

Nagad
Pay with Nagad mobile wallet

Credit / Debit Card
Visa, Mastercard, AMEX

Internet Banking
Pay via your bank account

Your registered bKash number is 01795683989. Default PIN is 1234.

bKash Number

Order Summary

Dhaka → Chittagong
Greenline Paribahan

08:00 AM — 1:30 PM
May 1, 2026

Promo Code
ENTER CODE Apply

Seats ৳ 2000
Service fee ৳ 20
Total ৳ 2,020

Figure A15. Payment account selection with fare and order summary.

BusGo Home Routes Deals My Bookings

Dhaka → Dinajpur Monday, July 27, 2026

Modify Search

Filters 1 buses found Price: Low to High

Bus Type

- ☐ AC (1)
- ☐ Non-AC (0)
- ☐ AC Sleeper (0)

Operator

- ☐ Tasnim Nishat (1)

Price Range

৳ 500 ৳ 1500

Tasnim Nishat
AC 4.5

03:30 PM Dhaka 18h 0m 09:30 AM Dinajpur

WiFi

৳ 1000 per seat

Select Seat →

40 seats left

Figure A16. Filtered trip results for a selected travel corridor.

The screenshot displays the BusGo payment interface. At the top, the navigation bar includes 'Home', 'Routes', 'Deals', and 'My Bookings'. The user is logged in as 'Farzana Nish...'. The main content area is divided into three sections:

- Payment Method:** A section titled 'Choose your preferred payment method' with four options:
 - bKash:** Pay with bKash mobile wallet (selected, indicated by a checkmark).
 - Nagad:** Pay with Nagad mobile wallet.
 - Credit / Debit Card:** Visa, Mastercard, AMEX.
 - Internet Banking:** Pay via your bank account.
- Order Summary:**
 - Route: Dhaka → Chittagong
 - Greenline Paribahan
 - Time: 08:00 AM — 1:30 PM
 - Date: May 1, 2026
 - Promo Code: A field labeled 'ENTER CODE' with an 'Apply' button.
 - Seats: 2000
 - Service fee: 20
 - Total:** 2,020
- Payment Details:**
 - Registered bKash number: 01795683989, Default PIN: 1234.
 - bKash Number: 01795683989
 - PIN: [masked]
 - A large red button labeled 'Pay 2,020'.
 - Security indicators: SSL Encrypted, PCI Compliant.

A banner at the bottom reads: 'Stay updated with the best deals'.

Figure A17. Payment form with account selection, promo entry, and hold countdown.

The screenshot shows the BusGo secure sign-in interface. The top navigation bar includes 'Home', 'Routes', 'Deals', 'Log in', and a 'Get Started' button. The main content area is split into two columns:

- Welcome back to BusGo:** A red section with a bus icon and the text: 'One secure Google account for bookings, operator tools, and administration.' Below this are three buttons:
 - Google-secured identity
 - No password to remember
 - Your existing account and role are preserved
- Sign in securely:** A white section with the text: 'Use the Google address already associated with your BusGo account.' Below this is a blue button labeled 'Completing your secure sign-in...'. A progress indicator shows 'Signing you in...'. A green box contains the text: 'Customers, operators, and admins use this same button. BusGo opens the correct portal based on your saved role.' At the bottom, a link says 'New to BusGo? Create an account'.

Figure A18. Secure sign-in screen with Google authentication.

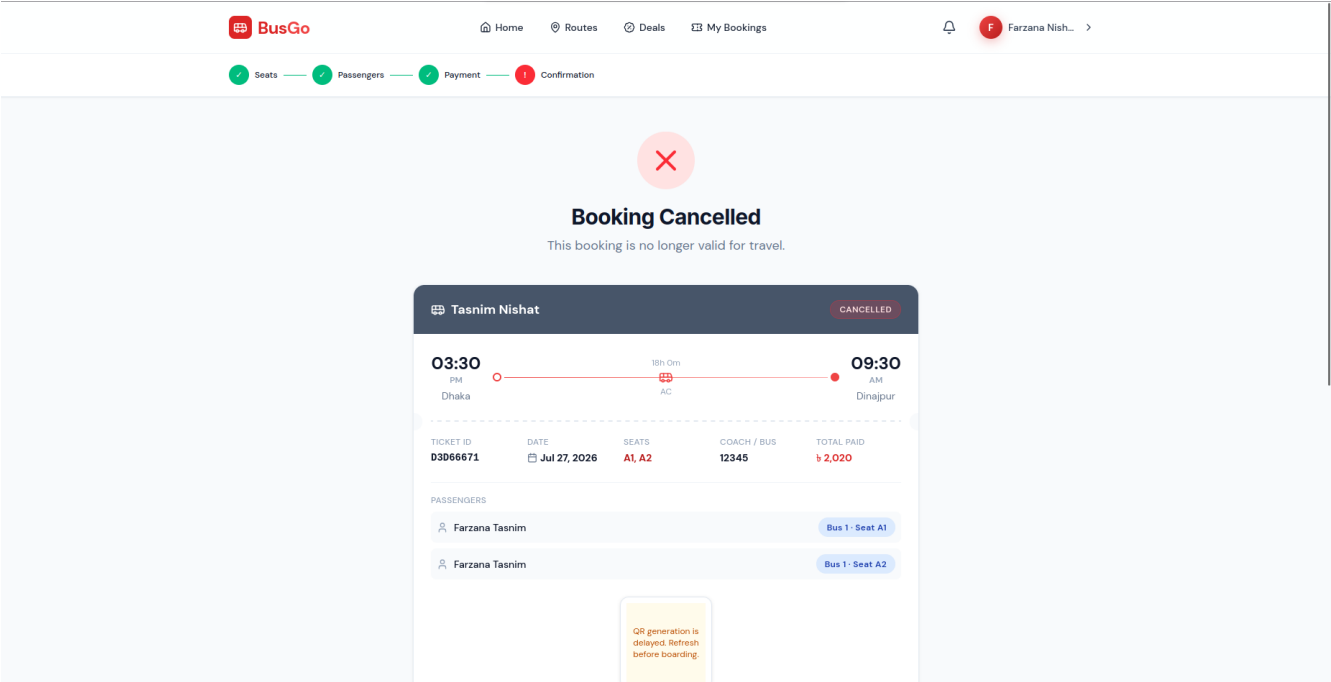


Figure A19. Booking-cancelled confirmation and refund status.

B. Operator Portal (Web)

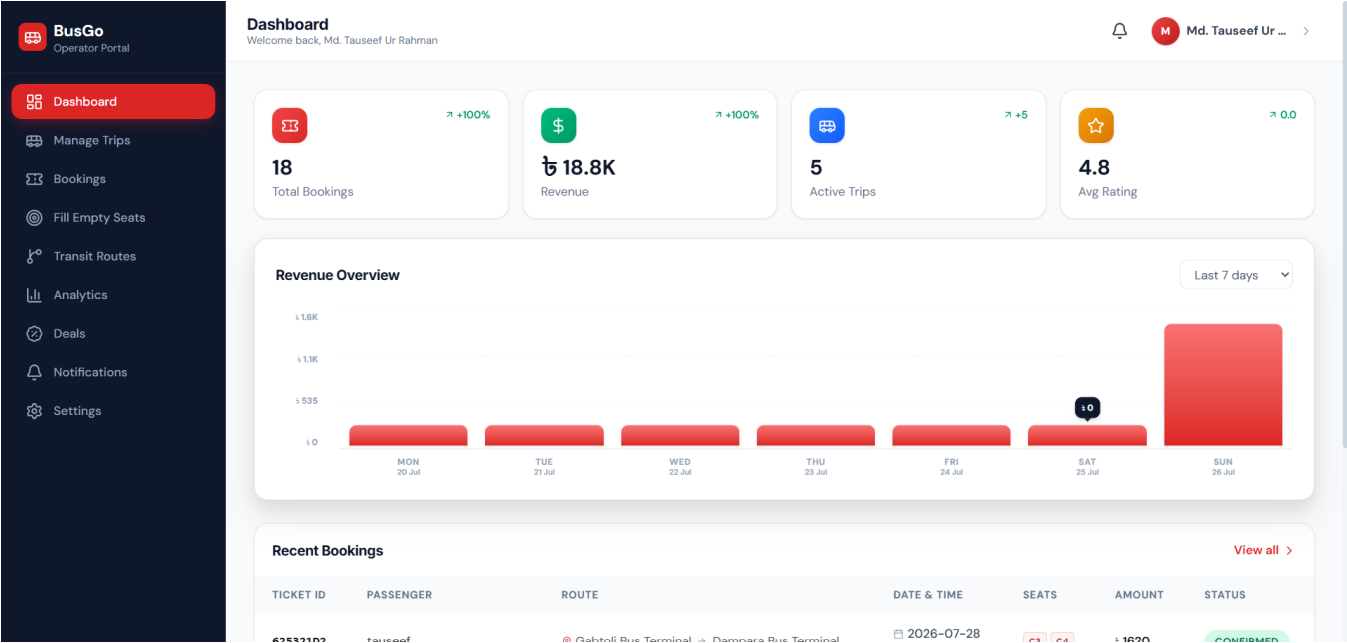


Figure B1. Operator dashboard with booking, revenue, trip, and rating summaries.

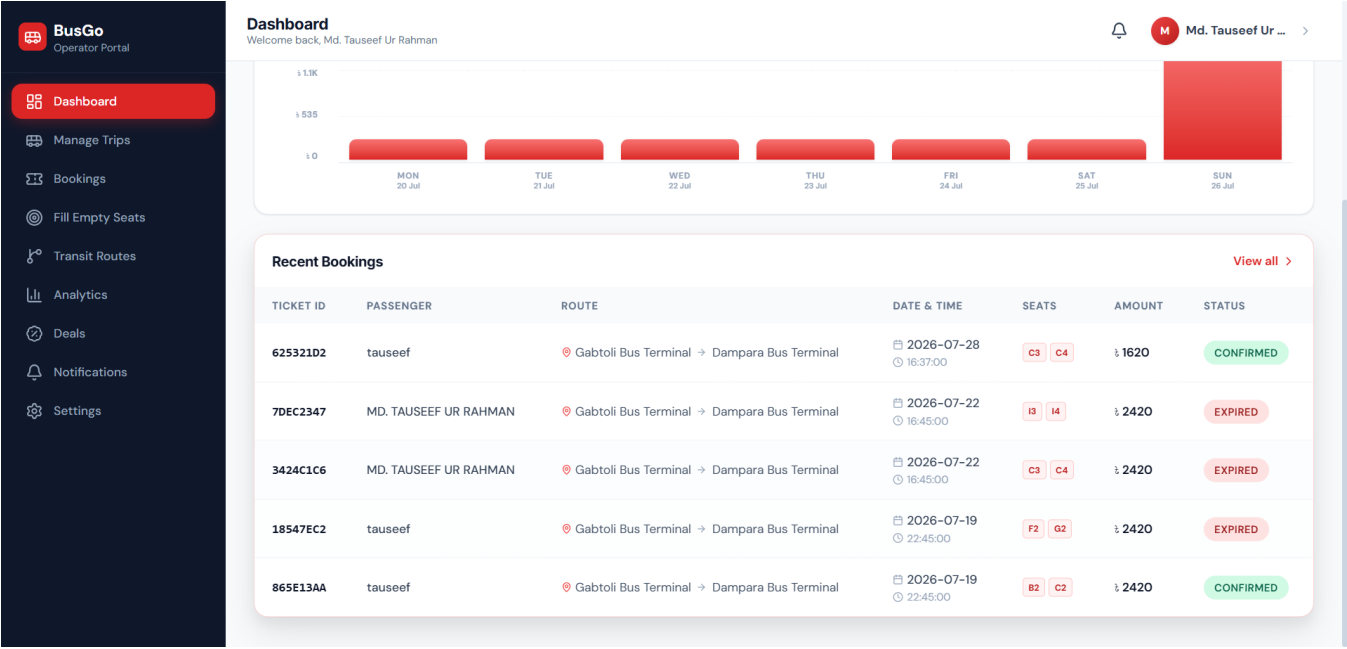


Figure B2. Operator dashboard analytics and recent-booking activity.

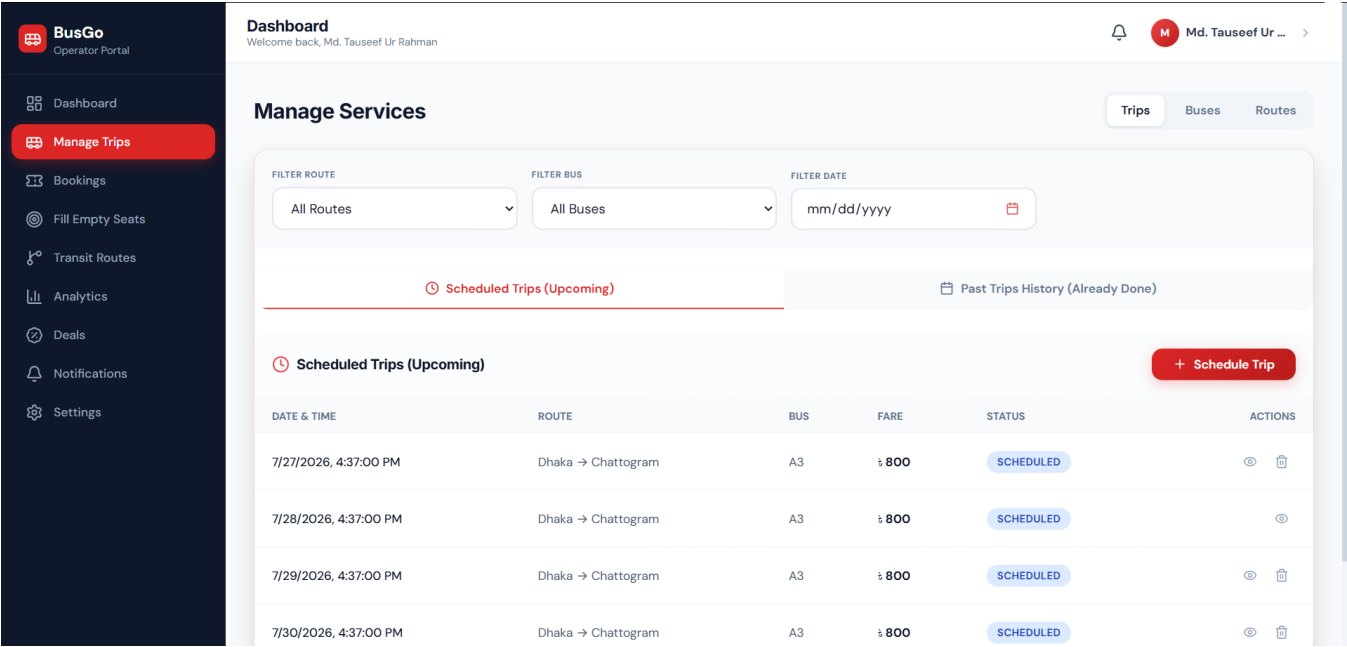


Figure B3. Trip management and scheduling controls.

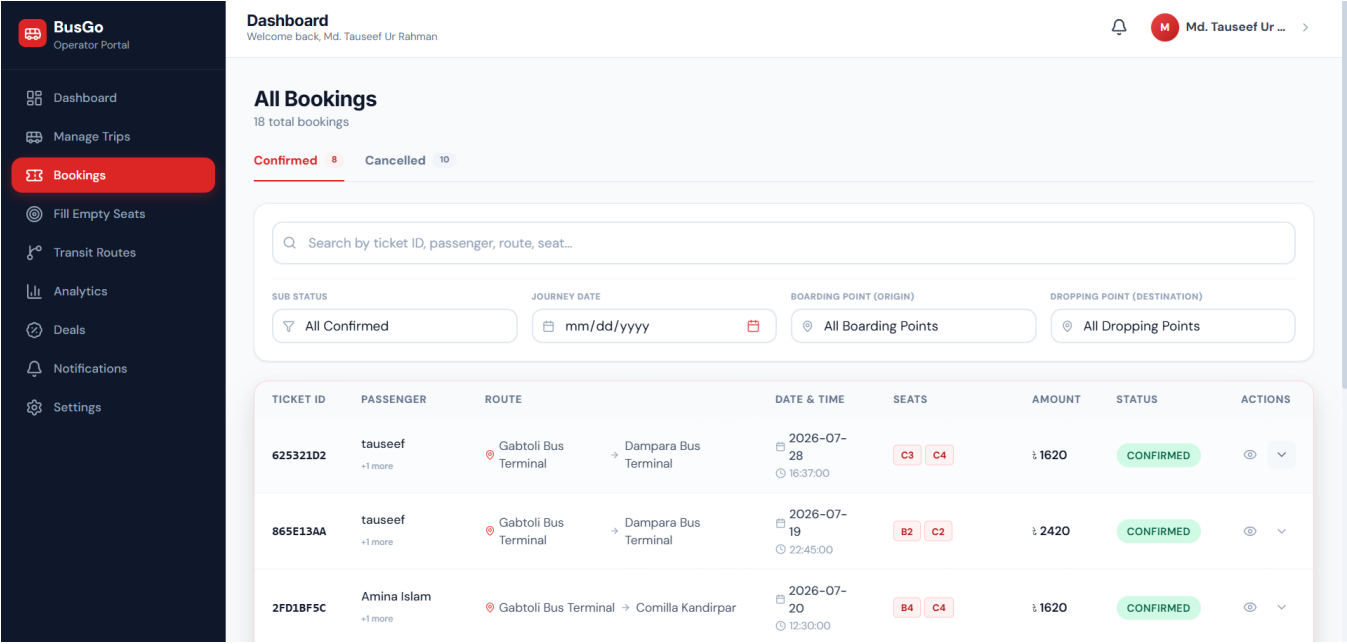


Figure B4. Operator booking list and passenger-manifest filters.

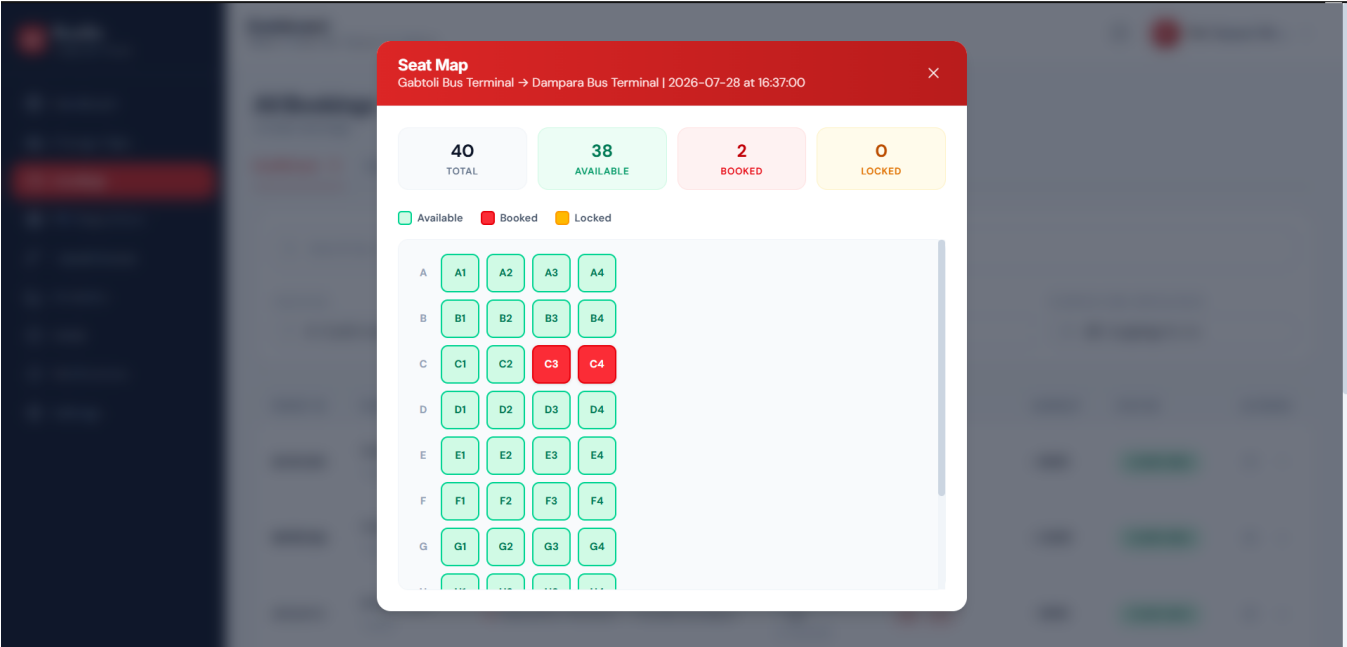


Figure B5. Trip seat map showing booked, available, and held seats.

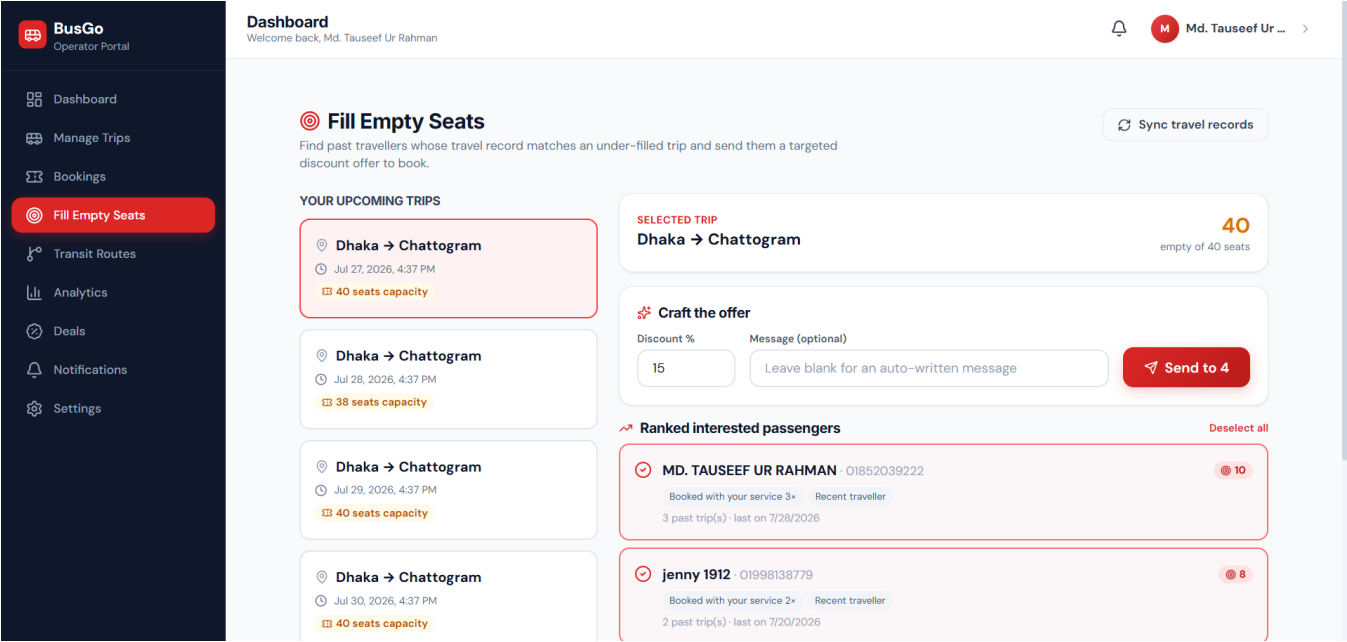


Figure B6. Fill Empty Seats workflow with matched passengers and notification action.

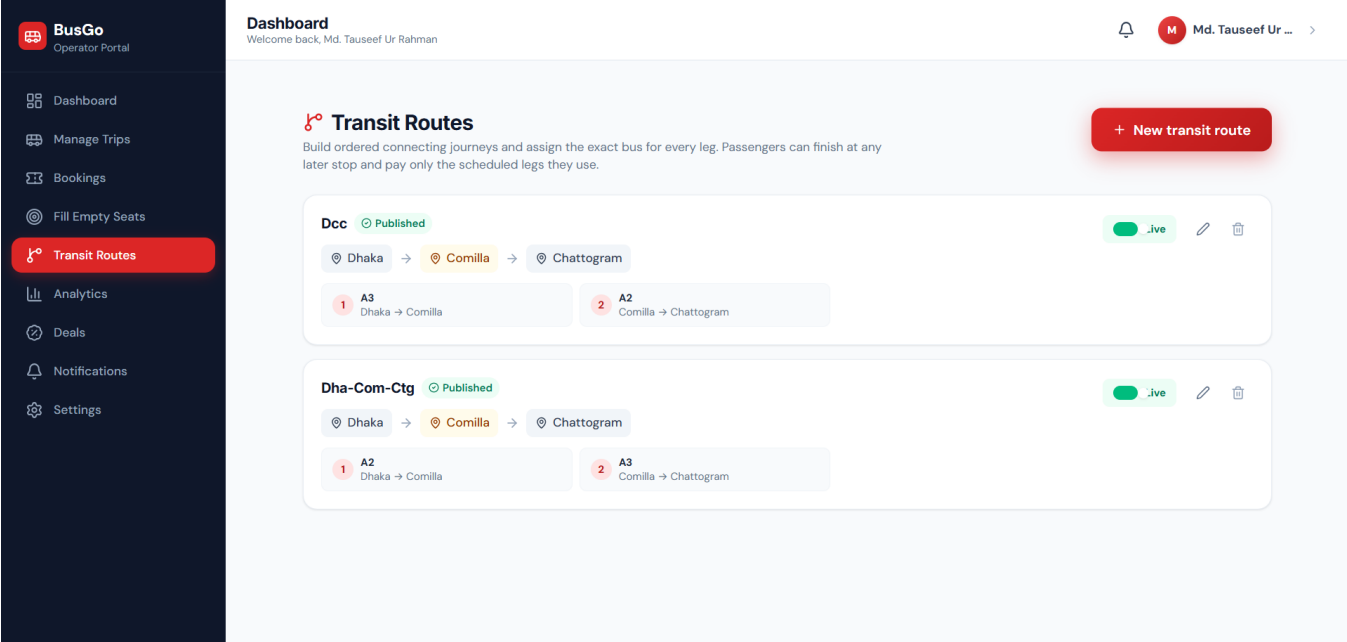


Figure B7. Curated transit-route management.

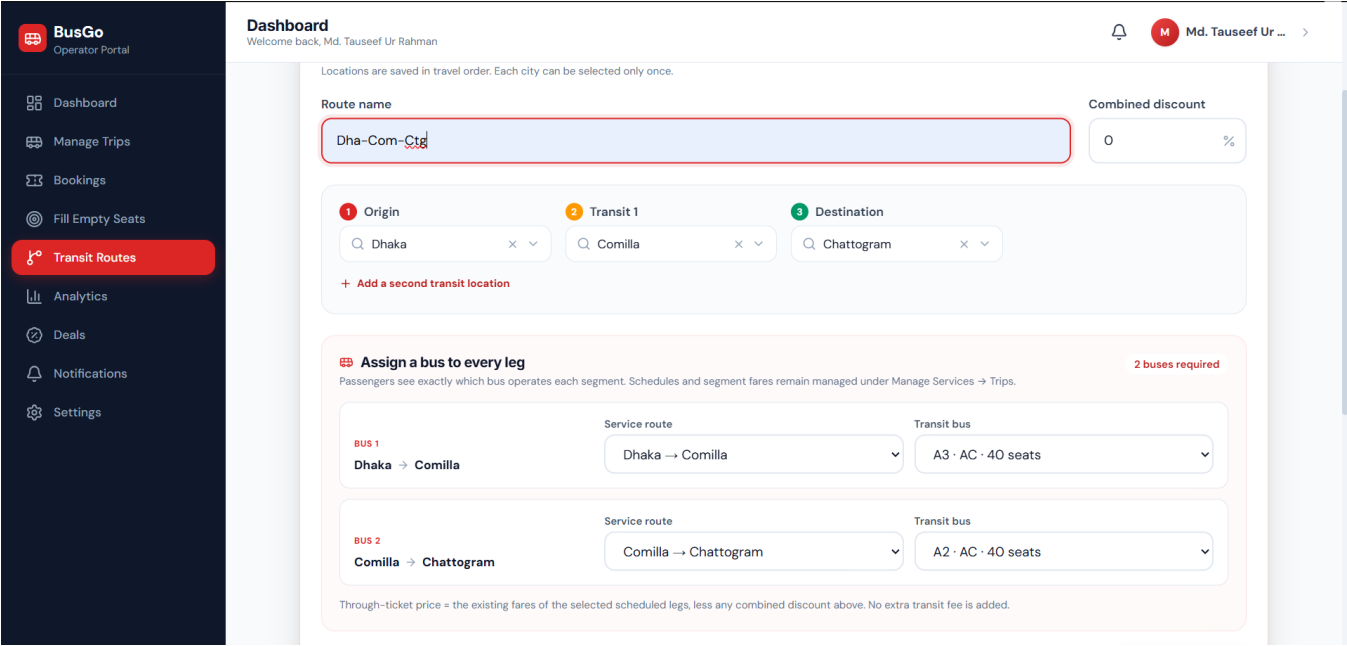


Figure B8. Transit-route builder with via-city and leg assignment controls.

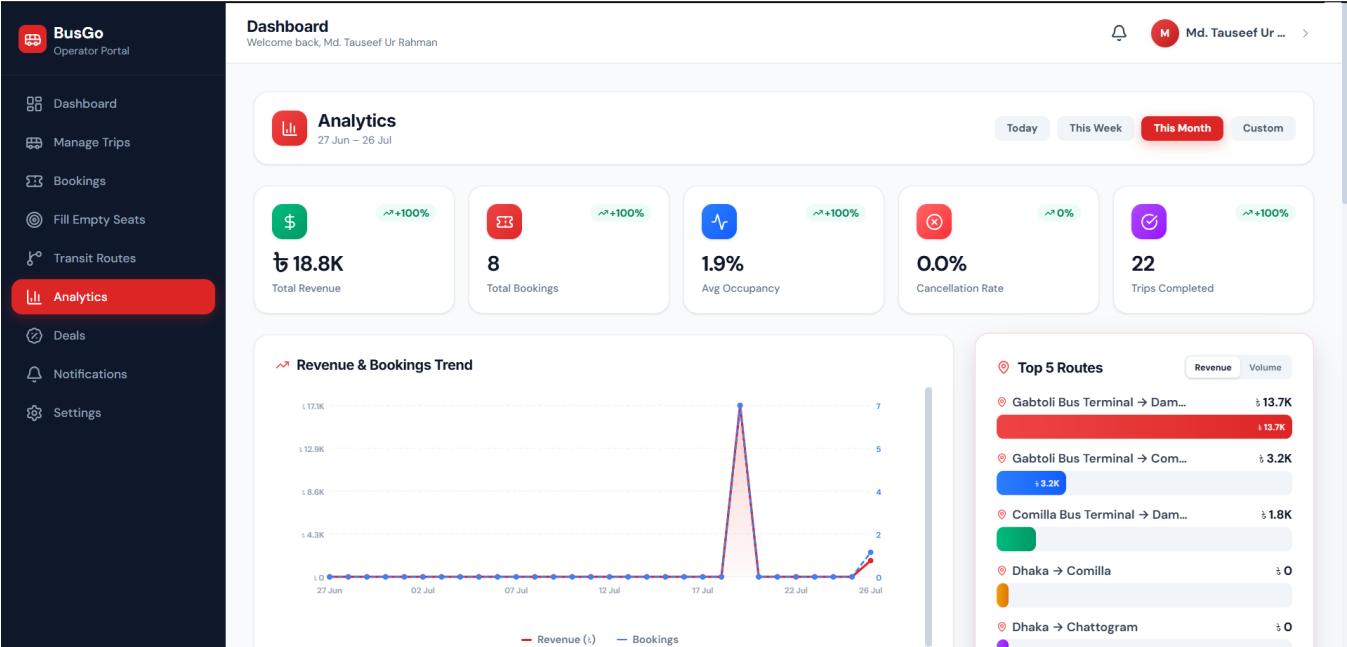


Figure B9. Revenue, booking, occupancy, and trip-status analytics.

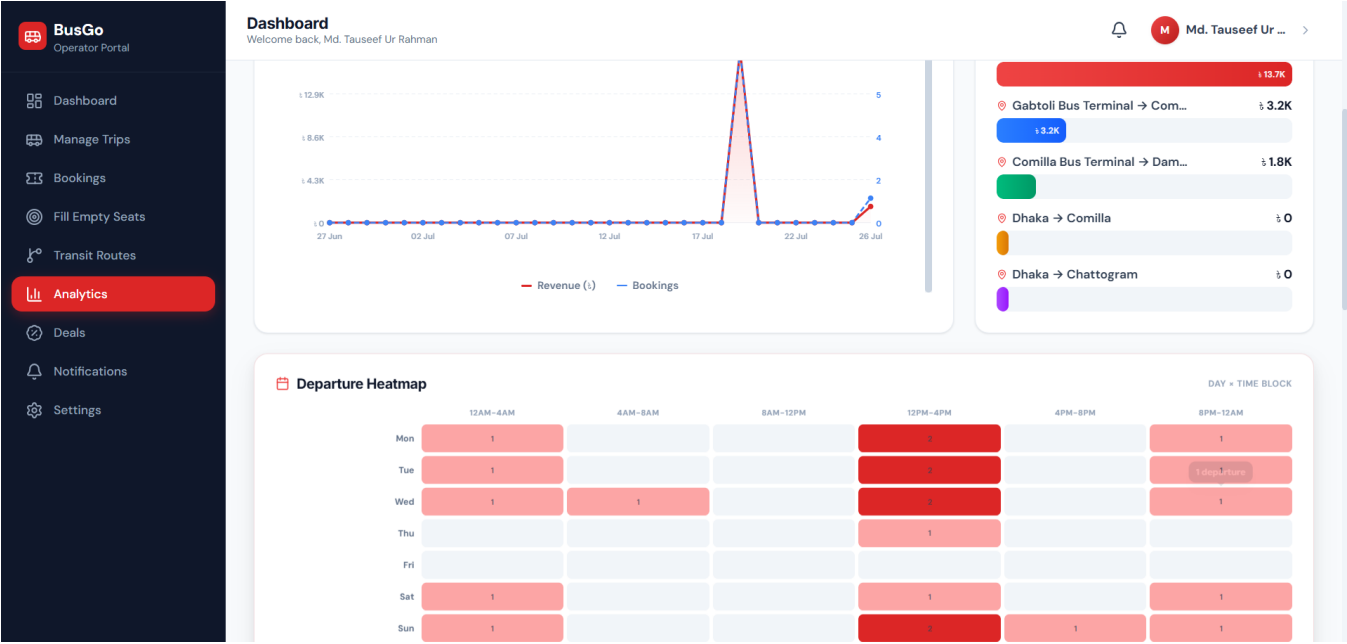


Figure B10. Analytics detail with revenue trend and departure heatmap.

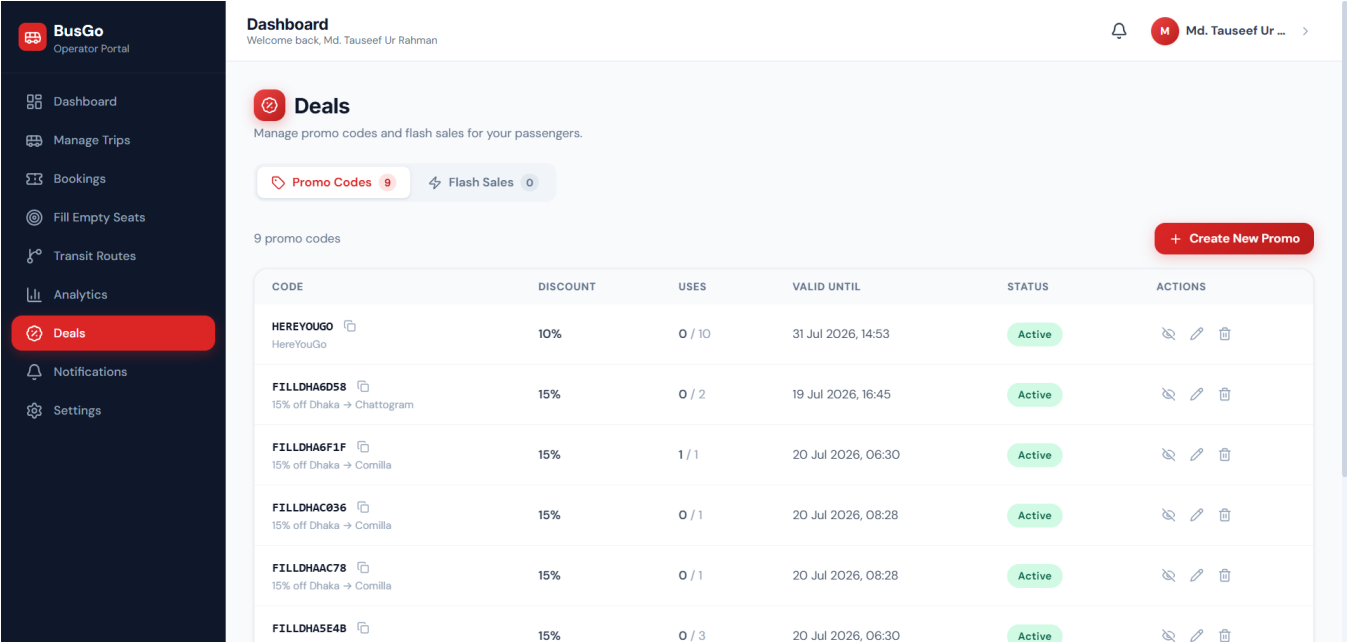


Figure B11. Operator promo-code and flash-sale management.

BusGo
Operator Portal

Dashboard
Welcome back, Md. Tauseef Ur Rahman

Settings
Manage your business information and public operator profile.

Public Profile

Operator Name (Visible to Customers) License Number

This name will be displayed in all search results, routes, and tickets.

Contact & Business Details

Contact Phone Contact Email

Business Address

Save Changes

<https://busgo.farefin.com/operator>

Figure B12. Operator profile and business settings.

C. Admin Portal (Web)

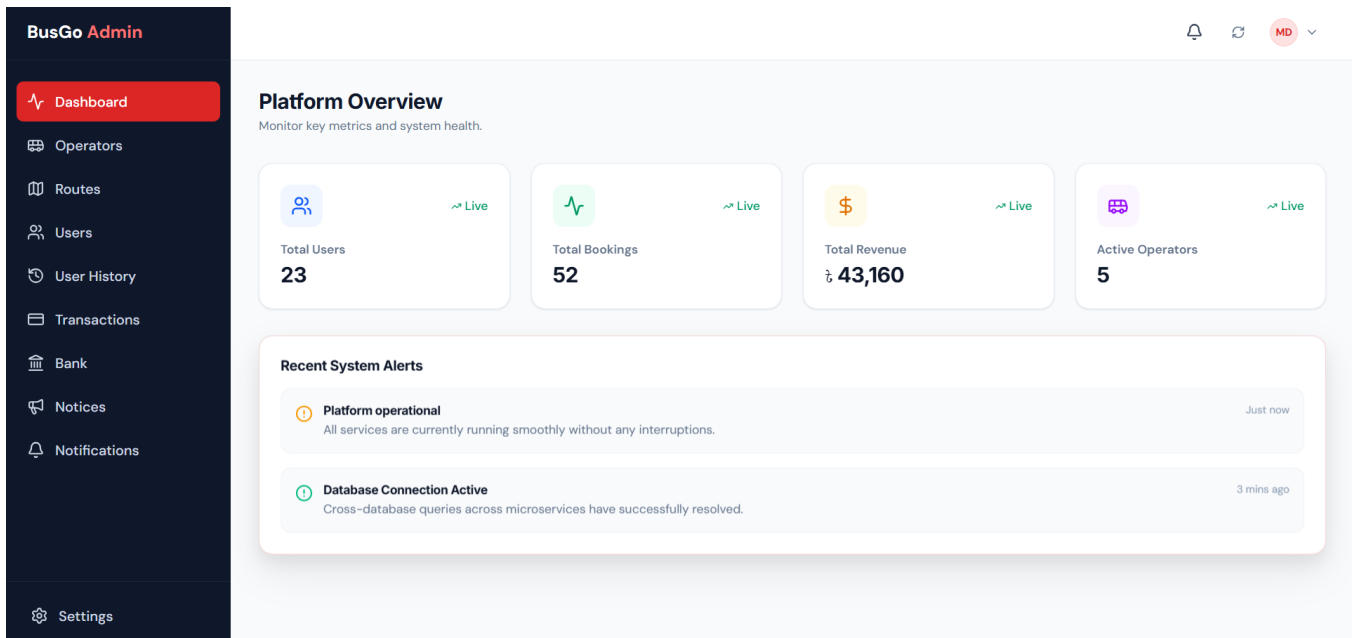


Figure C1. Admin platform overview with users, bookings, revenue, and operator counts.

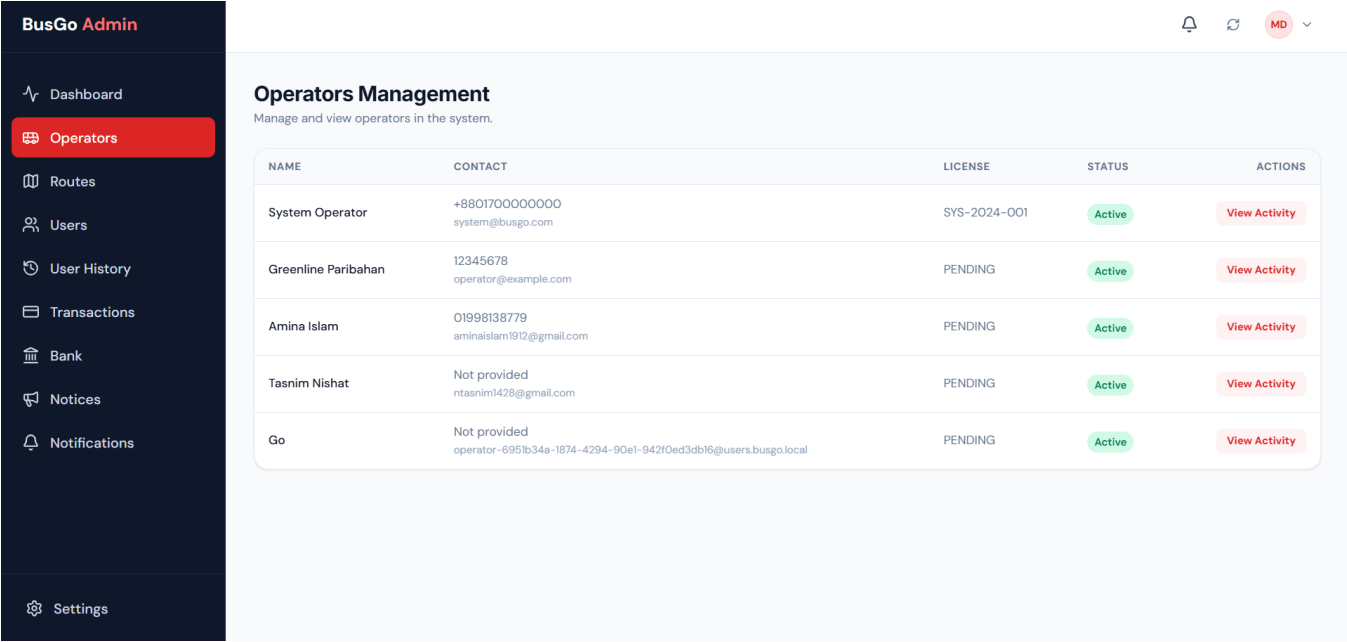


Figure C2. Operator management with license, status, and review actions.

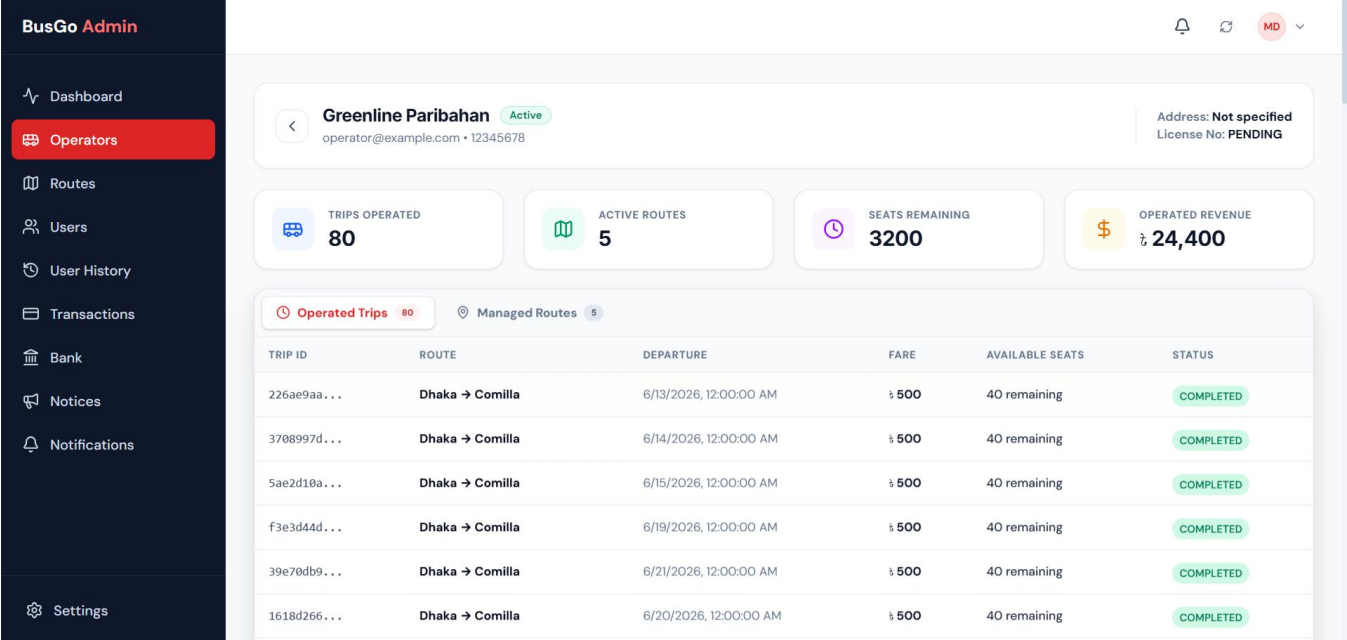


Figure C3. Operator detail with fleet, routes, trips, and revenue statistics.

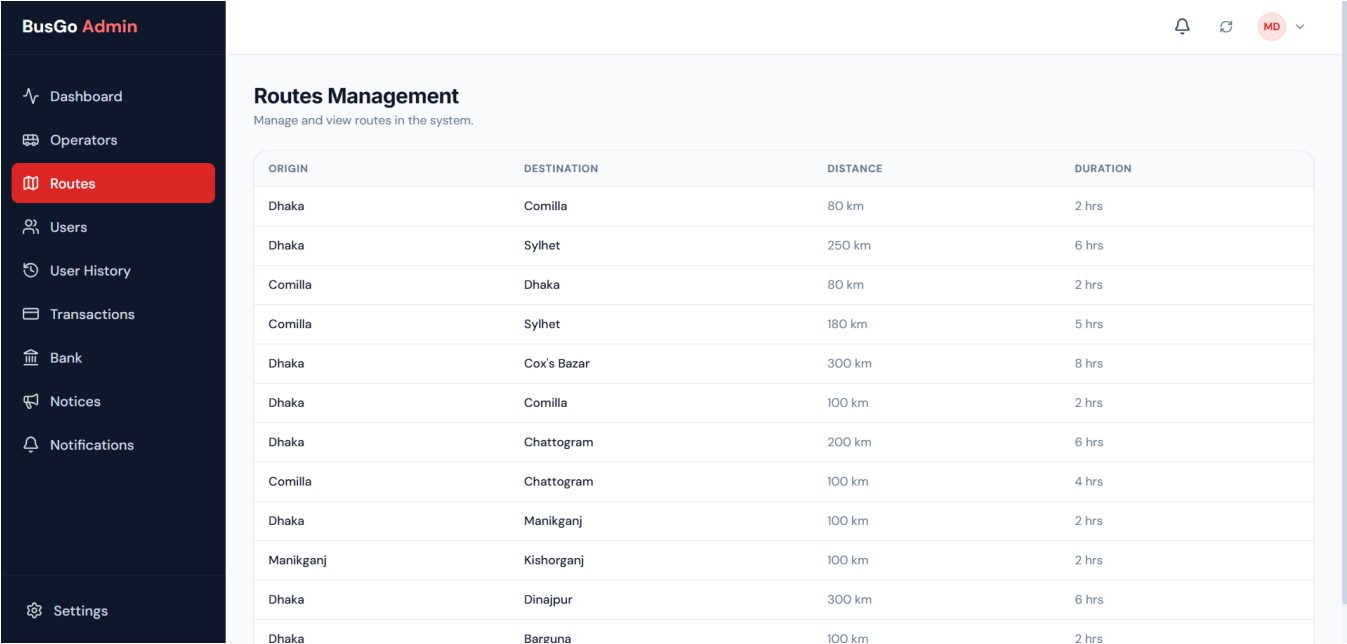


Figure C4. Platform route-management catalogue.

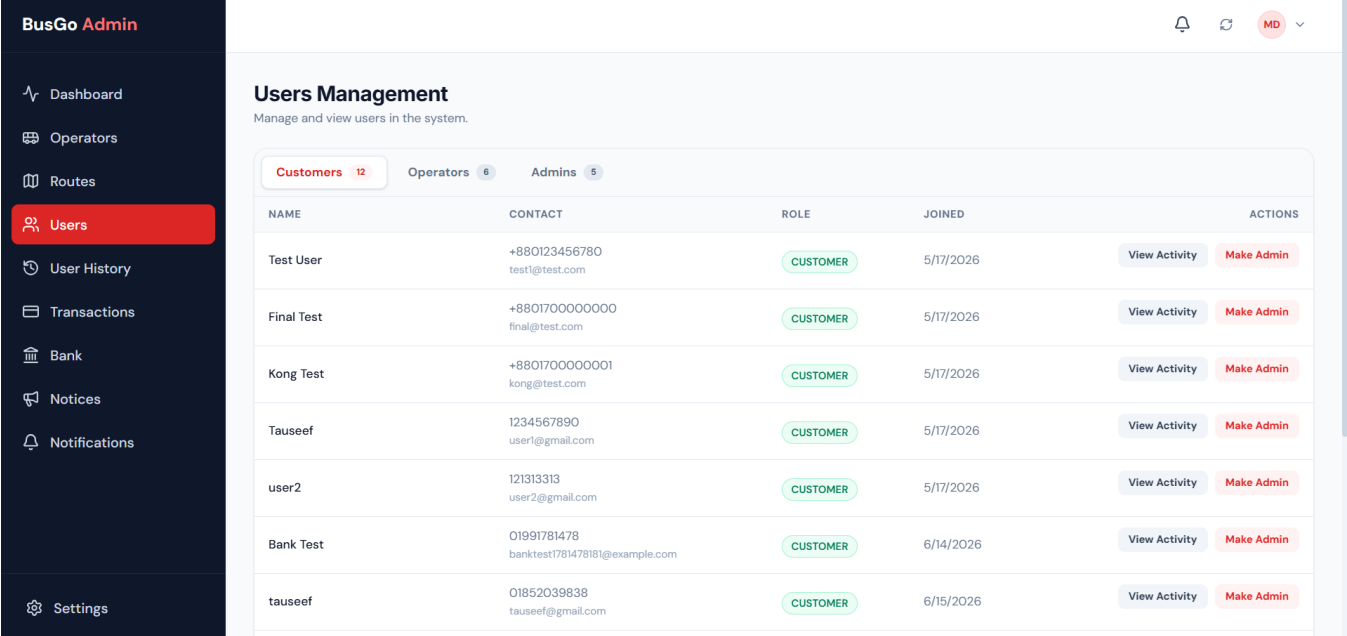


Figure C5. User management with role and account-status controls.

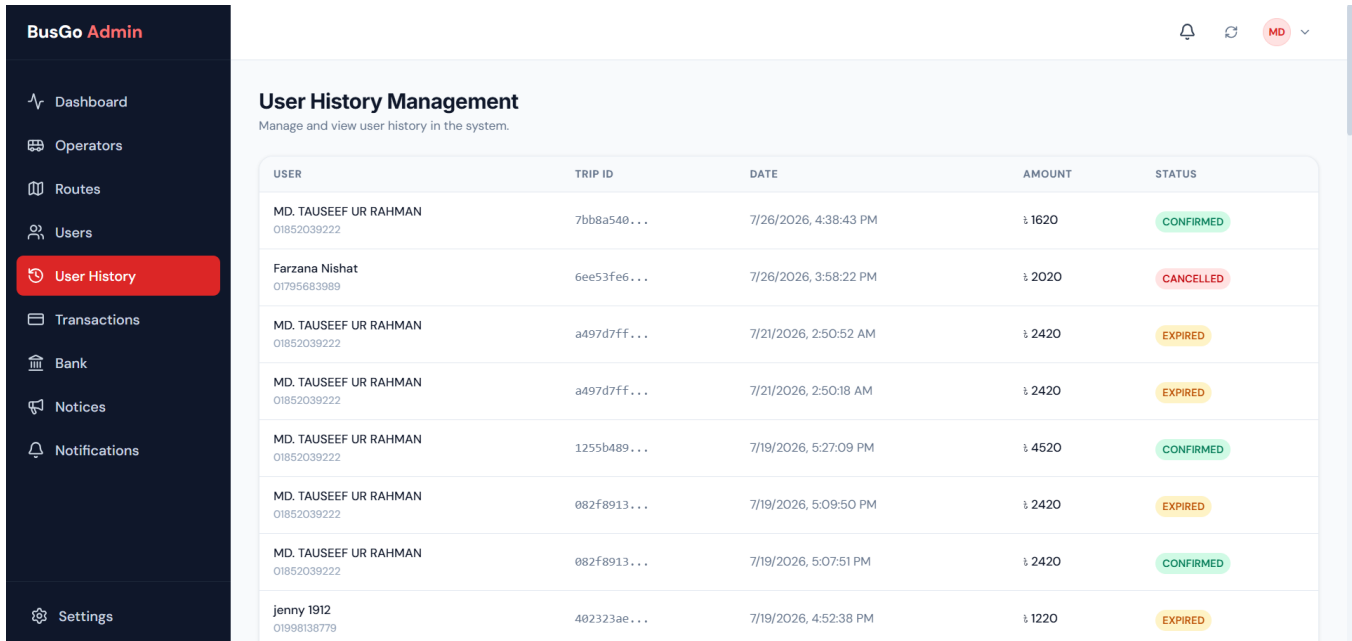


Figure C6. User travel-history management.

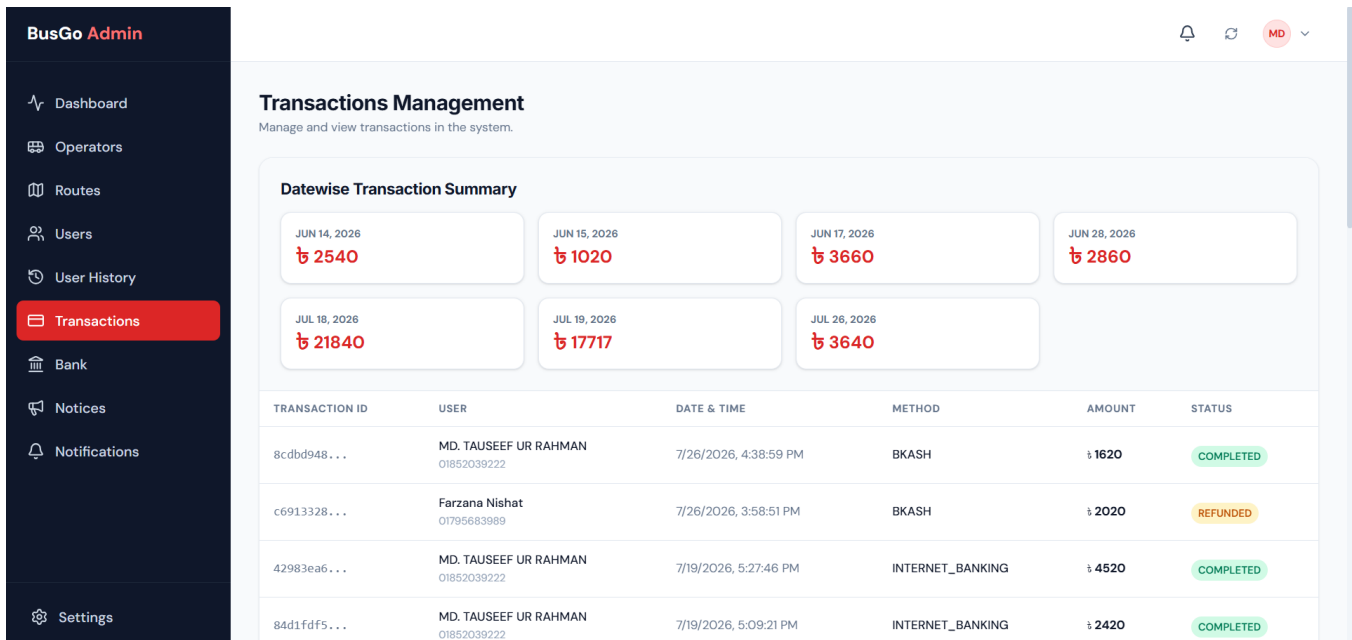


Figure C7. Transaction management and gateway-level totals.

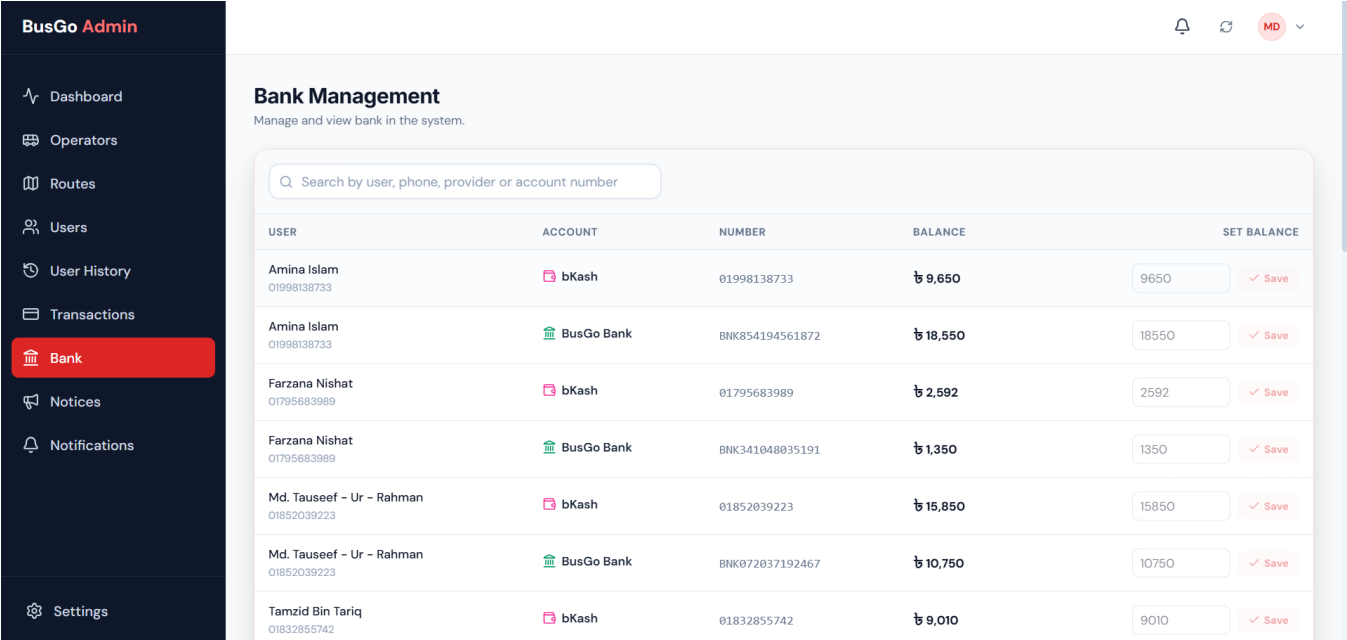


Figure C8. Simulated bank-account and balance management.

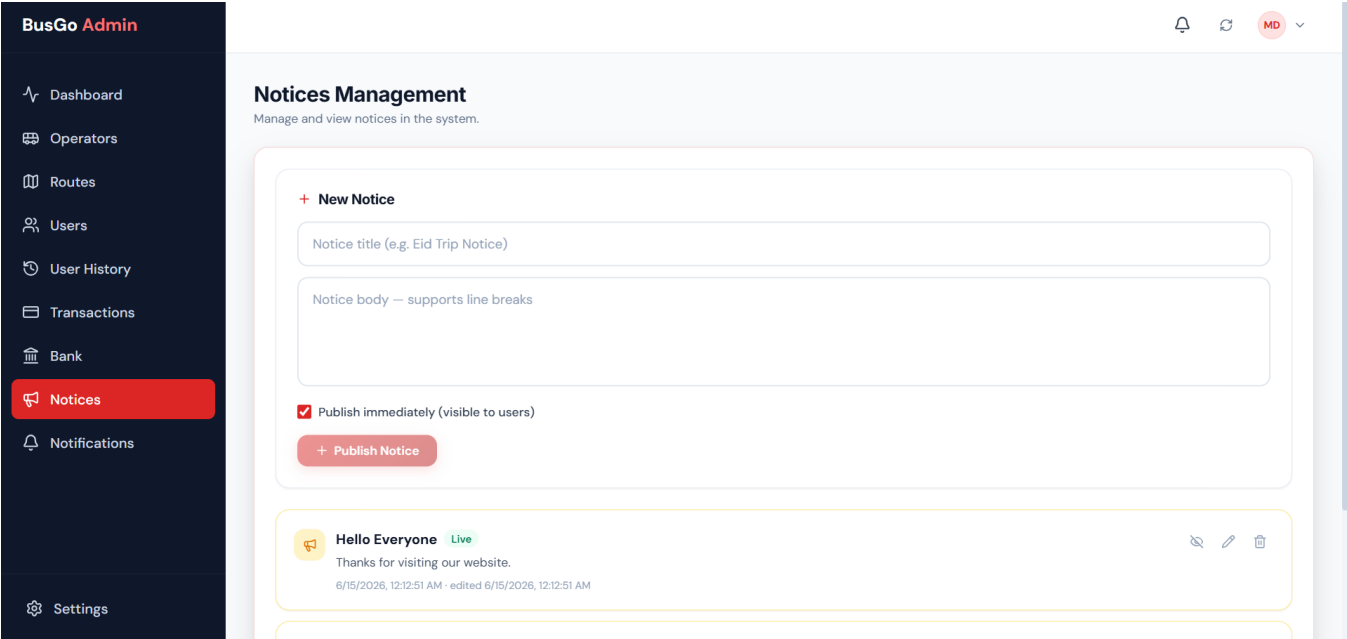


Figure C9. Platform notice publishing and visibility controls.

D. Mobile Applications Overview

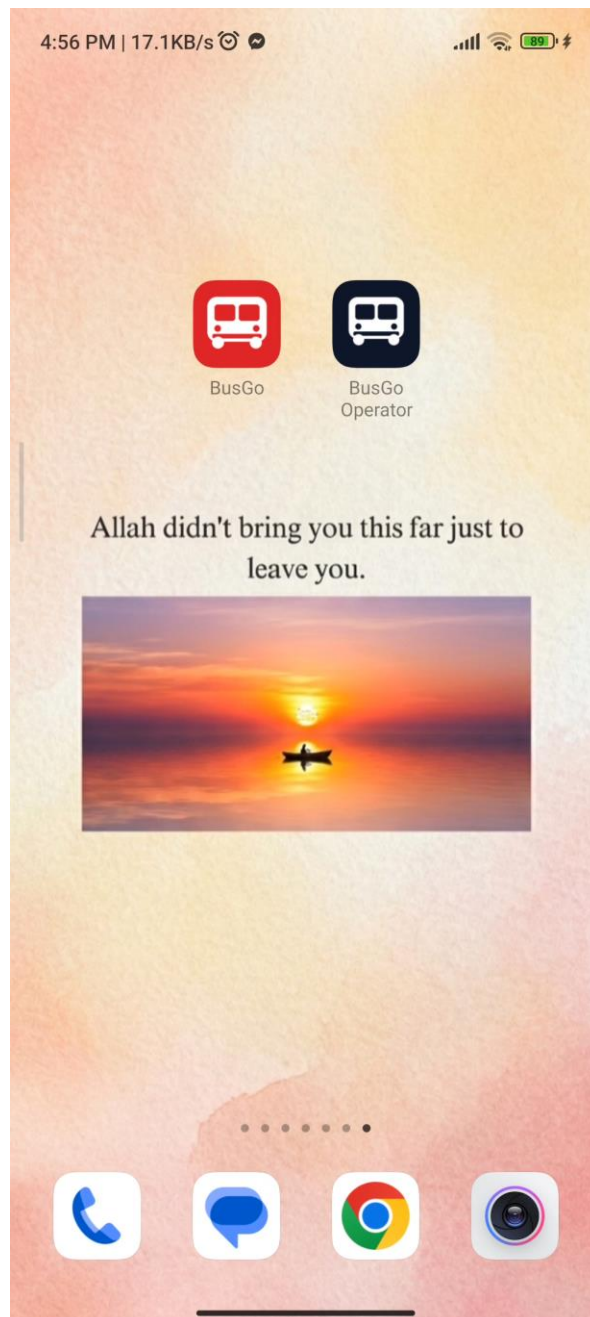


Figure D1. Android launcher view showing the BusGo passenger and BusGo Operator applications.

E. Passenger Mobile Application

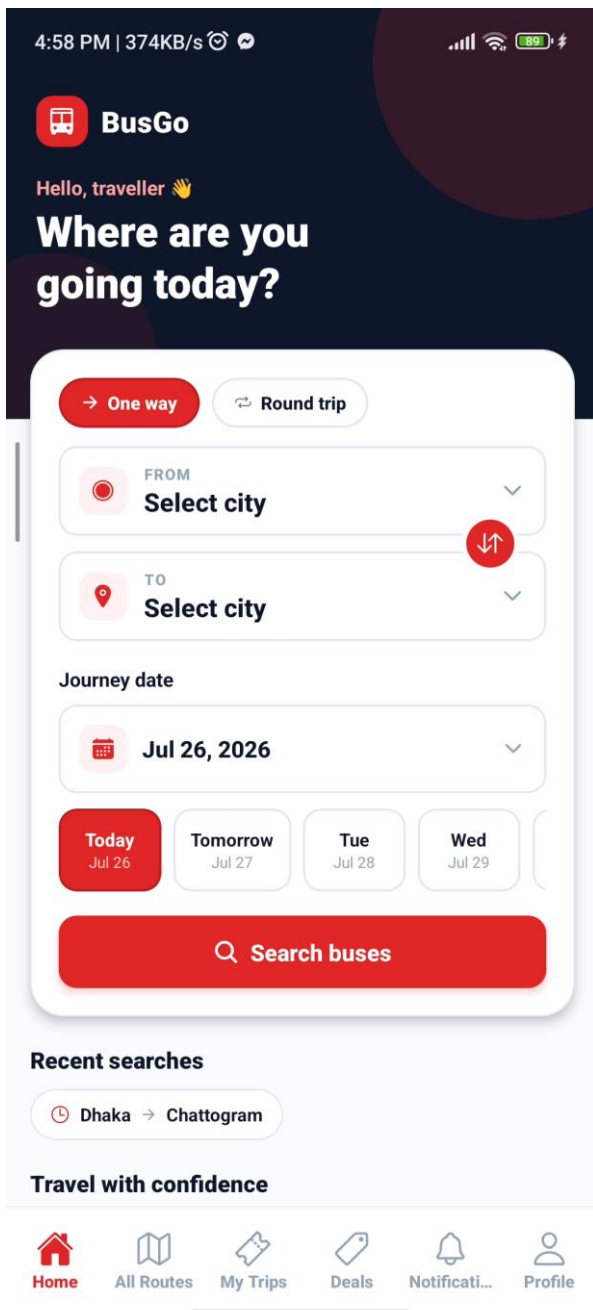


Figure E1. Passenger mobile home screen with route, date, and journey-type search controls.

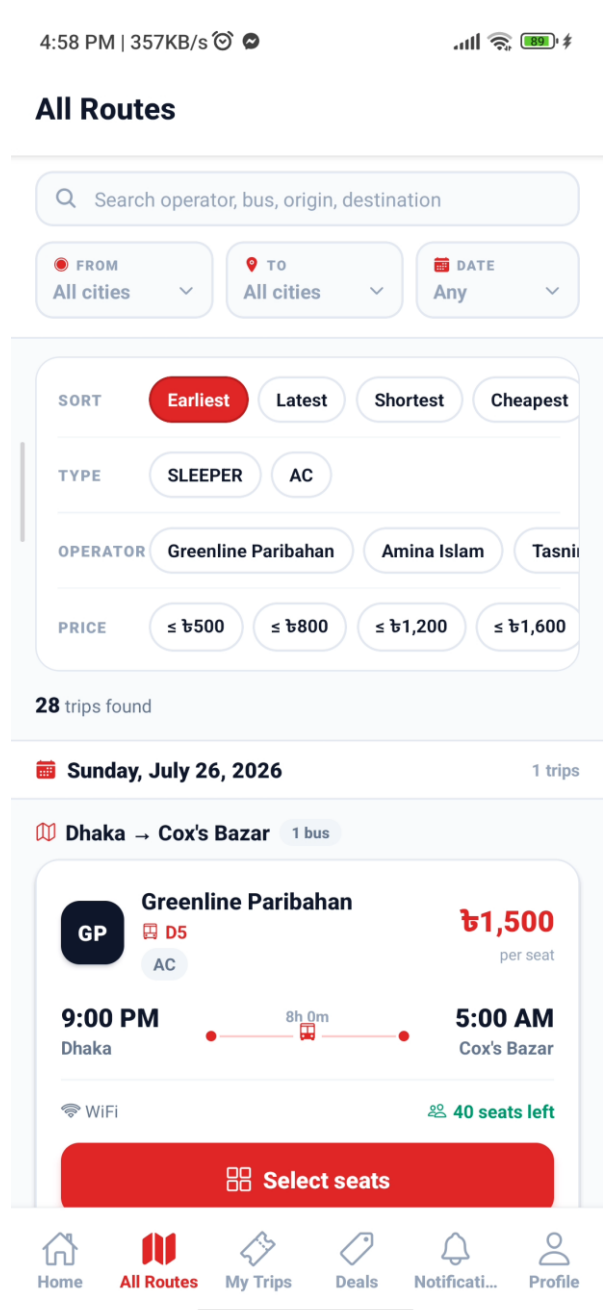


Figure E2. Mobile route catalogue with sorting, filters, fares, and seat availability.

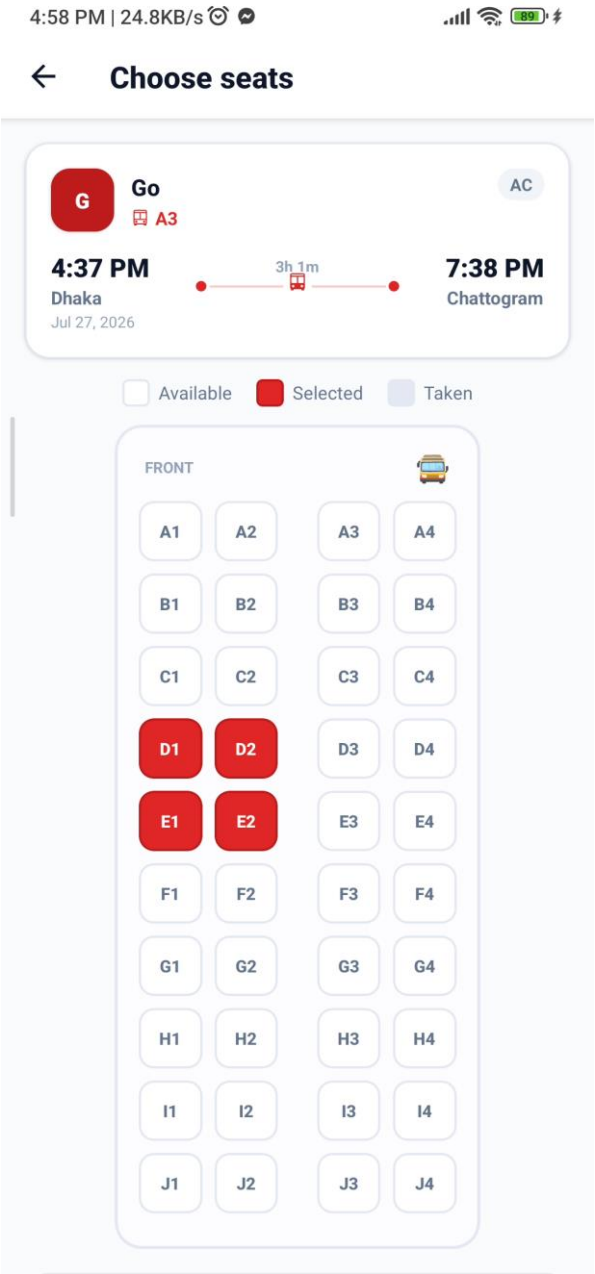


Figure E3. Seat-selection screen with trip timing and interactive seat map.

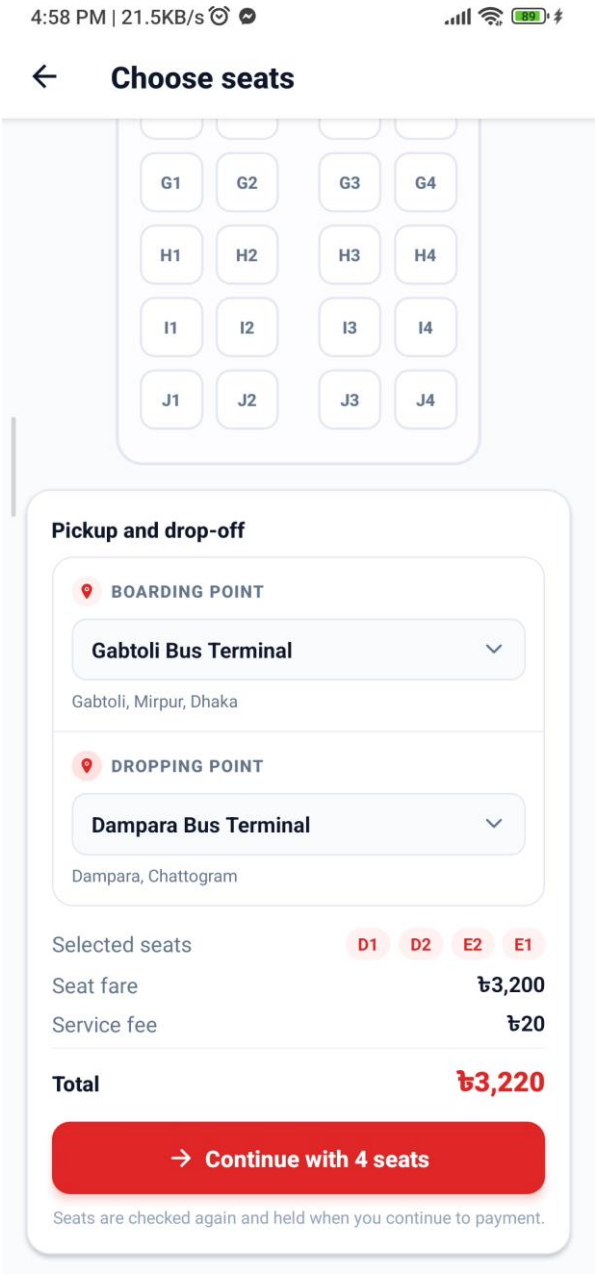


Figure E4. Pickup and drop-off selection with fare summary and checkout action.

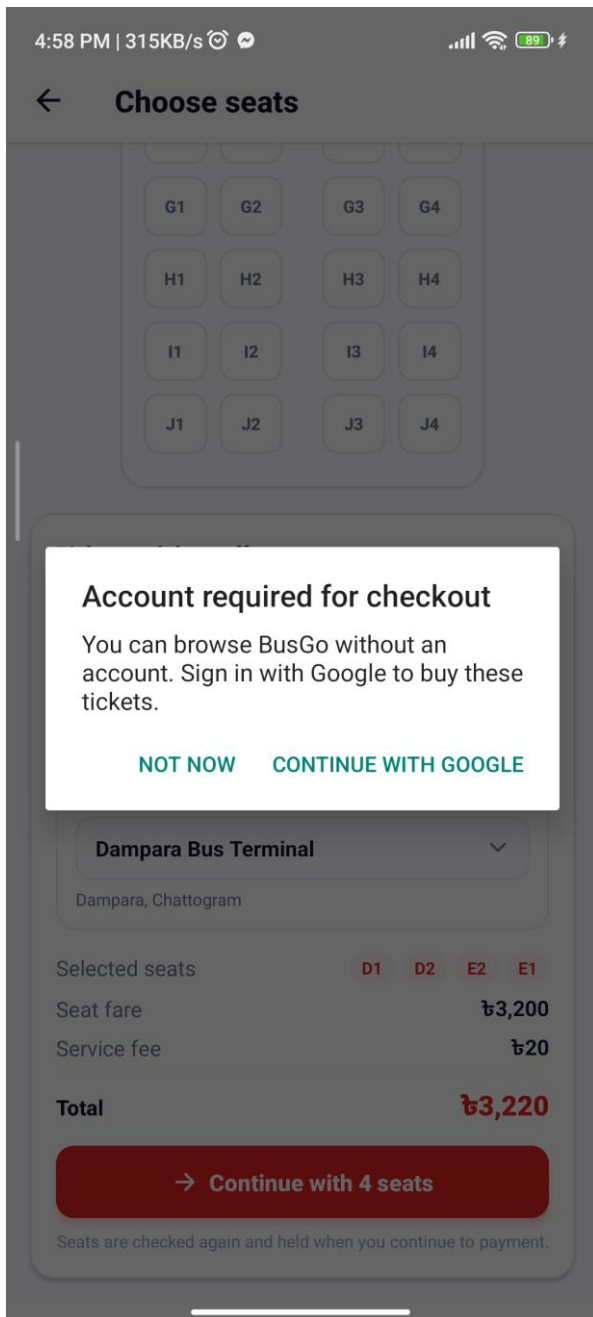


Figure E5. Account-required prompt displayed before mobile checkout.

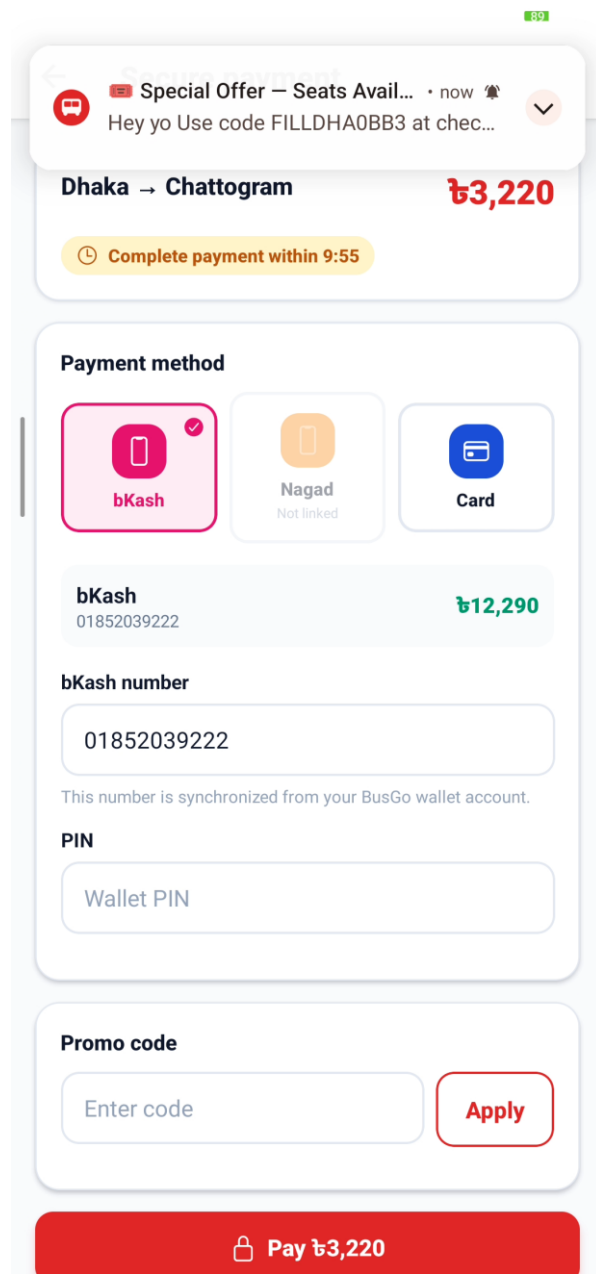
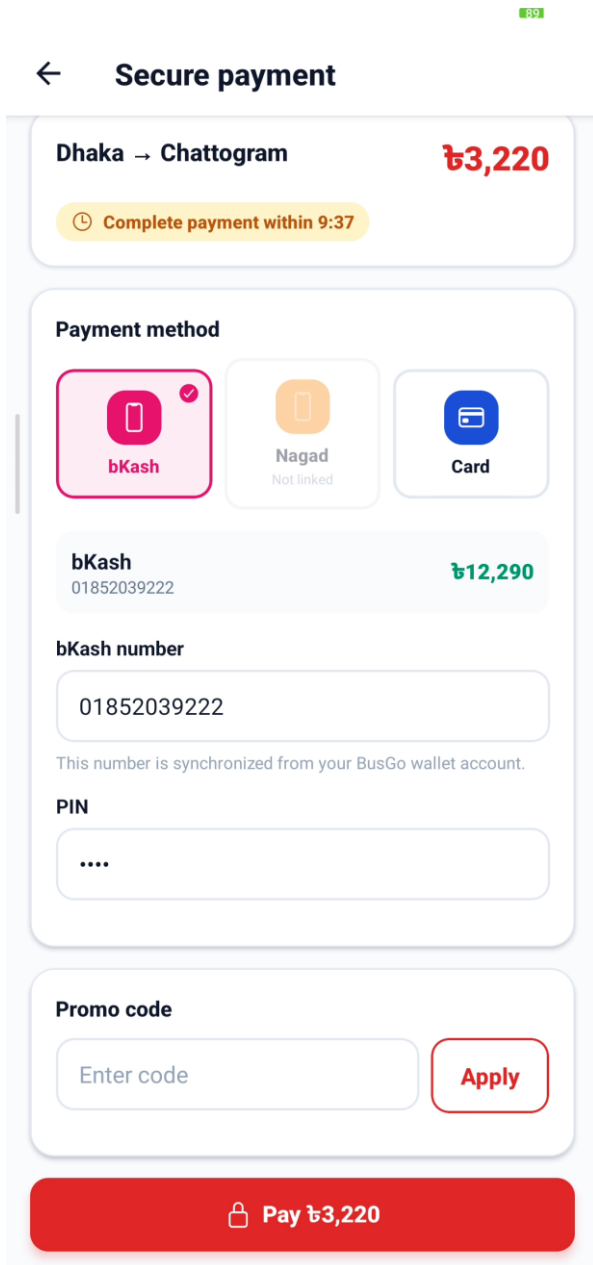


Figure E6. Mobile payment screen with wallet selection, promo-code entry, and booking countdown.

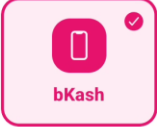
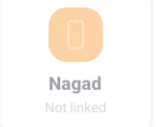
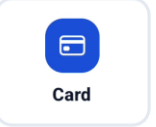


← **Secure payment**

Dhaka → Chattogram **₳3,220**

🕒 Complete payment within 9:37

Payment method

 **bKash**
 Nagad
Not linked
  Card

bKash
01852039222 **₳12,290**

bKash number

01852039222

This number is synchronized from your BusGo wallet account.

PIN

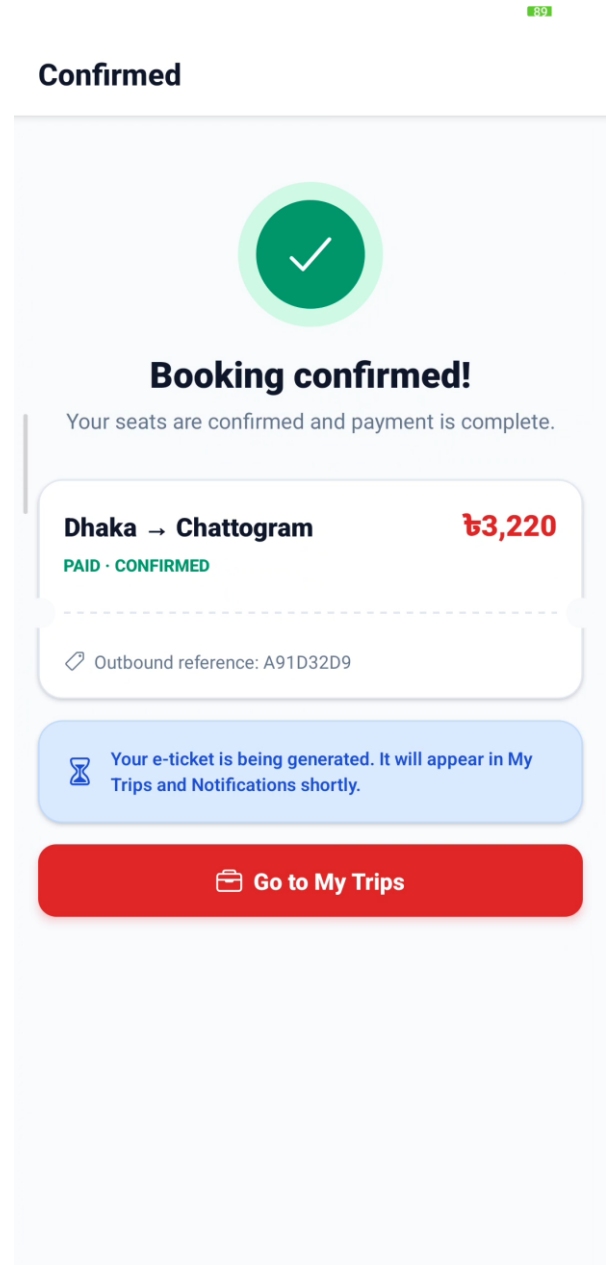
....

Promo code

Enter code **Apply**

Pay ₳3,220

Figure E7. Secure bKash payment form with synchronized account number and PIN entry.



Confirmed



Booking confirmed!

Your seats are confirmed and payment is complete.

Dhaka → Chattogram **₳3,220**
PAID · CONFIRMED

🔗 Outbound reference: A91D32D9

 Your e-ticket is being generated. It will appear in My Trips and Notifications shortly.

Go to My Trips

Figure E8. Mobile booking confirmation with payment status and trip reference.

5:00 PM | 71.6KB/s



My Trips

Bookings (14)

Tickets (0)

Upcoming 2

Completed 6

Cancelled 6

Search operator, city, or reference

G

Dhaka → Chattogram

CONFIRMED

Go

A3 AC

4:37 PM

Dhaka

Gabtolli Bus Terminal

3h 1m

7:38 PM

Chattogram

Dampara Bus Terminal

Seats D1, D2, E2, E1

Jul 27, 2026

WiFi

Ref A91D32D9

PAID

₹3,220

G

Dhaka → Chattogram

CONFIRMED

Go

A3 AC

4:37 PM

Dhaka

Gabtolli Bus Terminal

3h 1m

7:38 PM

Chattogram

Dampara Bus Terminal

Seats C3, C4

Jul 28, 2026

WiFi

Ref 625321D2

PAID

₹1,620

Home

All Routes

My Trips

Deals

Notificati...

Profile

Figure E9. My Trips view with booking-status filters and journey cards.

5:00 PM | 193KB/s



Deals

Deals and promo codes

Copy a code and apply it before payment.

Promo codes

15% OFF

1000 uses left

15% Off for Frequent Travellers

Loyal customers get 15% off on all routes. No minimum fare required.

TRAVEL15

No minimum fare

Until Jun 24, 2026

20% OFF

499 uses left

20% Off Any Route

Get 20% discount on any booking. Perfect for first-time travellers!

BUSGO20

Minimum ₹500

Until Jul 1, 2026

₹100 OFF

199 uses left

Flat 100 Off

Get a flat 100 taka discount on bookings over 800.

SAVE100

Home

All Routes

My Trips

Deals

Notificati...

Profile

Figure E10. Passenger deals screen with available promo codes and discount conditions.

5:00 PM | 85.2KB/s



Notifications

Notifications

1 unread

Read all



Special Offer — Seats Available!

Hey yo Use code FILLDHA0BB3 at checkout.

Code: FILLDHA0BB3

Jul 20, 2026, 8:53 PM

See deals >



Special Offer — Seats Available!

yougo Use code FILLDHA5E4B at checkout.

Code: FILLDHA5E4B

Jul 19, 2026, 10:41 AM

See deals >



Special Offer — Seats Available!

Seats just opened on Dhaka → Comilla! Enjoy 15% off. Book now before they're gone. Use code FILLDHA424D at checkout.

Code: FILLDHA424D

Jul 18, 2026, 8:01 PM

See deals >



Home



All Routes



My Trips



Deals



Notificati...



Profile

5:00 PM | 243KB/s



Profile



MD. TAUSEEF UR RAHMAN

CUSTOMER

Travel summary

8

Active

0

Completed

6

Cancelled

Wallet balance

৳11,440

Total paid (completed payments)

৳24,460

Travel history



Dhaka → Chattogram

Go · Jul 27, 2026

৳3,220

CONFIRMED



Dhaka → Chattogram

Go · Jul 28, 2026

৳1,620

CONFIRMED



Dhaka → Chattogram

Go · Jul 22, 2026

৳2,420

EXPIRED



Dhaka → Chattogram

Go · Jul 22, 2026

৳2,420

EXPIRED



Home



All Routes



My Trips



Deals



Notificati...



Profile

Figure E11. Passenger notifications showing targeted seat offers and promo codes.

Figure E12. Passenger profile with wallet balance, travel summary, and journey history.

F. Operator Mobile Application

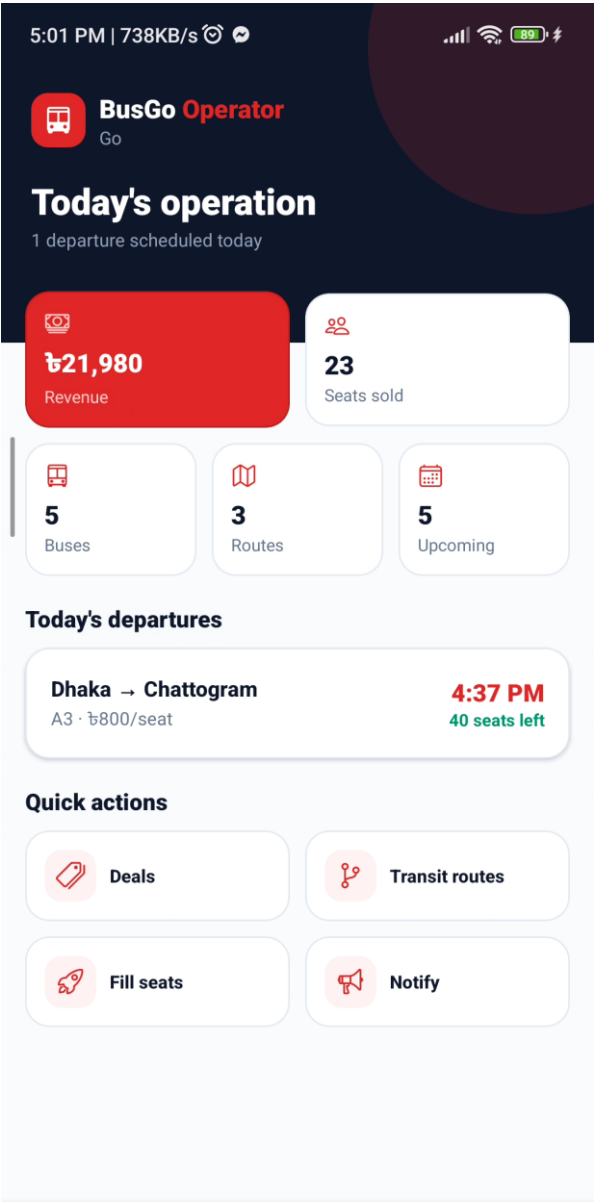


Figure F1. Operator mobile dashboard with daily revenue, seat sales, departures, and quick actions.

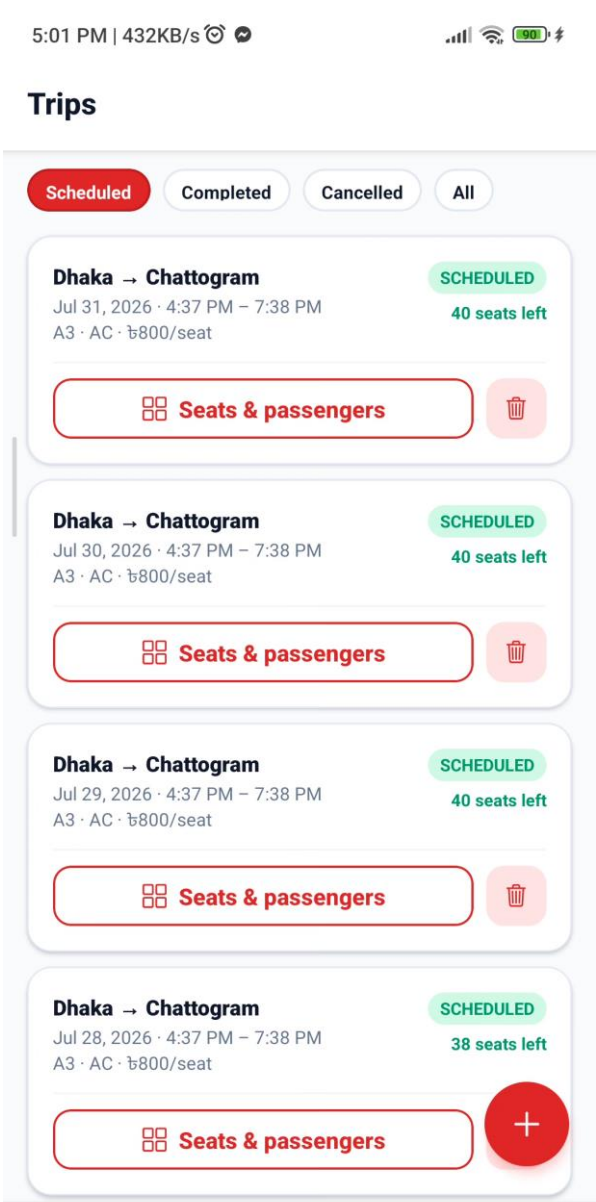


Figure F2. Mobile trip-management view with status filters and seat-manifest actions.

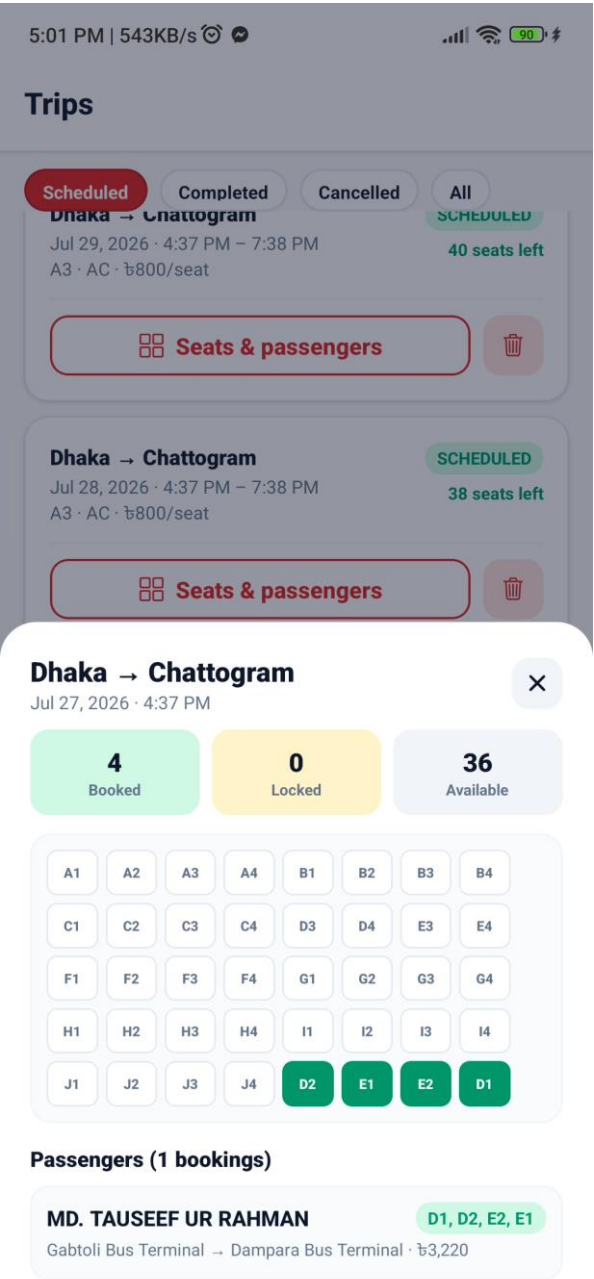


Figure F3. Trip seat map and passenger manifest for a scheduled departure.

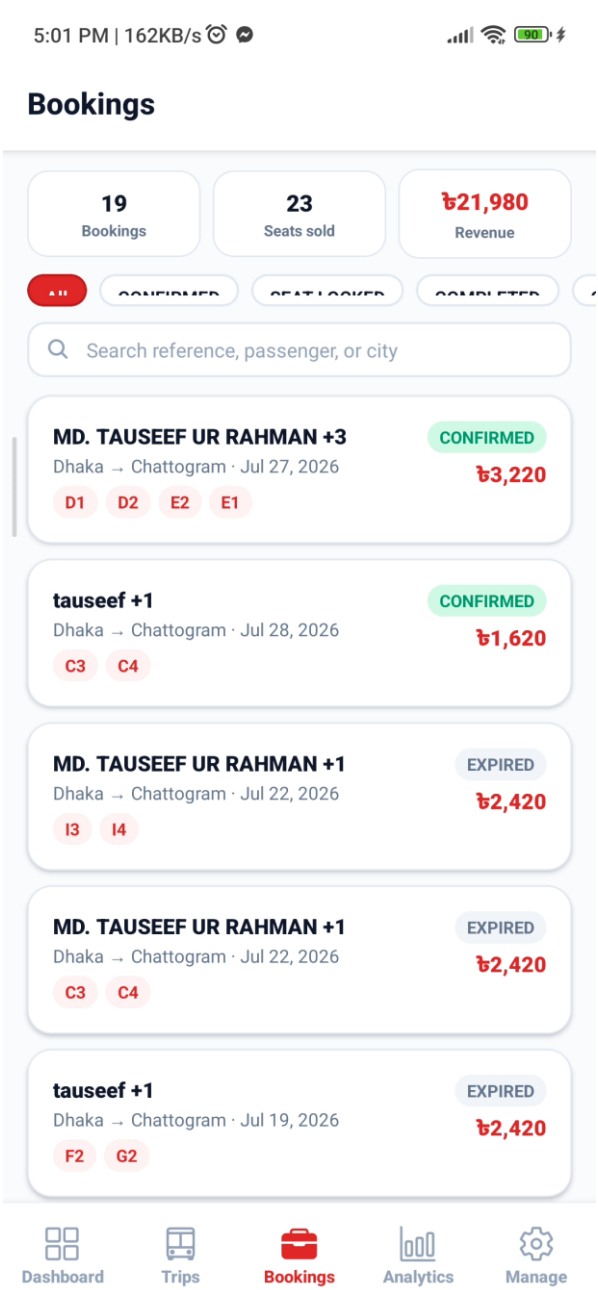
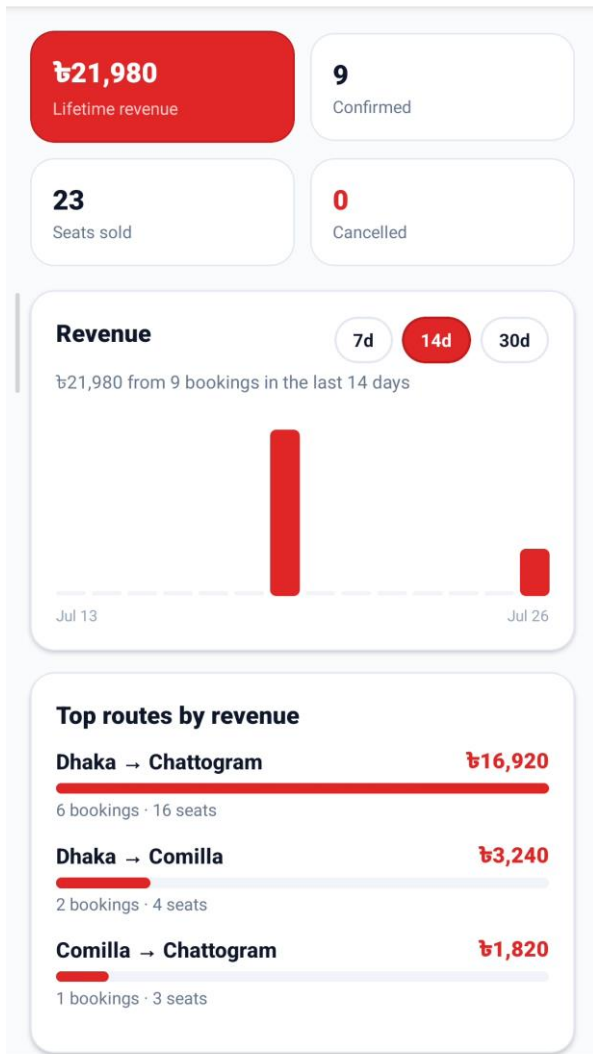


Figure F4. Operator booking list with search, status filters, revenue, and seat details.

5:02 PM | 176KB/s



Analytics



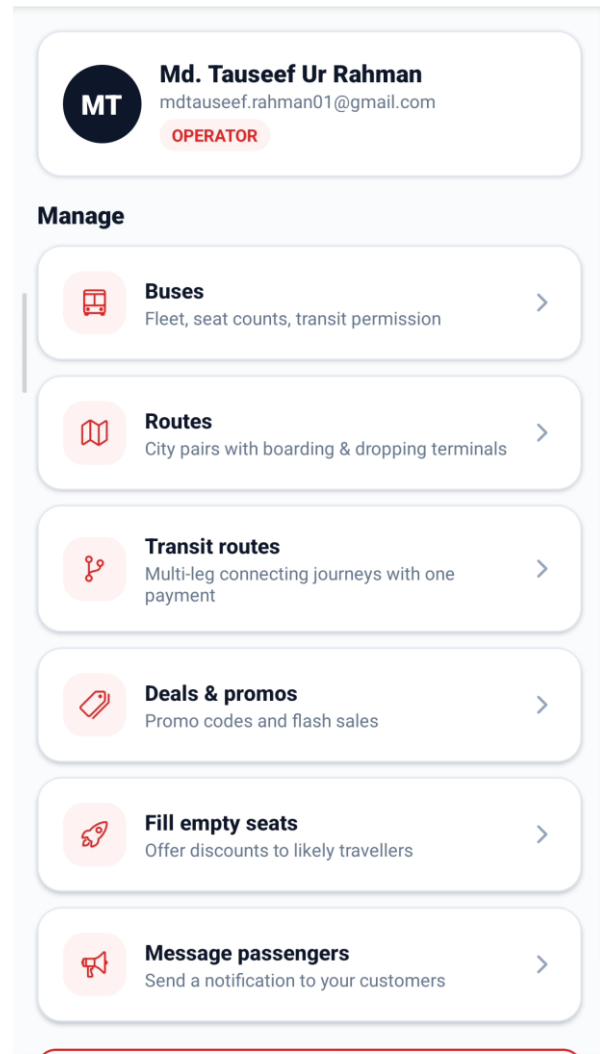
Dashboard Trips Bookings **Analytics** Manage

Figure F5. Mobile analytics dashboard with revenue period and route-performance summaries.

5:02 PM | 383KB/s



Manage



Dashboard Trips Bookings Analytics **Manage**

Figure F6. Operator management hub for buses, routes, promotions, seat filling, and messaging.

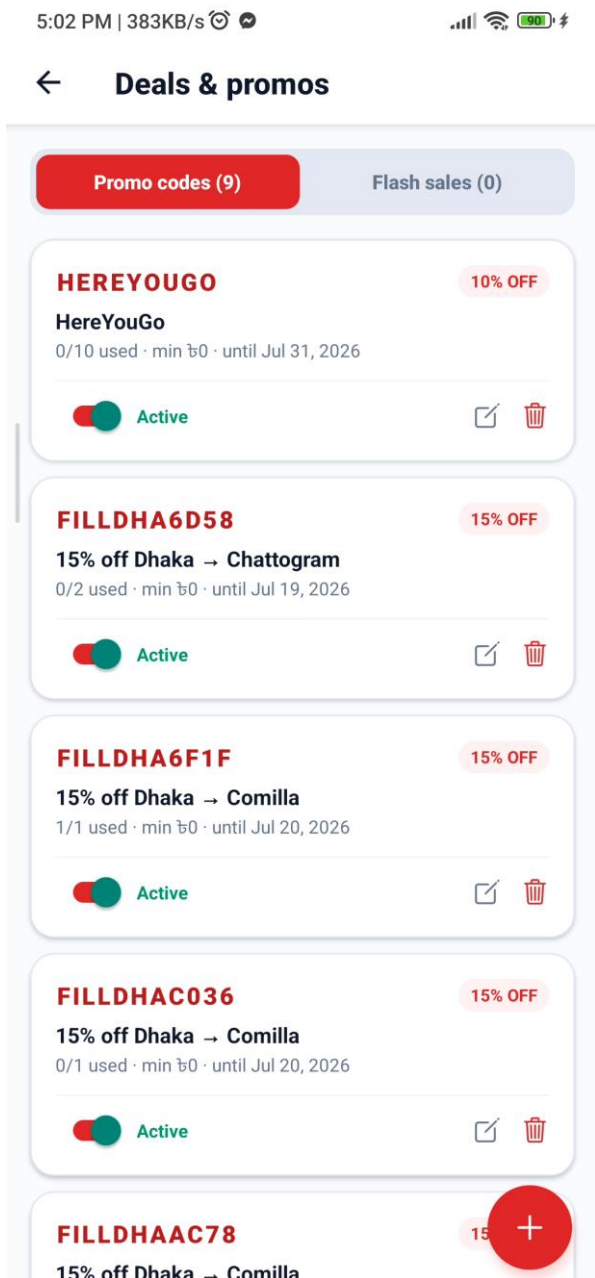


Figure F7. Mobile deals-and-promotions management with promo status and edit controls.

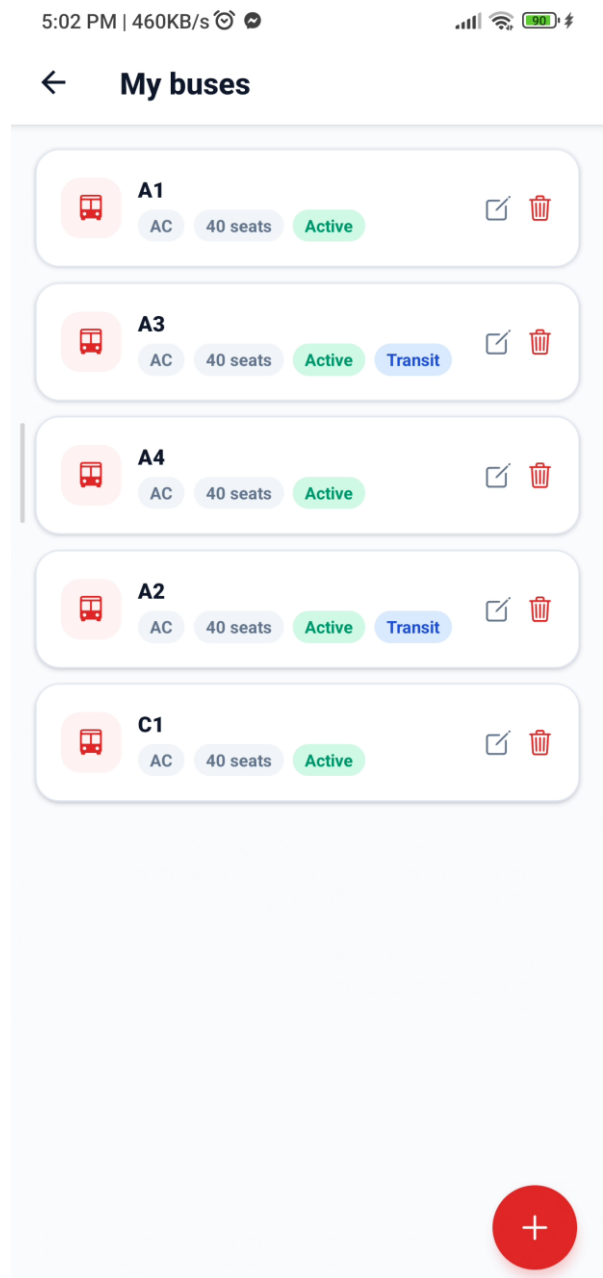


Figure F8. Operator fleet-management screen with bus status and transit assignments.

5:03 PM | 267KB/s

← **My routes**

Dhaka → Comilla

100 km · ~2h · 2 board · 2 drop

Add a route ✕

Origin city: Dhaka

Destination city: Chattogram

Distance (km): 100

Duration (hours): 4

📍 **BOARDING POINTS** + Add

Point 1 name (e.g. terminal)

Address (optional)

📍 **DROPPING POINTS** + Add

Point 1 name (e.g. terminal)

Address (optional)

✓ **Add route**

Figure F9. Mobile route-creation form with boarding and dropping-point controls.

5:03 PM | 231KB/s

← **Fill empty seats**

← **All trips**

Dhaka → Chattogram

Jul 27, 2026 · 4:37 PM

Candidates (3/3 selected) Clear all

✓ **a4c890a2** score 8
0 past trips on this route · last Jul 20, 2026

✓ **30e8b247** score 7
0 past trips on this route · last Jun 28, 2026

✓ **f0e5378e** score 6
0 past trips on this route · last Jul 20, 2026

Offer

10% 15% 20% 30%

Custom discount (%)

15

Message (optional)

Leave blank for an auto-written message

✈ **Send offer to 3 passengers**

Figure F10. Fill Empty Seats workflow with scored passenger candidates and discount selection.

5:04 PM | 275KB/s    

← **Message passengers**

Title

Schedule update from your operator

Message

Write the announcement your passengers will receive...

 **Send to 1 passenger**

Your passengers (3) Select all

☒

MD. TAUSEEF UR RAHMAN
Dhaka → Chattogram · last Jul 26, 2026

11 trips

☐

jenny 1912
Comilla → Chattogram · last Jul 19, 2026

7 trips

☐

Tamzid Bin Tariq
Dhaka → Chattogram · last Jul 19, 2026

1 trip

Figure F11. Targeted passenger-messaging form with recipient selection and announcement controls.